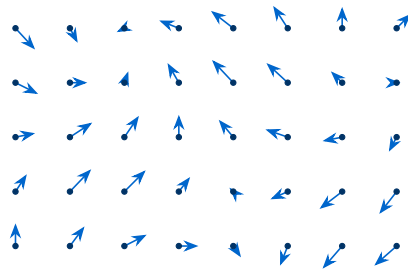


Optical Flow

From Pixels to Motion

A Complete Personal Textbook



“Motion is the language of vision. Optical flow is its grammar.”

Tools: Python, NumPy, OpenCV, PyTorch
Methods: Lucas-Kanade · Farnebäck · RAFT · FlowFormer
Datasets: KITTI, FlyingChairs, MPI-Sintel

May 31, 2026

Preface

The best way to understand something deeply is to derive it yourself, implement it yourself, and then teach it to someone else.

— Paraphrased wisdom from countless research labs

This textbook was written for one person: you — someone who has already built a Conformer for Nepali ASR, implemented LSTM and BiLSTM from scratch, trained CNNs and ResNets on real data, and worked with OpenCV on an autonomous vehicle project. You are not a beginner. You are a builder.

But you stand at an interesting threshold. You want to enter a 3D vision research lab, and you recognize that *optical flow* — the estimation of apparent motion between image frames — is the conceptual and technical bridge between the 2D world you know well and the 3D world you want to explore. This textbook is designed to take you across that bridge completely.

Every major concept follows the same structure: **Intuition first, Mathematics second, Code third, Analysis last**. Throughout the book you will find boxes labeled “Connection to Your Background” that explicitly link new concepts to things you have already implemented — from the correlation layer connecting to your understanding of convolution, to RAFT’s recurrent update operator connecting to your GRU/LSTM experience, to FlowFormer connecting to your Conformer architecture.

After reading this book, you will be able to implement a complete comparative study of classical and deep methods, explain every architectural decision at research level, and walk into a 3D vision lab interview discussing optical flow with genuine confidence.

Written for a builder. Refined for a researcher.

Contents

Preface	3
Notation and Symbols	9
1 Why Optical Flow	11
1.1 The Problem of Motion in Vision	11
1.2 Real-World Applications	12
1.2.1 Autonomous Driving	12
1.2.2 Video Compression	13
1.2.3 Action Recognition, Medical Imaging, and Video Stabilization .	13
1.3 Where Optical Flow Fits in the CV Pipeline	13
1.4 The Connection to 3D Vision	13
1.5 Connection to Your Autonomous Vehicle Project	14
2 Mathematical Foundation	17
2.1 The Image as a Mathematical Function	17
2.1.1 Spatial and Temporal Gradients	17
2.2 The Brightness Constancy Assumption	19
2.3 The Optical Flow Constraint Equation — Full Derivation	19
2.3.1 Geometric Interpretation	20
2.4 The Aperture Problem	20
2.5 Taylor Series Approximation	21
3 Sparse Optical Flow: Lucas-Kanade	25
3.1 The Core Idea	25
3.2 Least Squares Formulation — Full Derivation	25
3.3 Eigenvalue Analysis: When Does Lucas-Kanade Fail?	26
3.4 Connection to the Harris Corner Detector	26
3.5 Pyramidal Lucas-Kanade	27
3.6 Complete OpenCV Implementation	27
3.7 NumPy Implementation of Core LK Mathematics	30
3.8 Failure Case Analysis	32
4 Dense Optical Flow: Farnebäck’s Method	35
4.1 Why Sparse Flow Is Not Enough	35
4.2 Polynomial Expansion: The Key Idea	35
4.2.1 From Polynomial to Flow	36
4.3 OpenCV HSV Visualization Implementation	36
5 Horn-Schunck: Global Optical Flow	41

5.1	Local vs. Global Approaches	41
5.2	The Variational Formulation	41
5.3	Euler-Lagrange Equations and Iterative Solution	42
5.4	Horn-Schunck vs. Lucas-Kanade	42
5.5	Legacy and Influence	43
6	Why Classical Methods Fail: The Motivation for Deep Learning	45
6.1	The Case Against Classical Methods	45
6.2	Failure Mode 1: Large Displacements	45
6.3	Failure Mode 2: Illumination Changes	45
6.4	Failure Mode 3: Occlusion	46
6.5	Failure Mode 4: Texture-Less Regions	46
6.6	Summary and Motivation for Deep Learning	46
7	Deep Learning for Optical Flow: Foundation	49
7.1	The Paradigm Shift	49
7.2	What Is a Cost Volume?	49
7.3	The Correlation Layer	50
7.4	Encoder-Decoder Architecture for Flow	50
7.5	Training Data for Optical Flow	51
7.6	Loss Functions and Evaluation Metrics	51
7.6.1	Supervised Loss: EPE	51
7.6.2	Multi-Scale Loss	51
7.6.3	End-Point Error (EPE) and Fl-all	52
8	FlowNet: The Pioneer	55
8.1	The 2015 Revolution	55
8.2	FlowNetS: The Simple Architecture	56
8.3	FlowNetC: Explicit Correlation	57
8.4	Conceptual PyTorch Implementation	58
8.5	Why FlowNet Was Revolutionary	65
9	PWC-Net: Pyramid, Warping, Cost Volume	67
9.1	The Three Pillars	67
9.2	Learned Feature Pyramid	67
9.3	Warping: The Key Innovation	67
9.4	Level-wise Cost Volume	68
9.5	Warping Implementation	68
9.6	Context Network and Final Architecture	70
10	RAFT: Recurrent All-Pairs Field Transforms	73
10.1	Why RAFT Was a Breakthrough	73
10.2	Feature Encoder and Context Encoder	74
10.2.1	Feature Encoder g_θ	74
10.2.2	Context Encoder h_θ	74
10.3	All-Pairs Correlation: The 4D Correlation Volume	74
10.3.1	The Correlation Pyramid	75
10.4	The GRU Update Operator	75
10.4.1	Standard GRU Review	75

10.4.2	What the GRU Receives as Input	76
10.4.3	Correlation Lookup	76
10.4.4	The Update	76
10.5	Complete RAFT PyTorch Implementation	77
10.6	Visualization of Iterative Refinement	92
11	FlowFormer: Transformers for Optical Flow	97
11.1	Why Transformers for Optical Flow?	97
11.2	FlowFormer’s Architecture at a Conceptual Level	98
11.3	Cost Volume Encoding with Transformers	98
11.4	Cross-Attention for Frame Matching	99
11.5	Connection to Your Conformer Architecture	99
11.6	What Questions FlowFormer Raises for Future Research	100
12	KITTI Dataset and PyTorch Data Pipeline	103
12.1	What KITTI Contains	103
12.2	Complete PyTorch DataLoader Implementation	104
13	Complete Comparative Study Pipeline	113
13.1	Designing a Fair Comparison	113
13.2	Complete Evaluation Pipeline	113
13.3	Writing Findings Like a Researcher	121
14	Connecting to 3D Vision	125
14.1	How Optical Flow Implies 3D Motion	125
14.2	Scene Flow: The 3D Analog of Optical Flow	125
14.3	Structure from Motion Connection	126
14.4	Depth from Flow	126
14.5	What to Study Next	127
15	Research Narrative and Future Directions	129
15.1	Presenting This Project in a Lab Application	129
15.2	Connecting Your Conformer Insight to FlowFormer	130
15.3	Suggested Future Experiments	130
15.4	Key Papers to Read Next	131
15.5	Key Researchers in Optical Flow	131
15.6	How This Project Fits into the Broader CV Research Landscape	132
A	Quick Reference: OpenCV Optical Flow Functions	135
B	Training Curriculum Reference	137
C	Evaluation Checklist for Your Comparative Study	139

Notation and Symbols

Symbol	Type	Meaning
$I(x, y, t)$	Scalar	Image intensity at pixel (x, y) at time t
I_x, I_y	Scalar	Spatial partial derivatives of I
I_t	Scalar	Temporal partial derivative of I
(u, v)	2D vector	Optical flow: u = horizontal, v = vertical
$\mathbf{f}$	2D vector	Generic flow vector $(u, v)^T$
∇I	2D vector	Spatial gradient $(I_x, I_y)^T$
E_{EPE}	Scalar	End-Point Error (primary evaluation metric)
λ	Scalar	Regularization weight
$\mathbf{M}$	2×2 matrix	Structure tensor (Harris/LK)
$\mathcal{L}$	Scalar	Loss function
$\otimes$	Operator	Cross-correlation
$*$	Operator	Convolution
$\ \cdot\ _1, \ \cdot\ _2$	Norm	L1 and L2 norms

Image coordinate convention: x increases right, y increases downward, origin at top-left. **Flow convention:** (u, v) at pixel (x, y) means the content there in frame t moved to $(x + u, y + v)$ in frame $t + 1$.

Chapter 1

Why Optical Flow

A still photograph captures a moment. Optical flow captures the story between moments.

1.1 The Problem of Motion in Vision

Imagine you are driving through Kathmandu and your camera captures a sequence of frames. In each individual frame, you see cars, pedestrians, buildings, and road markings. But here is the question that separates 2D image understanding from genuine scene understanding: *what is moving, how fast, and in which direction?*

This question sounds deceptively simple. Your brain answers it instantaneously, without apparent effort. You see a ball flying through the air and you immediately know its trajectory. You notice a pedestrian stepping into the road and you predict where they will be in half a second. You have been doing this since you learned to walk.

But your camera does not have this understanding built in. Each frame is just an array of pixel intensities. There is no flag that says “this group of pixels is a car moving at 30 km/h to the right.” The motion information is implicit, buried in the differences between consecutive frames. Extracting this information explicitly is the problem of **optical flow**.

Definition

Optical Flow is the apparent motion of visual content (brightness patterns) between two consecutive images, caused by relative motion between the camera and the scene. It is represented as a dense vector field $\mathbf{f}(x, y) = (u(x, y), v(x, y))$ where each vector describes how far the content at pixel (x, y) in frame t has moved by frame $t + 1$.

The word “apparent” is important and often overlooked. Optical flow is not exactly the same as the true 3D motion projected onto the image plane (called the *motion field*). A rotating sphere with uniform texture has a non-zero motion field, but zero optical flow, because the brightness pattern looks identical frame to frame. In practice,

for most real scenes, optical flow closely approximates the projected 3D motion, and we treat them as equivalent until Chapter 14 where the distinction becomes critical.

1.2 Real-World Applications

1.2.1 Autonomous Driving

This is the application closest to your existing work. In your autonomous vehicle project, you worked with OpenCV to process visual data from cameras. Optical flow adds a temporal dimension to that work.

Consider what an autonomous vehicle needs to know:

- **Moving obstacle detection:** A parked car and a moving car look similar in a single frame. In an optical flow field, the moving car produces large flow vectors; the parked car produces near-zero flow.
- **Ego-motion estimation:** When the camera itself moves (your car is driving), the entire scene appears to flow. This pattern encodes your own speed and direction — called *ego-flow*.
- **Time-to-collision (TTC):** The magnitude of flow from an approaching object is directly related to how quickly it is approaching. Large, rapidly expanding flow means imminent collision.
- **Pedestrian intent prediction:** A pedestrian with flow vectors pointing toward the road is likely about to step out.

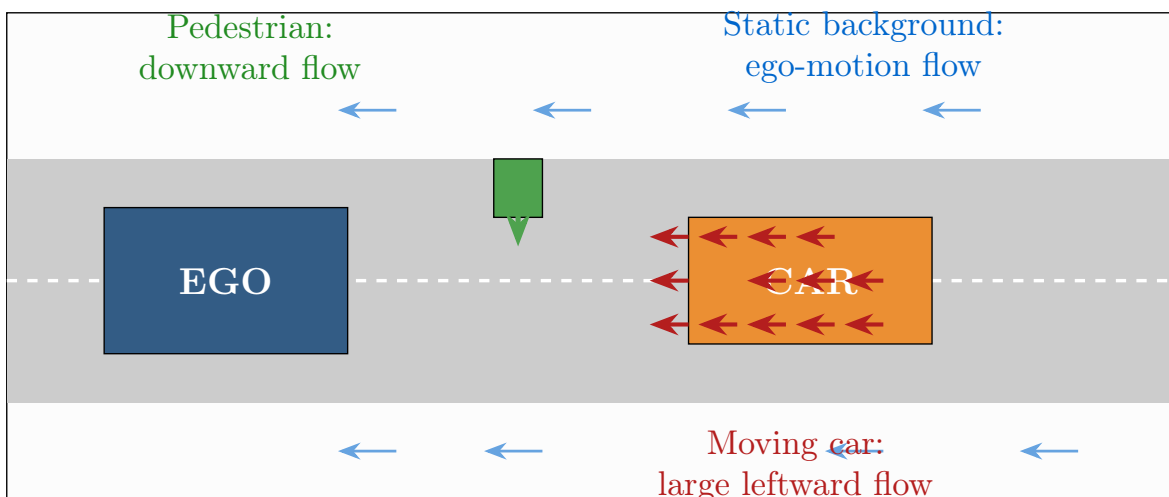


Figure 1.1: Optical flow in an autonomous driving scene. Static background produces backward flow from ego-motion. The moving car produces flow different from ego-motion. The pedestrian moving toward the road produces downward flow. All of this is invisible in a single frame.

1.2.2 Video Compression

Every time you watch a YouTube video, optical flow works behind the scenes. Video codecs like H.264 and H.265 do not store every frame independently. Instead, they store one reference frame completely, then for subsequent frames store only *motion vectors* (essentially optical flow) and the *residual* (the difference that motion alone cannot explain). This reduces video file sizes by 10–100 $\times$.

1.2.3 Action Recognition, Medical Imaging, and Video Stabilization

Two-stream networks for action recognition use one stream for RGB frames and a second stream for optical flow, providing motion-specific features that appearance alone cannot capture. In cardiac MRI, optical flow tracks heart wall motion to measure cardiac function. When your phone stabilizes shaky video, it computes optical flow between frames to understand and counteract unwanted camera motion.

1.3 Where Optical Flow Fits in the CV Pipeline

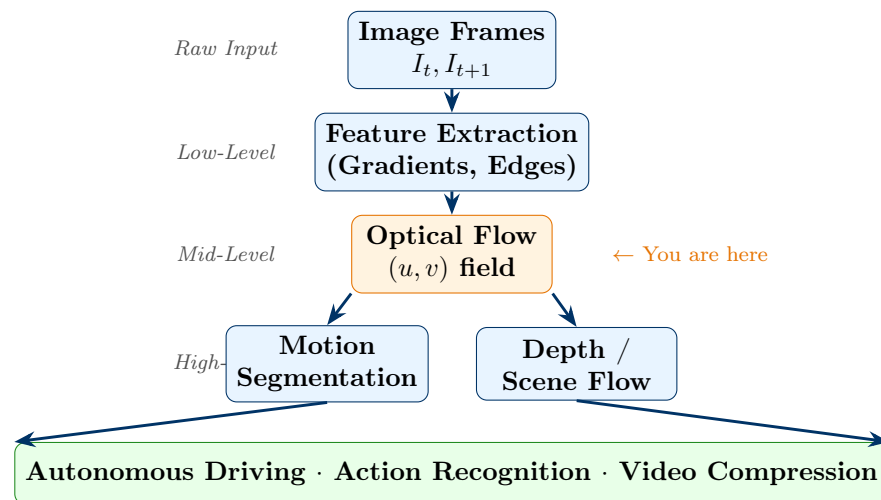


Figure 1.2: Optical flow in the computer vision processing pipeline. It sits between low-level feature extraction and high-level semantic understanding.

1.4 The Connection to 3D Vision

Here is the perspective that makes everything in this book feel purposeful: **optical flow is the shadow of 3D motion on the image plane.**

When a 3D point $\mathbf{P} = (X, Y, Z)$ moves between two frames, its image projection moves accordingly. Under a pinhole camera model with focal length f and camera motion (rotation $\mathbf{R}$, translation $\mathbf{t}$), the 2D flow vector (u, v) at each pixel encodes the 3D scene motion and structure. If you know the flow and the camera parameters, you can recover depth, 3D motion, and camera pose. This is why optical flow is not a detour on the way to 3D vision — it is a direct path. The field of **scene flow** (the 3D

analog of optical flow) is a major research area that directly extends everything you will learn here.

1.5 Connection to Your Autonomous Vehicle Project

In most introductory AV projects, the vision pipeline uses frame-by-frame detection — detect objects in each frame independently and associate them across frames using IoU matching or Kalman filters. Optical flow offers a fundamentally different approach: instead of *detecting* where objects are in each frame independently, you *track* them continuously using flow. Instead of estimating your vehicle’s speed from GPS, you estimate it from the ego-flow pattern using *visual odometry*. The OpenCV functions you already know — `cv2.calcOpticalFlowPyrLK` for sparse flow and `cv2.calcOpticalFlowFarneback` for dense flow — are the industrial-strength implementations of the algorithms we derive from scratch in Chapters 3 and 4.

Chapter 1 Summary: Why Optical Flow

- **Optical flow** is the dense 2D vector field describing apparent motion of brightness patterns between consecutive frames.
- It is a **mid-level vision task** bridging raw pixels and high-level understanding.
- Key applications: autonomous driving, video compression, action recognition, medical imaging, video stabilization.
- Optical flow is mathematically **connected to 3D vision**: it is the projection of 3D scene motion onto the image plane.
- Methods span three eras: **classical** (LK, Farneback, Horn-Schunck), **early deep learning** (FlowNet, PWC-Net), and **state-of-the-art** (RAFT, FlowFormer).

Connection to Your Background

You implemented an autonomous vehicle perception system using OpenCV. Every `cv2.calcOpticalFlowPyrLK` call is decades of computer vision research compressed into a function. This book gives you the mathematical and algorithmic foundation to understand, modify, and improve those calls — and to replace them with neural network approaches when classical methods fail. Your Conformer for ASR processes temporal sequences to extract meaningful representations; optical flow is doing the same for video: extracting meaningful motion representations from temporal sequences of frames.

Research Questions to Explore

1. **The boundary between motion and appearance:** We defined optical flow as “apparent motion of brightness patterns.” If an object moves but has uniform brightness, there is no optical flow. Is this a fundamental limitation, or can we augment flow estimation with semantic information? How do modern methods handle texture-less regions?
2. **Camera motion vs. scene motion:** When your AV moves, the background

appears to flow due to ego-motion. How do state-of-the-art methods separate ego-motion-induced flow from independently moving object (IMO) flow?

3. **From 2D flow to 3D motion:** 2D flow encodes 3D motion, but multiple 3D motions can produce the same 2D flow. Under what conditions is the 3D motion uniquely recoverable? What additional information (stereo baseline, multiple frames) resolves this ambiguity?

Chapter 2

Mathematical Foundation

The Brightness Constancy Assumption is the simplest model that is often wrong — but wrong in the most useful way.

2.1 The Image as a Mathematical Function

An image is a function. This foundational abstraction makes everything else possible. Formally, a grayscale image at time t is:

$$I : \Omega \times \mathbb{R}_{\geq 0} \rightarrow [0, 255] \quad \text{where } \Omega \subset \mathbb{R}^2 \quad (2.1)$$

$I(x, y, t)$ gives the brightness at position (x, y) at time t . In practice we work with a discretized version, but it is mathematically convenient to treat the image as continuous because we need to compute derivatives.

Intuition First

Intuition: Think of the image as a landscape of brightness. Mountains are bright regions, valleys are dark. A moving object is a mountain that shifts its position between frames. Optical flow asks: for each mountain, which direction and how far did it move?

2.1.1 Spatial and Temporal Gradients

The **spatial gradient** is the rate of change of brightness in the x and y directions:

$$I_x = \frac{\partial I}{\partial x}, \quad I_y = \frac{\partial I}{\partial y} \quad (2.2)$$

The **temporal gradient** is the rate of change over time:

$$I_t = \frac{\partial I}{\partial t} \quad (2.3)$$

The spatial gradient vector $\nabla I = (I_x, I_y)^T$ points in the direction of maximum brightness increase, and $\|\nabla I\|$ gives its magnitude. I_t tells us how rapidly brightness

at a fixed pixel location is changing over time. In practice these are approximated by finite differences.

```

1 import numpy as np
2
3 def compute_gradients(frame1: np.ndarray, frame2: np.ndarray) -> tuple:
4     """
5     Compute spatial and temporal image gradients.
6
7     These are the foundational quantities for optical flow estimation.
8     Every classical method (LK, Horn-Schunck, Farneback) uses these.
9
10    Args:
11        frame1: First frame, shape (H, W), float32, range [0,1]
12        frame2: Second frame, shape (H, W), float32, range [0,1]
13
14    Returns:
15        Ix: Horizontal gradient, shape (H, W)
16        Iy: Vertical gradient, shape (H, W)
17        It: Temporal gradient (frame2 - frame1), shape (H, W)
18    """
19    f1 = frame1.astype(np.float32)
20    f2 = frame2.astype(np.float32)
21
22    # Average the two frames before computing spatial gradients.
23    # This reduces temporal aliasing: we estimate gradients at the
24    # "midpoint" between the two frames rather than either extreme.
25    I_avg = 0.5 * (f1 + f2)
26
27    # Central difference:  $f'(x)$  approximately  $[f(x+1) - f(x-1)] / 2$ 
28    # np.gradient computes this automatically with boundary handling.
29    # Note the order: np.gradient returns (row_grad, col_grad)
30    # row direction = y direction; col direction = x direction
31    Iy, Ix = np.gradient(I_avg)
32
33    # Temporal gradient: how much did brightness change between frames?
34    # Positive It: pixel got brighter. Negative It: pixel got darker.
35    It = f2 - f1
36
37    return Ix, Iy, It
38
39
40 # ----- ILLUSTRATION -----
41 # A bright square shifting right by 2 pixels
42 H, W = 50, 50
43 frame1 = np.zeros((H, W), dtype=np.float32)
44 frame2 = np.zeros((H, W), dtype=np.float32)
45
46 frame1[20:30, 15:25] = 1.0    # White square in frame1
47 frame2[20:30, 17:27] = 1.0    # Same square shifted right by 2 pixels in
48                                 frame2

```

```

49 Ix, Iy, It = compute_gradients(frame1, frame2)
50
51 # OBSERVATION:
52 # - Ix is large at left/right edges of the square (vertical edges)
53 # - Iy is large at top/bottom edges (horizontal edges)
54 # - It is negative where frame1 was bright but frame2 is dark (square left
55   )
56 # - It is positive where frame2 is bright but frame1 was dark (square
57   arrived)
58 print("Max |Ix|:", np.max(np.abs(Ix)))
59 print("Max |Iy|:", np.max(np.abs(Iy)))
60 print("Left region It (square departed):", It[20:30, 15:17].mean()) #
61   negative
62 print("Right region It (square arrived):", It[20:30, 25:27].mean()) #
63   positive

```

Listing 2.1: Computing image gradients in NumPy — the fundamental building block of optical flow

What to observe: The temporal gradient I_t is negative where the square departed (was bright in frame1, dark in frame2) and positive where it arrived (dark in frame1, bright in frame2). The spatial gradient I_x is large at vertical boundaries. This asymmetry is the signal that the optical flow constraint equation exploits.

2.2 The Brightness Constancy Assumption

The core assumption of all classical optical flow is the **Brightness Constancy Assumption (BCA)**:

Definition

Brightness Constancy Assumption: The apparent brightness of a moving surface element remains constant over time:

$$I(x, y, t) = I(x + u, y + v, t + 1) \quad (2.4)$$

Or in continuous form: $\frac{d}{dt}I(x(t), y(t), t) = 0$

When BCA holds: Uniform, constant illumination; Lambertian (diffuse) surfaces.

When BCA fails: Specular reflections (brightness depends on viewpoint); changing illumination (flickering lights); transparent objects; auto-exposure adjustments. Chapter 6 explores these failures. For now, we adopt BCA and derive its consequences.

2.3 The Optical Flow Constraint Equation — Full Derivation

This is the most important derivation in this book. Read it slowly. Every symbol matters.

Starting point: Brightness Constancy in continuous form:

$$\frac{d}{dt}I(x(t), y(t), t) = 0 \quad (2.5)$$

Expanding via the multivariate chain rule:

$$\frac{\partial I}{\partial x} \cdot \frac{dx}{dt} + \frac{\partial I}{\partial y} \cdot \frac{dy}{dt} + \frac{\partial I}{\partial t} = 0 \quad (2.6)$$

Identifying terms: $\frac{\partial I}{\partial x} = I_x$, $\frac{\partial I}{\partial y} = I_y$, $\frac{\partial I}{\partial t} = I_t$, $\frac{dx}{dt} = u$, $\frac{dy}{dt} = v$, we obtain:

$$\boxed{I_x \cdot u + I_y \cdot v + I_t = 0} \quad (2.7)$$

This is the **Optical Flow Constraint Equation (OFCE)**. All quantities on the left are computable from the two image frames. The unknowns are u and v .

Common Pitfall

This single equation has two unknowns. We cannot uniquely solve for (u, v) from Equation 2.7 alone. This is the fundamental ill-posedness of optical flow, and it gives rise to the *aperture problem*. Every classical optical flow method is a different strategy for adding constraints to resolve this ambiguity.

2.3.1 Geometric Interpretation

The OFCE says (u, v) must lie on the line $I_x u + I_y v = -I_t$ in the (u, v) plane. Any point on this line is a valid flow vector. There are infinitely many such points. We can determine only the component of flow in the direction of ∇I (the normal component); the component along the constraint line (tangential to the edge) is completely unknown from a single pixel.

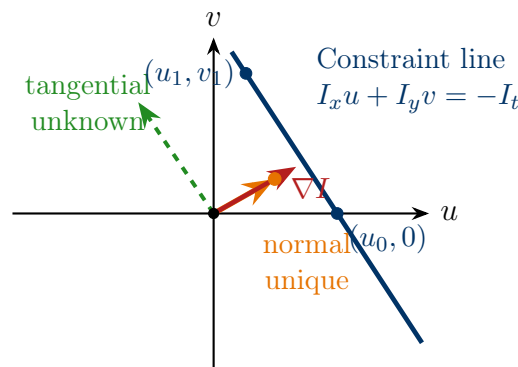


Figure 2.1: Geometric interpretation of the OFCE. The constraint line contains all valid flow vectors. Only the normal component (orange) is uniquely determined; the tangential component (green) is the aperture problem.

2.4 The Aperture Problem

The aperture problem is one of the most beautiful phenomena in visual perception. Imagine looking at a large moving barber pole through a small circular hole in a wall.

Through the small aperture, you can see only a tiny patch of the diagonal stripes. As the pole rotates, the stripes move — but which direction? You genuinely cannot tell whether the stripe is moving diagonally, horizontally, or vertically. Through the small aperture, all produce identical brightness changes.

Mathematically: at any single pixel, only the flow component perpendicular to the local edge (the **normal flow**) is determinable:

$$u_n = \frac{-I_t \cdot I_x}{I_x^2 + I_y^2}, \quad v_n = \frac{-I_t \cdot I_y}{I_x^2 + I_y^2} \quad (2.8)$$

The component of flow along the edge is completely unknown from a single pixel.

Two principled solutions:

1. **Local constraints (Lucas-Kanade, Ch. 3):** Assume flow is constant over a small spatial window, providing many equations for the same (u, v) .
2. **Global constraints (Horn-Schunck, Ch. 5):** Assume the flow field is spatially smooth everywhere.

2.5 Taylor Series Approximation

The OFCE derivation used the Taylor expansion. If displacement (u, v) is *small*, we expand $I(x + u, y + v, t + 1)$ to first order around (x, y, t) :

$$I(x + u, y + v, t + 1) \approx I(x, y, t) + I_x \cdot u + I_y \cdot v + I_t \cdot 1 \quad (2.9)$$

Substituting into BCA gives exactly the OFCE. The key assumption is that (u, v) is small enough that higher-order terms are negligible.

```

1 import numpy as np
2
3 def test_taylor_accuracy(displacement: float) -> float:
4     """
5     Test how accurate the Taylor approximation is for a given displacement
6     .
7     We use f(x) = sin(x) and compare:
8     Exact: f(x + dx)
9     Taylor: f(x) + f'(x) * dx = sin(x) + cos(x) * dx
10
11     This illustrates WHY small displacement matters for the OFCE.
12     """
13     x = np.linspace(0, 2 * np.pi, 1000)
14     f_exact = np.sin(x + displacement)
15     f_taylor = np.sin(x) + np.cos(x) * displacement # 1st-order Taylor
16     return np.max(np.abs(f_exact - f_taylor))
17
18 displacements = [0.01, 0.1, 0.5, 1.0, 2.0, 5.0]
19 print(f"{'Displacement':>14} | {'Max Taylor Error':>16}")
20 print("-" * 35)
21 for dx in displacements:
```

```

22     err = test_taylor_accuracy(dx)
23     print(f"    {dx:10.2f}    |    {err:.6f}")
24
25     # OBSERVATION (approximate):
26     # dx = 0.01: error ~ 0.000050    (excellent - well within valid regime)
27     # dx = 0.10: error ~ 0.005000    (good)
28     # dx = 1.00: error ~ 0.460000    (poor! - this is 1 pixel at full
    resolution)
29     # dx = 5.00: error ~ 1.790000    (terrible - 5 pixels = completely wrong)
30     #
31     # This is the quantitative reason why pyramidal LK and RAFT's
32     # iterative refinement exist: to handle displacements > 1 pixel reliably.

```

Listing 2.2: NumPy illustration of Taylor approximation accuracy vs. displacement magnitude

What to observe: Error grows rapidly beyond 1 pixel displacement. At 5 pixels, the approximation is useless. This motivates pyramidal approaches (coarse-to-fine) and RAFT’s iterative strategy for large displacements.

Chapter 2 Summary: Mathematical Foundation

- An image is a function $I(x, y, t)$; its partial derivatives I_x, I_y, I_t are the building blocks for flow estimation.
- The **Brightness Constancy Assumption** states that moving pixels carry their brightness: $\frac{d}{dt}I(x(t), y(t), t) = 0$.
- The chain rule applied to BCA yields the **Optical Flow Constraint Equation**: $I_x u + I_y v + I_t = 0$.
- The OFCE is **ill-posed**: one equation, two unknowns — a line of valid solutions in (u, v) space.
- The **Aperture Problem**: at any single pixel, only the flow component perpendicular to the local edge is determinable.
- The **Taylor approximation** makes the OFCE linear, but only holds for small displacements ($|u|, |v| \lesssim 1$ pixel). This motivates pyramidal and iterative approaches.
- Two classes of solution: *local* constraints (Lucas-Kanade) and *global* constraints (Horn-Schunck).

Connection to Your Background

The mathematical machinery here — Taylor expansion, partial derivatives, linear system formulation — is the same toolkit you used in backpropagation. When you derived gradients flowing backward through your neural network, you were computing $\partial \mathcal{L} / \partial W$ — partial derivatives of a scalar function with respect to parameters. Here we compute partial derivatives of a vector-valued function $I(x, y, t)$. The conceptual skill (“what changes when I wiggle this variable slightly?”) is identical. Your comfort with the chain rule from implementing backprop is a significant head start here.

Research Questions to Explore

1. **Beyond brightness constancy:** Several stronger assumptions exist, such as *gradient constancy* ($\nabla I(x + u, y + v) = \nabla I(x, y)$) which is more robust to global illumination changes. How does this modify the OFCE, and when is it more appropriate? What is the computational cost?
2. **The information content of edges vs. corners:** The aperture problem is severe at edges but not at corners (the normal constraint points in different directions at nearby edge segments). How does the local structure — quantified by structure tensor eigenvalues — determine how much flow information we can reliably extract?
3. **Higher-order flow models:** The OFCE assumes constant flow within the approximation window. What happens if we model flow as an affine transformation instead? How does this change the system of equations, and what kinds of motion does it capture more accurately?

Chapter 3

Sparse Optical Flow: Lucas-Kanade

Lucas and Kanade had a simple insight: instead of solving one equation with two unknowns, collect many equations from a neighborhood. It works embarrassingly well.

3.1 The Core Idea

In Chapter 2 we saw that the OFCE is insufficient to determine flow at a single pixel. Lucas and Kanade’s [?] key insight: *do not solve it at a single pixel — solve it for a small patch of pixels together, assuming they all share the same flow.*

This is the **constant flow assumption**: within a small window W of size $(2k+1) \times (2k+1)$ pixels around the point of interest, the flow (u, v) is approximately constant. This is reasonable because most real-world objects are larger than a few pixels, so a 15×15 patch is likely to lie entirely within the same moving object. The assumption breaks at motion boundaries, but holds well for most pixels.

With this assumption, every pixel (x_i, y_i) in the window gives one equation:

$$I_x(x_i, y_i) \cdot u + I_y(x_i, y_i) \cdot v = -I_t(x_i, y_i) \quad \forall (x_i, y_i) \in W \quad (3.1)$$

For a 15×15 window, we have $n = 225$ equations for 2 unknowns. This overdetermined system is solved by least squares.

3.2 Least Squares Formulation — Full Derivation

In matrix form, the n equations become $A\mathbf{f} = \mathbf{b}$, where:

$$A = \begin{pmatrix} I_{x,1} & I_{y,1} \\ I_{x,2} & I_{y,2} \\ \vdots & \vdots \\ I_{x,n} & I_{y,n} \end{pmatrix} \in \mathbb{R}^{n \times 2}, \quad \mathbf{f} = \begin{pmatrix} u \\ v \end{pmatrix}, \quad \mathbf{b} = - \begin{pmatrix} I_{t,1} \\ \vdots \\ I_{t,n} \end{pmatrix} \quad (3.2)$$

We minimize $\|A\mathbf{f} - \mathbf{b}\|_2^2$. The least-squares solution is given by the **normal equations**:

$$(A^T A)\hat{\mathbf{f}} = A^T \mathbf{b} \quad (3.3)$$

Expanding $A^T A$:

$$M \triangleq A^T A = \begin{pmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{pmatrix} \quad (3.4)$$

This 2×2 matrix M is the **structure tensor** (also called the second-moment matrix or Harris matrix). The solution is:

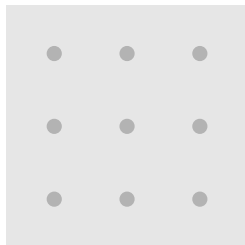
$$\begin{pmatrix} u \\ v \end{pmatrix} = M^{-1} \begin{pmatrix} -\sum I_x I_t \\ -\sum I_y I_t \end{pmatrix} \quad (3.5)$$

3.3 Eigenvalue Analysis: When Does Lucas-Kanade Fail?

Solving Equation 3.5 requires M to be invertible. Let $\lambda_1 \geq \lambda_2 \geq 0$ be the eigenvalues of M .

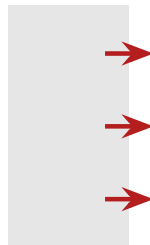
Case	Eigenvalues	Region type	LK solvable?
Flat region	$\lambda_1 \approx \lambda_2 \approx 0$	Uniform intensity	✗ No
Edge	$\lambda_1 \gg \lambda_2 \approx 0$	One gradient direction	✗ No (aperture)
Corner	$\lambda_1 \approx \lambda_2 \gg 0$	Two gradient directions	✓ Yes!

Flat Region



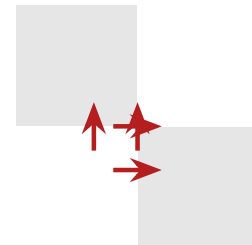
$\lambda_1 \approx \lambda_2 \approx 0$
LK fails

Edge



$\lambda_1 \gg \lambda_2 \approx 0$
Aperture problem

Corner



$\lambda_1 \approx \lambda_2 \gg 0$
LK succeeds!

Figure 3.1: Three types of image regions and their structure tensor eigenvalue profiles. LK succeeds only at corner-like regions where both eigenvalues are large.

3.4 Connection to the Harris Corner Detector

The points where LK is most reliable are exactly the points that the Harris corner detector [?] identifies! The Harris response:

$$R = \det(M) - k \cdot (\text{tr}(M))^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2 \quad (3.6)$$

Large positive R indicates a corner (λ_1 and λ_2 both large). The Shi-Tomasi criterion [?] uses $R_{ST} = \min(\lambda_1, \lambda_2)$, and OpenCV's `goodFeaturesToTrack()` implements exactly this to select the best tracking points.

3.5 Pyramidal Lucas-Kanade

The vanilla LK derivation assumes small displacements ($|u|, |v| \lesssim 1$ pixel). For fast motion, this fails. The pyramidal solution: build an image pyramid (downsample each frame by 2 at each level) and compute flow from coarse to fine.

A displacement of 16 pixels at level 0 becomes only 2 pixels at level 3 — within the small-displacement regime. We compute flow at the coarsest level, scale it up by 2, use it to warp image 2, then compute the residual flow at the next finer level.

Algorithm 1: Pyramidal Lucas-Kanade

Input: Frames I_1, I_2 ; points $\{p_i\}$; levels L ; window size k
Output: Flow vectors $\mathbf{f}_i = (u_i, v_i)$

```

1 Build image pyramids  $\{I_1^{(\ell)}\}, \{I_2^{(\ell)}\}$  // downsample by 2 per level
2 for each point  $p_i$  do
3    $\mathbf{g} \leftarrow (0, 0)$  // accumulated flow estimate
4   for  $\ell = L$  to 0 do
5      $q^{(\ell)} \leftarrow p_i / 2^\ell + \mathbf{g}$  // scale point and apply accumulated flow
6     Compute  $M = A^T A$  in window around  $q^{(\ell)}$  in  $I_1^{(\ell)}$ 
7     if  $\min(\lambda_1, \lambda_2) < \epsilon$  then
8       skip // unreliable point
9     for  $j = 1$  to  $max\_iter$  do
10      Compute  $I_t$  by warping  $I_2^{(\ell)}$  and differencing
11       $\delta \mathbf{f} \leftarrow M^{-1}(-A^T \mathbf{I}_t)$ 
12       $\mathbf{g} \leftarrow \mathbf{g} + \delta \mathbf{f}$ 
13      if  $\|\delta \mathbf{f}\| < \text{threshold}$  then
14        break
15    if  $\ell > 0$  then
16       $\mathbf{g} \leftarrow 2 \cdot \mathbf{g}$  // upscale for next finer level
17   $\mathbf{f}_i \leftarrow \mathbf{g}$ 

```

3.6 Complete OpenCV Implementation

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5
6 def detect_features(frame: np.ndarray) -> np.ndarray:
7     """
8     Detect 'good features to track' using the Shi-Tomasi criterion.
9
10    These are points where min(lambda1, lambda2) of the structure tensor
11    is maximized --- corners and corner-like structures.
12
13    We track SPARSE features (specific points), unlike dense methods
14    which compute flow at every pixel.

```

```

15
16     Args:
17         frame: Grayscale image, uint8
18
19     Returns:
20         corners: Shape (N, 1, 2) with corner coordinates.
21                 The nested shape is OpenCV's convention for flow APIs.
22     """
23     corners = cv2.goodFeaturesToTrack(
24         image=frame,
25         maxCorners=200,          # Maximum number of corners
26         qualityLevel=0.01,      # Min quality relative to best corner score
27         minDistance=10,         # Min Euclidean distance between detected
28         corners                # Window size for structure tensor
29         blockSize=7,           # Window size for structure tensor
30         computation
31         useHarrisDetector=False, # False = Shi-Tomasi; True = Harris R
32         score
33     )
34     return corners
35
36 def compute_sparse_flow(
37     frame1: np.ndarray,
38     frame2: np.ndarray,
39     points: np.ndarray
40 ) -> tuple:
41     """
42     Compute sparse optical flow using Pyramidal Lucas-Kanade.
43
44     OpenCV's calcOpticalFlowPyrLK implements the Bouguet algorithm:
45     1. Builds image pyramids for both frames
46     2. At each level, solves the LK normal equations
47     3. Iterates until convergence or max iterations
48     4. Propagates refined flow from coarse to fine levels
49
50     Args:
51         frame1: Reference frame, grayscale uint8
52         frame2: Target frame, grayscale uint8
53         points: Points to track from frame1, shape (N, 1, 2)
54
55     Returns:
56         good_old: Points in frame1 that were successfully tracked
57         good_new: Corresponding positions in frame2
58         status:   Track status for all input points (1=success, 0=lost)
59     """
60     lk_params = dict(
61         winSize=(15, 15),          # Window size at each pyramid level
62                                     # Larger: more robust, less spatial detail
63         maxLevel=3,                # Number of pyramid levels above base (0 =
64         no pyramid)

```

```

62                                     # Level 3 means 1/8 resolution is the
starting point
63         criteria=(
64             cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
65             30,          # Max iterations per level
66             0.03         # Epsilon: stop when correction < 0.03 pixels
67         )
68     )
69
70     points_new, status, error = cv2.calcOpticalFlowPyrLK(
71         prevImg=frame1,
72         nextImg=frame2,
73         prevPts=points,
74         nextPts=None,    # None = compute fresh; pass initial guess to
warm-start
75         **lk_params
76     )
77
78     # Filter to only successfully tracked points
79     # status == 1: point was found and tracked
80     # status == 0: point was lost (out-of-bounds, low texture, etc.)
81     good_mask = status.flatten() == 1
82     good_old = points[good_mask]      # Positions in frame1
83     good_new = points_new[good_mask]  # Corresponding positions in frame2
84
85     # Flow vector at good_old[i]: (good_new[i] - good_old[i])
86     return good_old, good_new, status
87
88
89 def visualize_sparse_flow(
90     frame1: np.ndarray,
91     frame2: np.ndarray,
92     good_old: np.ndarray,
93     good_new: np.ndarray
94 ) -> np.ndarray:
95     """
96     Visualize sparse optical flow as colored arrows overlaid on blended
97     frames.
98     Each arrow goes FROM the original position TO the new position.
99     Color encodes flow magnitude: blue (slow) to red (fast).
100    """
101    frame_blend = cv2.addWeighted(
102        cv2.cvtColor(frame1, cv2.COLOR_GRAY2BGR), 0.5,
103        cv2.cvtColor(frame2, cv2.COLOR_GRAY2BGR), 0.5, 0
104    )
105
106    for new, old in zip(good_new, good_old):
107        x_new, y_new = new.ravel().astype(int)
108        x_old, y_old = old.ravel().astype(int)
109
110        dx, dy = x_new - x_old, y_new - y_old

```

```

110     magnitude = np.sqrt(dx**2 + dy**2)
111
112     # Color: blue (small flow) -> red (large flow)
113     c = min(int(magnitude * 20), 255)
114     color = (c, 50, 255 - c) # BGR
115
116     cv2.arrows(frame_blend, (x_old, y_old), (x_new, y_new),
117                color, thickness=2, tipLength=0.3)
118     cv2.circle(frame_blend, (x_old, y_old), 3, (0, 255, 0), -1)
119
120     return frame_blend

```

Listing 3.1: Sparse optical flow with Pyramidal Lucas-Kanade in OpenCV — heavily commented

3.7 NumPy Implementation of Core LK Mathematics

```

1  import numpy as np
2  from scipy.ndimage import convolve
3
4  def lk_optical_flow_numpy(
5      frame1: np.ndarray,
6      frame2: np.ndarray,
7      point: tuple,
8      window_size: int = 15
9  ) -> tuple:
10     """
11     Compute Lucas-Kanade flow at a single point using pure NumPy.
12     This is a mathematical illustration, NOT production code.
13     Use cv2.calcOpticalFlowPyrLK in production.
14
15     Args:
16         frame1, frame2: Float32 images in [0,1], shape (H,W)
17         point: (x, y) pixel coordinate
18         window_size: Neighborhood window (should be odd)
19
20     Returns:
21         u, v: Flow components
22         confidence: min eigenvalue of M (reliability measure)
23     """
24     x, y = point
25     H, W = frame1.shape
26     k = window_size // 2 # Half-window radius
27
28     # --- Step 1: Compute spatial gradients using Sobel filters ---
29     # Sobel approximates the image derivative robustly in the presence of
30     # noise
31     sobel_x = np.array([[[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=np.
float32)

```

```

31     sobel_y = np.array([[ -1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=np.
float32)
32
33     I_avg = 0.5 * (frame1 + frame2)    # Average frames for spatial
gradient
34     Ix = convolve(I_avg, sobel_x) / 8.0    # /8 normalizes the Sobel kernel
35     Iy = convolve(I_avg, sobel_y) / 8.0
36
37     # --- Step 2: Temporal gradient ---
38     It = frame2 - frame1    # How much did brightness change?
39
40     # --- Step 3: Extract window around point ---
41     x_min, x_max = max(0, x-k), min(W, x+k+1)
42     y_min, y_max = max(0, y-k), min(H, y+k+1)
43
44     Ix_win = Ix[y_min:y_max, x_min:x_max].flatten()
45     Iy_win = Iy[y_min:y_max, x_min:x_max].flatten()
46     It_win = It[y_min:y_max, x_min:x_max].flatten()
47
48     # --- Step 4: Build system  $A*f = b$  ---
49     # Each row is one pixel's OFCE:  $Ix*u + Iy*v = -It$ 
50     A = np.stack([Ix_win, Iy_win], axis=1)    # (n, 2)
51     b = -It_win                                # (n,)
52
53     # --- Step 5: Structure tensor  $M = A^T A$  ---
54     M = A.T @ A    # (2, 2): [[sum(Ix^2), sum(Ix*Iy)], [sum(Ix*Iy), sum(Iy
^2)]]
55
56     # --- Step 6: Eigenvalue analysis for reliability ---
57     eigenvalues = np.linalg.eigvals(M)
58     lambda_min = np.min(np.real(eigenvalues))
59
60     if lambda_min < 1e-6:
61         # Flat region or pure edge -- cannot reliably solve
62         return 0.0, 0.0, 0.0
63
64     # --- Step 7: Solve normal equations  $(A^T A) f = A^T b$  ---
65     # np.linalg.solve is more numerically stable than explicit  $M^{-1}$ 
66     Atb = A.T @ b    # (2,): right-hand side of normal equations
67     try:
68         flow = np.linalg.solve(M, Atb)    # Solves 2x2 system
69     except np.linalg.LinAlgError:
70         return 0.0, 0.0, 0.0
71
72     # Large lambda_min = corner region = highly reliable flow
73     return float(flow[0]), float(flow[1]), float(lambda_min)

```

Listing 3.2: Core Lucas-Kanade mathematics from scratch in NumPy

3.8 Failure Case Analysis

Understanding failure modes is as important as understanding successes — this is the mindset of a researcher.

Failure Mode	Root Cause	Symptom
Large displacement	Taylor approximation breaks for $ u , v > 2\text{px}$	Points lost or wildly incorrect
Aperture problem	Insufficient eigenvalues	Drift along edge direction
Illumination change	BCA violated	Wrong direction or overestimated magnitude
Occlusion	Feature disappears	Lost track (<code>status=0</code>)
Texture-less region	$M \approx 0$, near-singular	Unstable, noisy flow

Chapter 3 Summary: Lucas-Kanade Sparse Flow

- **Core idea:** Assume constant flow over a small window. Provides n equations for 2 unknowns.
- **Solution:** Least-squares via normal equations $(A^T A)\mathbf{f} = A^T \mathbf{b}$.
- **Structure tensor** $M = A^T A$: its eigenvalues determine whether LK is applicable.
- Corners (both eigenvalues large): solvable. Edges ($\lambda_2 \approx 0$): aperture problem. Flat ($\lambda_1 \approx \lambda_2 \approx 0$): intractable.
- **Harris/Shi-Tomasi** corner detectors select the most reliable tracking points.
- **Pyramidal LK** handles larger displacements via coarse-to-fine processing.
- LK is **sparse**: it tracks specific feature points, not every pixel.

Connection to Your Background

The least-squares derivation here — overdetermined system $A\mathbf{f} = \mathbf{b}$, normal equations $A^T A\mathbf{f} = A^T \mathbf{b}$ — is the same mathematical machinery as linear regression. The structure tensor eigenvalue analysis is also how PCA works: eigenvectors of a covariance matrix, where here the “covariance” is of image gradients, not data features. Additionally, the pyramidal approach — coarse-to-fine refinement — is a recurring pattern you will see again in RAFT’s iterative refinement and in multi-scale feature extraction in CNNs.

Research Questions to Explore

1. **Gaussian weighting in the window:** Standard LK weights all pixels in the window equally. Gaussian weighting (pixels near center contribute more) can improve accuracy. How does this change the normal equations mathematically? What is the theoretical optimal window weighting?
2. **Bidirectional consistency check:** A common way to detect tracking failures is to run LK forward ($I_1 \rightarrow I_2$) and backward ($I_2 \rightarrow I_1$) and flag points where

the round-trip displacement exceeds a threshold. Why does this work as an outlier detector? What failure modes does it miss?

3. **The limits of sparsity:** LK gives flow at 100–500 corner points in a typical frame. What is the minimum density of flow information needed to fully understand scene dynamics? Is there a principled way to select the most informative points to track?

Chapter 4

Dense Optical Flow: Farnebäck's Method

Why track 200 points when you can understand every pixel?

4.1 Why Sparse Flow Is Not Enough

Lucas-Kanade gives flow at corners — perhaps 200–500 points in a typical frame. For many applications this sparsity is adequate: tracking a pedestrian's trajectory or estimating camera motion requires only a handful of feature points. But consider:

- **Video segmentation:** To separate a running person from the background, you need flow at every pixel of the person's silhouette.
- **Action recognition:** Describing how an entire body moves requires flow over the entire body region.
- **Depth from stereo:** Matching pixels across stereo images (disparity) requires dense correspondence — conceptually identical to dense flow.

Dense optical flow computes (u, v) at *every* pixel. Farnebäck's method [?] is the most widely used classical dense flow algorithm and is what OpenCV's `cv2.calcOpticalFlowFarneback` implements.

4.2 Polynomial Expansion: The Key Idea

Instead of computing gradients via finite differences, Farnebäck approximates each pixel's neighborhood using a **quadratic polynomial**:

$$f(\mathbf{x}) \approx \mathbf{x}^T A \mathbf{x} + \mathbf{b}^T \mathbf{x} + c \quad (4.1)$$

where $\mathbf{x} = (x, y)^T$, A is 2×2 , $\mathbf{b}$ is a 2-vector, c is a scalar. This polynomial is fitted separately for each pixel, using a Gaussian-weighted least squares fit over its neighborhood.

Intuition First

Intuition: Instead of looking at raw pixel brightness, Farneback represents each local image patch as a smooth mathematical surface (the polynomial). Flow is then estimated by asking: “How would this polynomial surface need to be displaced to match the next frame’s polynomial?” Working with polynomial fits is more robust to noise than working with raw pixel differences.

A quadratic polynomial can represent a constant region ($A = 0$, $\mathbf{b} = 0$), an edge ($\mathbf{b} \neq 0$), a corner (diagonal entries of A nonzero), or any smooth local intensity variation.

4.2.1 From Polynomial to Flow

If frame 1 at pixel $\mathbf{x}$ has polynomial $f_1(\mathbf{x}) = \mathbf{x}^T A_1 \mathbf{x} + \mathbf{b}_1^T \mathbf{x} + c_1$ and frame 2 has $f_2(\mathbf{x}) = \mathbf{x}^T A_2 \mathbf{x} + \mathbf{b}_2^T \mathbf{x} + c_2$, and if the region moved by displacement $\mathbf{d} = (u, v)^T$, then matching linear coefficients gives:

$$\mathbf{d} = \frac{1}{2} A_1^{-1} (\mathbf{b}_1 - \mathbf{b}_2) \quad (4.2)$$

The displacement is determined by the difference in linear terms, scaled by the curvature structure A_1 . Note the structural similarity to LK: the gradient structure (now encoded in A_1) must be invertible — the same condition as having large eigenvalues in the LK structure tensor.

4.3 OpenCV HSV Visualization Implementation

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5
6 def compute_dense_flow_farneback(
7     frame1: np.ndarray,
8     frame2: np.ndarray
9 ) -> np.ndarray:
10     """
11     Compute dense optical flow using Farneback's polynomial expansion
12     method.
13     Returns flow for EVERY pixel in the image (compare: LK at ~200 points)
14     .
15
16     Args:
17         frame1, frame2: Grayscale uint8 images, shape (H, W)
18
19     Returns:
20         flow: Shape (H, W, 2)
21             flow[:, :, 0] = u (horizontal displacement per pixel)
22             flow[:, :, 1] = v (vertical displacement per pixel)

```

```

21     """
22     flow = cv2.calcOpticalFlowFarneback(
23         prev=frame1,
24         next=frame2,
25         flow=None,          # None = compute fresh; pass existing flow to
warm-start
26         pyr_scale=0.5,      # Each pyramid level shrinks by 50%; standard
value
27         levels=3,           # Number of pyramid levels; more = handles
larger displacements
28         winsize=15,         # Window size for polynomial fitting; larger =
smoother
29         iterations=3,       # Iterations at each pyramid level; more = more
accurate
30         poly_n=5,           # Neighborhood for polynomial expansion (5 or 7
recommended)
31         poly_sigma=1.2,     # Gaussian sigma for polynomial fitting weight
32         flags=0              # Optional: cv2.OPTFLOW_USE_INITIAL_FLOW to warm
-start
33     )
34     return flow             # shape (H, W, 2)
35
36
37 def flow_to_hsv_visualization(flow: np.ndarray) -> np.ndarray:
38     """
39     Convert optical flow to an HSV color image for visualization.
40
41     This is the STANDARD way to visualize dense flow, used in all major
papers.
42     Encoding:
43     HUE    (color):    direction of flow (0=right, 90=down, 180=left,
270=up)
44     SATURATION:    always maximum (1.0) for vivid colors
45     VALUE (brightness): magnitude of flow (bright=fast, dark=stationary)
46
47     WHY HSV? A color wheel naturally encodes all directions without
discontinuities. Researchers use this convention universally.
48     Red=right, Green=down, Cyan=left, Blue=up (approximately).
49
50
51     Args:
52         flow: Shape (H, W, 2), float32 flow field
53
54     Returns:
55         bgr_image: Shape (H, W, 3), uint8 BGR image for display/saving
56     """
57     h, w = flow.shape[:2]
58     u = flow[:, :, 0]    # horizontal flow
59     v = flow[:, :, 1]    # vertical flow
60
61     # Compute magnitude (speed) and angle (direction) of flow vectors
62     magnitude, angle = cv2.cartToPolar(u, v)

```

```

63     # magnitude[y,x] = sqrt(u^2 + v^2) -- flow speed
64     # angle[y,x]     = atan2(v, u)      -- flow direction in radians
65
66     hsv = np.zeros((h, w, 3), dtype=np.float32)
67
68     # HUE: direction angle -> [0, 180] (OpenCV uint8 HSV uses 0-180 range)
69     hsv[:, :, 0] = angle * 180 / np.pi / 2
70
71     # SATURATION: always 1.0 (fully saturated)
72     hsv[:, :, 1] = 1.0
73
74     # VALUE: normalize magnitude to [0, 1] using 99th percentile
75     # (avoids a single outlier washing out the entire visualization)
76     max_mag = np.percentile(magnitude, 99)
77     if max_mag > 0:
78         hsv[:, :, 2] = np.clip(magnitude / max_mag, 0, 1)
79
80     # Convert float HSV to uint8 BGR for display
81     hsv_uint8 = (hsv * np.array([180, 255, 255])).astype(np.uint8)
82     bgr = cv2.cvtColor(hsv_uint8, cv2.COLOR_HSV2BGR)
83     return bgr
84
85
86 def compare_sparse_vs_dense(frame1: np.ndarray, frame2: np.ndarray):
87     """
88     Side-by-side comparison of LK (sparse) and Farneback (dense) flow.
89
90     Illustrates the fundamental tradeoff:
91     - Sparse: fewer vectors, only at trackable features, robust
92     - Dense: vectors at every pixel, noisier in texture-less regions
93     """
94     gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
95     gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)
96
97     # Dense flow (every pixel)
98     dense_flow = compute_dense_flow_farneback(gray1, gray2)
99     dense_vis = flow_to_hsv_visualization(dense_flow)
100
101     # Sparse flow (only corners)
102     features = cv2.goodFeaturesToTrack(gray1, 200, 0.01, 10)
103     if features is not None:
104         features_new, status, _ = cv2.calcOpticalFlowPyrLK(
105             gray1, gray2, features, None, winSize=(15,15), maxLevel=3
106         )
107         sparse_vis = frame1.copy()
108         ok = status.flatten() == 1
109         for new, old in zip(features_new[ok], features[ok]):
110             a, b = new.ravel().astype(int)
111             c, d = old.ravel().astype(int)
112             cv2.arrowedLine(sparse_vis, (c,d), (a,b), (0,255,0), 2)
113

```

```

114     fig, axes = plt.subplots(1, 3, figsize=(15, 5))
115     axes[0].imshow(cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB))
116     axes[0].set_title('Frame 1 (Reference)', fontsize=12); axes[0].axis('
off')
117     axes[1].imshow(cv2.cvtColor(sparse_vis, cv2.COLOR_BGR2RGB))
118     axes[1].set_title(f'Sparse LK\n({ok.sum()} tracked points)', fontsize
=12)
119     axes[1].axis('off')
120     axes[2].imshow(cv2.cvtColor(dense_vis, cv2.COLOR_BGR2RGB))
121     axes[2].set_title(f'Dense Farneback\n({gray1.size:,} pixels)',
fontsize=12)
122     axes[2].axis('off')
123     plt.tight_layout()
124     plt.savefig('sparse_vs_dense_flow.png', dpi=150, bbox_inches='tight')
125     plt.show()
126
127     u, v = dense_flow[:, :, 0], dense_flow[:, :, 1]
128     mag = np.sqrt(u**2 + v**2)
129     print(f"Dense flow stats: mean={mag.mean():.2f}px, "
130           f"max={mag.max():.2f}px, "
131           f"pixels>1px={100*(mag>1).mean():.1f}%")

```

Listing 4.1: Dense Farneback optical flow with HSV color wheel visualization

Chapter 4 Summary: Farneback Dense Flow

- Dense flow computes (u, v) at every pixel, unlike sparse LK which tracks only corners.
- **Polynomial expansion** approximates each pixel's neighborhood as a quadratic surface, making matching more robust than raw intensity differences.
- Flow is estimated by matching polynomial coefficients across frames: $\mathbf{d} = \frac{1}{2}A_1^{-1}(\mathbf{b}_1 - \mathbf{b}_2)$.
- **HSV color encoding** is the universal flow visualization standard: hue=direction, value=magnitude.
- Dense flow is slower and noisier in homogeneous regions but provides complete spatial coverage essential for segmentation and action recognition.
- OpenCV's `calcOpticalFlowFarneback()` exposes key parameters: pyramid levels, window size, polynomial neighborhood, iterations.

Connection to Your Background

Farneback's polynomial expansion is conceptually related to the feature extraction you perform in CNN architectures. Just as a CNN learns to represent local image patches as high-dimensional feature vectors, Farneback represents local patches as quadratic polynomial coefficients. The key difference: CNN features are learned from data; Farneback's coefficients are computed analytically. This distinction — learned vs. analytical representations — is the central theme of the deep learning chapters that follow.

Research Questions to Explore

1. **Spatial coherence assumptions:** LK assumes constant flow over a window; Farnebäck assumes polynomial structure over a neighborhood. Both are smoothness priors. What other prior structures over flow fields have been proposed, and what types of motion do they better capture?
2. **Density-accuracy tradeoff:** Dense flow gives more information but is noisier. What principled methods exist for combining sparse and dense estimates — e.g., using sparse LK as seed points and dense Farnebäck for interpolation?
3. **Flow for video compression:** H.264 motion estimation is similar to optical flow but uses block matching. How does block-matching differ from Farnebäck, and why might a sub-optimal (coarser) flow estimate be preferable in a compression setting?

Chapter 5

Horn-Schunck: Global Optical Flow

Horn and Schunck asked: what if we constrain not just one pixel, but the entire flow field at once?

5.1 Local vs. Global Approaches

Lucas-Kanade and Farnebäck are **local methods**: the flow at pixel (x, y) is estimated using only a small spatial neighborhood. Horn and Schunck [?] took a fundamentally different approach: formulate flow estimation as a global energy minimization over the entire image. This is a **variational** approach.

Intuition First

Intuition: Instead of saying “flow is constant in this small window,” Horn-Schunck says “the flow field should be smooth everywhere, with sharp changes only at motion boundaries.” This global, regularization-based view is enormously influential — it introduced the concept of regularized optical flow that remains central to modern deep learning methods.

5.2 The Variational Formulation

Horn-Schunck defines a global energy functional over the entire flow field:

$$E(u, v) = \underbrace{\iint_{\Omega} (I_x u + I_y v + I_t)^2 dx dy}_{\text{Data term: brightness constancy}} + \lambda \underbrace{\iint_{\Omega} (\|\nabla u\|^2 + \|\nabla v\|^2) dx dy}_{\text{Regularization: smoothness}} \quad (5.1)$$

The data term penalizes violations of brightness constancy — the squared OFCE residual integrated over the whole image.

The regularization term penalizes spatial variation in the flow field. $\|\nabla u\|^2 = u_x^2 + u_y^2$ measures how rapidly u changes spatially. Large values mean sharp spatial transitions — the regularizer discourages this.

The weight λ controls the balance:

- Large λ : strong smoothness, uniform flow, may miss genuine motion boundaries
- Small λ : weak smoothness, noisy flow following every spurious brightness change

This is the bias-variance tradeoff applied to flow fields. Recognize the structural analogy: L2 weight decay in neural networks penalizes large parameter magnitudes; Horn-Schunck's regularizer penalizes large flow gradients. Same principle, different domain.

5.3 Euler-Lagrange Equations and Iterative Solution

Taking the functional derivative and setting it to zero yields the Euler-Lagrange equations:

$$I_x(I_x u + I_y v + I_t) - \lambda \Delta u = 0 \quad (5.2)$$

$$I_y(I_x u + I_y v + I_t) - \lambda \Delta v = 0 \quad (5.3)$$

where $\Delta u = u_{xx} + u_{yy}$ is the Laplacian. These equations couple every pixel to its neighbors via the Laplacian, requiring a global solution. The standard iterative fixed-point scheme is:

$$u^{n+1} = \bar{u}^n - I_x \frac{I_x \bar{u}^n + I_y \bar{v}^n + I_t}{\lambda + I_x^2 + I_y^2}, \quad v^{n+1} = \bar{v}^n - I_y \frac{I_x \bar{u}^n + I_y \bar{v}^n + I_t}{\lambda + I_x^2 + I_y^2} \quad (5.4)$$

where $\bar{u}^n$ is the local average of u^n over the 4-connected neighborhood — the discrete Laplacian.

Definition

Beautiful structure of the update rule (Eq. 5.4): The new flow at each pixel is the neighborhood average (smoothness prior), corrected by a term proportional to how much the current estimate violates brightness constancy (data term). The correction is scaled by $1/(\lambda + I_x^2 + I_y^2)$: strong gradients lead to larger corrections (we trust brightness constancy more there); flat regions defer mostly to the smoothness prior.

5.4 Horn-Schunck vs. Lucas-Kanade

Property	Lucas-Kanade	Horn-Schunck
Approach	Local (window-based)	Global (whole image)
Constraint type	Constant flow over window	Smooth flow everywhere
Result	Sparse (at corners)	Dense everywhere
Motion boundaries	Sharp (independent windows)	Blurry (smoothness diffuses)
Computational cost	Efficient (per-point parallel)	Iterative (global coupling)
Historical influence	Tracking, feature matching	Variational methods, energy minimization

5.5 Legacy and Influence

Horn-Schunck’s variational framework is the ancestor of nearly all modern dense flow methods. The energy minimization structure — data term + regularization — appears in TV-L1 flow [?], Brox’s flow [?], and in the loss functions of deep learning flow networks. When you see a regularized deep learning loss:

$$\mathcal{L} = \|\mathbf{f}_{pred} - \mathbf{f}_{GT}\|_1 + \lambda \|\nabla \mathbf{f}_{pred}\|_1 \quad (5.5)$$

the second term is directly descended from Horn-Schunck’s smoothness regularizer. The “data fidelity + regularization” framework Horn-Schunck introduced appears in L2 regularization in neural networks, LASSO in statistics, and Tikhonov regularization in inverse problems. You are always trading off “fit to the data” against “fit to a prior belief about the solution.”

Chapter 5 Summary: Horn-Schunck Global Flow

- Horn-Schunck formulates flow as global energy minimization: data fidelity + smoothness regularization.
- The energy functional balances brightness constancy against spatial smoothness, controlled by λ .
- The Euler-Lagrange equations yield an iterative update: each pixel’s flow moves toward its neighbors, corrected for brightness inconsistency.
- Limitation: over-smooths flow at genuine motion boundaries.
- Profound legacy: introduced the variational framework that influenced all subsequent dense flow methods and modern deep learning loss functions.

Connection to Your Background

The regularization weight λ in Horn-Schunck is structurally identical to the weight decay parameter in your neural network training. In both cases, you add a penalty to encourage a simpler solution (smooth flow / small weights). The iterative update rule (Equation 5.4) is also reminiscent of gradient descent: each step moves the estimate toward both data consistency and prior consistency. This is explicitly variational inference under a Gaussian prior over the flow field gradient.

Research Questions to Explore

1. **Choosing λ :** Horn-Schunck’s performance depends critically on λ . What principled methods exist for adaptive selection — e.g., based on local image statistics or cross-validation? How does this relate to adaptive regularization in deep learning?
2. **From L^2 to L^1 smoothness:** The squared gradient term $\|\nabla u\|^2$ penalizes all flow variations equally, including legitimate motion boundaries. TV (Total Variation) methods use $\|\nabla u\|_1$ instead, which can preserve sharp edges. Why does the L^1 norm preserve edges while L^2 does not? How does this connect to L1 vs. L2 regularization in neural networks?
3. **Anisotropic diffusion:** The Horn-Schunck smoothness term diffuses flow isotropically. Modern variants use anisotropic diffusion that respects image

edges — diffusing along edges but not across them. How would you incorporate image edge information into the regularizer?

Chapter 6

Why Classical Methods Fail: The Motivation for Deep Learning

Understanding failure modes is not defeatism — it is the scientific method.

6.1 The Case Against Classical Methods

Classical optical flow methods — Lucas-Kanade, Farnebäck, Horn-Schunck — are elegant, mathematically principled, and remarkably effective in controlled conditions. But as computer vision confronted the complexity of real-world scenes, their limitations became systematically apparent. This chapter catalogs those failures with precision, because understanding *why* classical methods fail is what motivates every design decision in the deep learning methods of subsequent chapters.

6.2 Failure Mode 1: Large Displacements

Root cause: The first-order Taylor approximation is only valid for small (u, v) . As shown in Chapter 2, error grows rapidly beyond 1–2 pixels.

Pyramidal mitigation: Image pyramids extend the valid range to roughly ± 10 –20 pixels at full resolution. But on the KITTI benchmark [?], approximately 35% of pixels have displacements > 10 pixels. Pyramid LK with 3 levels handles up to ~ 8 pixels per level; many pixels fall outside this range.

6.3 Failure Mode 2: Illumination Changes

Root cause: BCA states $I(x, y, t) = I(x + u, y + v, t + 1)$. This breaks when:

- A car drives past a streetlight, altering shadow patterns
- A camera with auto-exposure adjusts gain between frames
- Specular reflections change with viewpoint (wet road, glossy car hood)

- Outdoor scenes at dawn or dusk under changing natural lighting

Gradient constancy is more robust to global illumination changes but fails for spatially varying illumination.

6.4 Failure Mode 3: Occlusion

Root cause: When an object moves, it uncovers background pixels that were not visible in frame 1. For these pixels, there is no corresponding point — the BCA is meaningless. Classical methods have no explicit mechanism for detecting occlusion; the OFCE at an occluded pixel produces a nonsensical flow vector with no signal to the algorithm that something is wrong. Forward-backward consistency checking helps but is a post-hoc heuristic.

6.5 Failure Mode 4: Texture-Less Regions

Root cause: In a uniform intensity region, $I_x = I_y = 0$ everywhere. The structure tensor $M = 0$ (the zero matrix), completely uninvertible. There is literally zero gradient information with which to estimate flow.

Real examples: painted car hoods, the sky, painted walls, sections of road without lane markings. Deep learning methods address this by: (1) using learned feature representations that are rich even in visually flat regions, and (2) propagating flow estimates from textured regions to texture-less neighbors through implicit spatial regularization.

6.6 Summary and Motivation for Deep Learning

Table 6.1: Prevalence of classical method failure modes in KITTI driving sequences

Failure Mode	Approx. % pixels	Root Mechanism
Large displacement ($> 10\text{px}$)	35%	Taylor linearization breakdown
Occlusion boundaries	8–12%	No valid correspondence exists
Texture-less regions	15–25%	Zero gradient, $M \approx 0$
Illumination artifacts	Scene- dependent	BCA violated
Motion blur	5–10%	Shutter speed / fast motion

Each deep learning architecture addresses specific failure modes:

- **FlowNet** (Ch. 8): Learned features beyond hand-crafted gradients; correlation layers for large-displacement matching.
- **PWC-Net** (Ch. 9): Explicit warping at each pyramid level for large displacements.

- **RAFT** (Ch. 10): All-pairs correlation eliminates the small-displacement assumption entirely.
- **FlowFormer** (Ch. 11): Transformer attention over cost volumes handles occlusions and texture-less regions via learned global reasoning.

The progression from LK to FlowFormer is not just a story of “better accuracy” — it is a systematic story of each identified failure mode getting its principled solution.

Chapter 6 Summary: Why Classical Methods Fail

- **Large displacement:** Taylor linearization breaks; pyramids only partially help.
- **Illumination changes:** BCA is violated; gradient constancy partially helps.
- **Occlusion:** No correspondence exists; classical methods produce arbitrary flow.
- **Texture-less regions:** Zero gradient = uninvertible structure tensor = no flow possible.
- These affect 35–50% of pixels in real autonomous driving data — not edge cases.
- Deep learning methods address each failure mode through learned features, explicit warping, all-pairs matching, and attention-based global reasoning.

Connection to Your Background

In your AV project, you likely encountered situations where frame-by-frame detection was inconsistent or objects were momentarily occluded. These are exactly the failure modes cataloged here. The frustrating unreliability of classical methods in real driving scenes is what drove the research community toward learned flow estimation starting with FlowNet in 2015. As you present your comparative study, you can position RAFT’s dramatically better performance on occluded and fast-moving regions as a direct consequence of addressing these specific, nameable failure modes.

Research Questions to Explore

1. **Do deep methods introduce new failure modes?** Deep learning dramatically improves over classical methods. But adversarial patches on road signs can fool detection networks — can they fool flow networks? What would an adversarial perturbation designed to fool RAFT look like mathematically?
2. **Uncertainty quantification:** Classical methods can provide reliability estimates (structure tensor eigenvalues, forward-backward error). Deep flow networks typically output a single flow estimate with no uncertainty. How would you augment a flow network to produce per-pixel confidence estimates? What ground truth would you train against?
3. **The generalization problem:** RAFT trained on synthetic FlyingChairs may not generalize to KITTI without fine-tuning. Which failure modes become worse with distributional shift? Is illumination change a larger or smaller problem for deep methods than for classical ones?

Chapter 7

Deep Learning for Optical Flow: Foundation

The fundamental insight: don't engineer features, learn them. Don't match brightness, match representations.

7.1 The Paradigm Shift

Classical methods estimate flow from hand-crafted features (gradients I_x, I_y, I_t) and hand-designed optimization (least squares, variational energy). This works well when its assumptions hold but fails systematically when they don't.

Deep learning for optical flow makes two radical changes:

1. **Replace hand-crafted features with learned ones:** Instead of computing I_x, I_y , run both frames through a CNN to extract rich feature representations that encode texture, boundaries, semantic content, and other cues no hand-designed gradient can capture.
2. **Replace analytical optimization with learned inference:** Instead of iterating to minimize an energy functional, train a network that directly predicts flow from feature representations.

7.2 What Is a Cost Volume?

The cost volume is the most important architectural concept in deep learning optical flow, appearing in every modern method from FlowNet to RAFT.

The idea: To determine where a pixel in frame 1 went in frame 2, compare its feature representation against feature representations at potential destinations in frame 2. The cost volume encodes this comparison explicitly.

Formally: Let $F_1, F_2 \in \mathbb{R}^{H \times W \times C}$ be feature maps for frames 1 and 2. The cost volume C is:

$$C(x, y, \Delta x, \Delta y) = \langle F_1(x, y), F_2(x + \Delta x, y + \Delta y) \rangle \quad (7.1)$$

$C(x, y, \Delta x, \Delta y)$ is a similarity score: “How similar is the content at (x, y) in frame 1 to the content at $(x + \Delta x, y + \Delta y)$ in frame 2?” High cost = high similarity = good match candidate.

The cost volume is a 4D tensor: two spatial dimensions (x, y) for the source pixel, and two dimensions $(\Delta x, \Delta y)$ for the displacement. For a feature map of size $H' \times W'$ and search range $[-D, D]$, the cost volume has shape $(H' \times W' \times (2D + 1)^2)$.

Intuition First

Why is this more powerful than the OFCE? LK asks: “what is the gradient here?” and solves a local linear system. The cost volume asks: “of all possible destinations within a search range, which is most similar?” This is fundamentally more powerful for handling large displacements and occlusions — you are doing explicit search rather than local linearization.

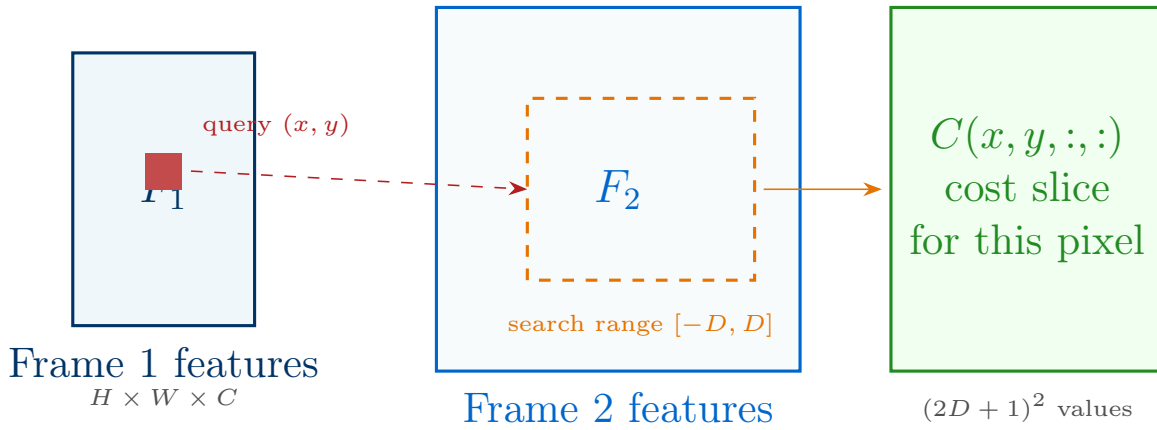


Figure 7.1: Cost volume construction. For each pixel (x, y) in F_1 , we compute similarity (dot product) against all positions within search range $[-D, D]$ in F_2 . The resulting $(2D + 1)^2$ values encode the likelihood of each displacement.

7.3 The Correlation Layer

The correlation layer implements cost volume computation as a differentiable neural network operation:

$$\text{Corr}(F_1, F_2)(\mathbf{x}_1, d) = \sum_{\mathbf{o} \in [-k, k]^2} \langle F_1(\mathbf{x}_1 + \mathbf{o}), F_2(\mathbf{x}_1 + d + \mathbf{o}) \rangle \quad (7.2)$$

for displacement d in a search range $[-D, D]^2$. The neighborhood sum over $\mathbf{o}$ introduces local spatial context. **Connection to convolution:** For a fixed displacement d , computing $\text{Corr}(F_1, F_2)(\cdot, d)$ over all positions is exactly a cross-correlation operation — identical to `F.conv2d` in PyTorch. The correlation layer is a generalization that computes this for *all* displacements simultaneously.

7.4 Encoder-Decoder Architecture for Flow

Most deep flow networks use an encoder-decoder (U-Net [?] -like) architecture:

- **Encoder:** Progressively downsamples both frames, extracting increasingly abstract features. At the bottleneck, feature maps are small but semantically rich.
- **Cost volume:** Computed at multiple pyramid levels, comparing features across frames.
- **Decoder:** Upsamples the flow estimate from coarse to fine, incorporating skip connections from the encoder.

7.5 Training Data for Optical Flow

A critical challenge: *how do you get ground-truth flow?* Unlike detection or segmentation, you cannot ask a human to label per-pixel flow.

Synthetic datasets:

- **FlyingChairs** [?]: Randomly textured 2D chair images on background images, moved with known 2D transformations. Ground truth is exact. ~22,000 training pairs.
- **FlyingThings3D** [?]: 3D objects rendered with realistic lighting and random 3D trajectories. ~21,818 training pairs.
- **MPI-Sintel** [?]: A rendered animated film with Clean and Final passes (atmospheric effects, motion blur). Realistic occlusions and complex motion.

Real-world datasets:

- **KITTI** [?]: Ground truth from LIDAR projection onto image pairs (sparse, partially covering). 200 training pairs. Noisy but real.

Standard training curriculum: FlyingChairs → FlyingThings3D → Sintel/KITTI fine-tuning.

7.6 Loss Functions and Evaluation Metrics

7.6.1 Supervised Loss: EPE

The standard supervised loss is L1 distance between predicted and ground-truth flow:

$$\mathcal{L}_{EPE} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{f}_i^{\text{pred}} - \mathbf{f}_i^{\text{GT}}\|_2 \quad (7.3)$$

7.6.2 Multi-Scale Loss

Architectures producing multi-scale predictions sum EPE losses at each scale with increasing weights toward finer scales:

$$\mathcal{L}_{\text{multi}} = \sum_{\ell=1}^L w_{\ell} \cdot EPE^{(\ell)} \quad (7.4)$$

7.6.3 End-Point Error (EPE) and Fl-all

$$EPE = \frac{1}{|\Omega|} \sum_{(x,y) \in \Omega} \sqrt{(u_{pred} - u_{GT})^2 + (v_{pred} - v_{GT})^2} \quad (7.5)$$

EPE measures the average Euclidean distance (in pixels) between predicted and true flow vectors. Lower is better.

The **F1 outlier metric** counts pixels where the error is both absolutely large ($> 3\text{px}$) AND relatively large ($> 5\%$ of true flow magnitude). **Fl-all** applies this over all pixels, including those without LIDAR ground truth on KITTI, and is the primary KITTI benchmark metric.

Chapter 7 Summary

- Deep learning replaces hand-crafted gradient features with learned CNN features, enabling more robust matching.
- The **cost volume** is the key architectural innovation: encodes similarity between feature maps at all candidate displacements.
- The **correlation layer** implements cost volume computation as a differentiable neural network operation.
- Encoder-decoder architectures with cost volumes at each scale are the standard blueprint.
- Training data is predominantly synthetic (FlyingChairs, FlyingThings3D, Sintel) with real-world fine-tuning on KITTI.
- Primary metrics: EPE (average pixel error) and Fl-all (fraction of bad pixels).

Connection to Your Background

You have implemented encoder-decoder architectures and understand skip connections and multi-scale feature extraction from your CNN/ResNet work. The deep flow architecture is exactly this structure, with one key addition: the cost volume computed from *two* feature pyramids rather than one. Your understanding of how skip connections propagate fine-grained spatial information from encoder to decoder is directly applicable here. Additionally, the cost volume dot product $\langle F_1(\mathbf{x}), F_2(\mathbf{x}') \rangle$ is structurally identical to the QK^T computation in the Transformer attention you implemented for your Conformer — this connection becomes critical in Chapter 11.

Research Questions to Explore

1. **The cost of the cost volume:** For a feature map of size $H \times W$ and search range D , the cost volume has $H \cdot W \cdot (2D + 1)^2$ entries. RAFT uses all-pairs correlation (no bounded range) — how does this scale differently, and what computational tricks make it feasible?
2. **Contrastive learning for flow:** The cost volume does nearest-neighbor matching in feature space. Could we train the feature extractor with a contrastive loss (SimCLR/MoCo) to maximize similarity between corresponding pixels across frames? What would be the advantages and challenges vs. supervised EPE loss?

3. **Unsupervised flow training:** We could train a flow network without ground-truth labels by minimizing photometric error: $\|I_1 - W(I_2; \hat{\mathbf{f}})\|$ where W is a differentiable warping function. What are the failure modes of this unsupervised approach? How does it compare to supervised methods on KITTI?

Chapter 8

FlowNet: The Pioneer

FlowNet proved that end-to-end learning works for optical flow. The field was never the same.

8.1 The 2015 Revolution

In 2015, Dosovitskiy et al. [?] asked a simple but audacious question: can we train a deep convolutional network to estimate optical flow directly from image pairs? The answer was yes. FlowNet introduced two variants: **FlowNetS** (Simple) and **FlowNetC** (Correlation).

For a beginner, the important historical point is this: before FlowNet, optical flow was mostly treated as an optimization problem. We wrote down assumptions such as brightness constancy, smoothness, and small motion, then solved equations. FlowNet changed the framing. It said: if we have enough examples of image pairs and their correct flow fields, a CNN can learn the mapping

$$(I_1, I_2) \mapsto \mathbf{f} \tag{8.1}$$

directly. This does not mean the old mathematics disappeared. Instead, the network learned many useful matching and regularization behaviors from data.

Intuition First

Beginner intuition: Imagine showing a student thousands of before-and-after image pairs and also showing the correct arrows for how every pixel moved. At first, the student guesses badly. After many examples, the student learns visual patterns: cars move as coherent objects, nearby road pixels often have similar motion, object boundaries are special, and large displacements require matching similar-looking regions far apart. FlowNet is this student, implemented as a convolutional neural network.

FlowNet mattered for three beginner-friendly reasons:

1. **End-to-end learning:** the input is two images and the output is a dense flow field. There is no separate hand-designed corner detector, matching rule, or variational solver.

2. **Learned features:** instead of comparing raw brightness values, the network learns feature maps that are better for matching under appearance changes.
3. **A new research direction:** later methods such as PWC-Net, RAFT, and FlowFormer kept improving the same core idea: represent matching evidence with learned features and refine it into accurate flow.

The limitation was also clear: FlowNet was not yet as accurate as later specialized architectures. Its value was proving that optical flow could be learned by a deep network at all.

8.2 FlowNetS: The Simple Architecture

FlowNetS takes the most straightforward approach: concatenate both frames channel-wise into a 6-channel input, feed through a CNN, and predict flow. By concatenating frames as channels, the network is forced to learn to match pixels across channels using *learned features* rather than raw brightness.

The name FlowNetS means “FlowNet Simple.” It is simple because the two images are not processed separately. If each RGB image has 3 channels, then stacking two RGB images gives a tensor with 6 channels:

$$I_{\text{stack}} = I_1 \oplus I_2 \in \mathbb{R}^{H \times W \times 6}. \quad (8.2)$$

The CNN receives this stacked tensor and must internally discover which parts of image 1 correspond to which parts of image 2.

FlowNetS in One Sentence

FlowNetS is an encoder–decoder CNN that takes two frames stacked together as input and directly predicts a two-channel image, where channel 1 is horizontal flow u and channel 2 is vertical flow v .

A beginner should read the architecture in three stages:

1. **Input stacking:** the network sees both frames at the same time as a 6-channel image. This lets early convolution filters compare local patterns across the two frames.
2. **Encoder/contracting path:** the blue blocks progressively reduce spatial resolution while increasing semantic abstraction. This helps the network understand larger motion patterns, but fine pixel detail is lost.
3. **Decoder/expanding path:** the green blocks upsample the representation back toward image resolution and produce dense flow. Skip connections pass fine detail from encoder layers to decoder layers so boundaries are not completely lost.

The output has exactly two channels because every pixel needs two numbers: u for left/right motion and v for up/down motion. If the output at pixel (x, y) is $(u, v) = (5, -2)$, that means the pixel appears to move 5 pixels right and 2 pixels up from frame 1 to frame 2.

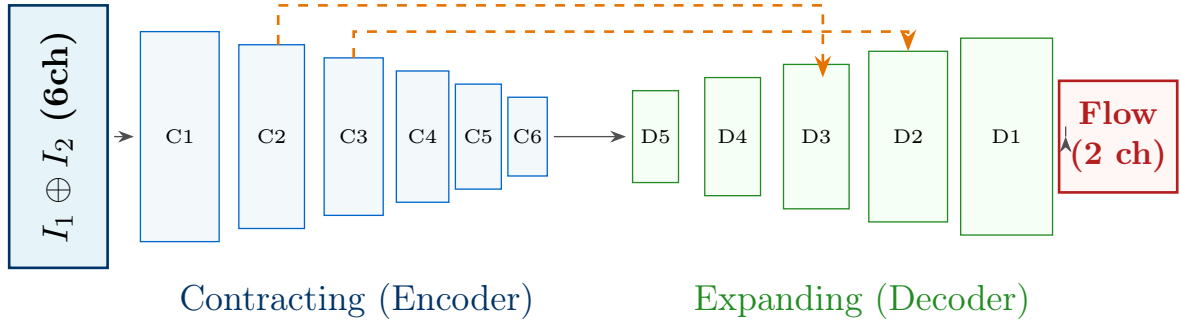


Figure 8.1: FlowNetS architecture. Both frames concatenated as 6-channel input; processed by encoder-decoder CNN. Skip connections (orange dashed) carry fine-grained spatial information from encoder to decoder.

8.3 FlowNetC: Explicit Correlation

FlowNetC adds an explicit correlation layer between two separate encoder paths.

FlowNetS asks the network to learn matching implicitly from stacked images. FlowNetC makes matching more explicit. It first sends frame 1 and frame 2 through two CNN encoders with shared weights, producing feature maps F_1 and F_2 . Then it compares features from frame 1 against nearby features from frame 2 using a correlation layer.

Intuition First

Beginner intuition: Suppose you cut a small patch from frame 1 and slide it over frame 2, asking: “Where does this patch look most similar?” Classical block matching does this with raw pixels. FlowNetC does a learned version of the same idea: it compares CNN feature vectors instead of raw pixel patches. The correlation layer produces a table of similarity scores for possible displacements.

Mathematically, for a feature vector at position (x, y) in frame 1, FlowNetC computes scores like

$$C(x, y, \Delta x, \Delta y) = \langle F_1(x, y), F_2(x + \Delta x, y + \Delta y) \rangle, \quad (8.3)$$

where $\langle \cdot, \cdot \rangle$ is a dot product. A high score means: “the feature at this displacement looks like a good match.”

The key difference between FlowNetS and FlowNetC is therefore:

- **FlowNetS:** lets the CNN discover matching by itself from concatenated images.
- **FlowNetC:** gives the CNN an explicit matching layer, making correspondence easier to learn.

After correlation, FlowNetC concatenates the correlation scores with frame-1 features and sends them through additional convolution and decoder layers. The decoder converts matching evidence into the final dense flow field.

Common Pitfall

A common beginner mistake is to think the correlation layer directly outputs optical flow. It does not. It outputs *matching evidence*. The later decoder still has to interpret that evidence, combine it with image context, smooth it, and produce the final (u, v) vector at every pixel.

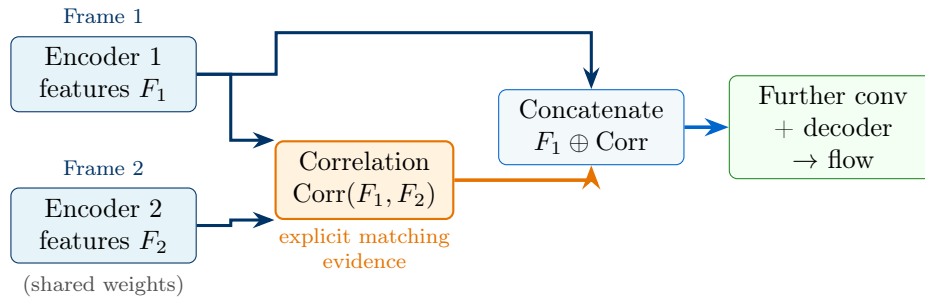


Figure 8.2: FlowNetC architecture. Separate encoders (shared weights) process each frame. An explicit correlation layer computes cross-frame similarities before further processing.

8.4 Conceptual PyTorch Implementation

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5
6 class CorrelationLayer(nn.Module):
7     """
8     Implements the correlation layer from FlowNetC.
9
10    For each source position (x,y) in F1, computes similarity with all
11    positions within displacement range D in F2.
12
13    This is the key innovation of FlowNetC: instead of forcing the network
14    to implicitly learn matching through concatenation (FlowNetS), the
15    correlation layer EXPLICITLY computes cross-frame similarity.
16
17    Connection to attention: this is  $\text{QK}^T$  without the softmax.
18    F1 features are the "query", F2 features are the "keys".
19    The output is the raw similarity (attention logits) for each
20    displacement.
21
22    Mathematical operation:
23     $C(x, y, dx, dy) = \langle F1(x,y), F2(x+dx, y+dy) \rangle$ 
24    for  $dx, dy$  in  $[-D, D]$ 
25
26    Output shape: (B, (2D+1)^2, H, W)
27    -- one channel per possible displacement vector
  
```

```

27     """
28
29     def __init__(self, max_displacement: int = 4, stride: int = 1):
30         """
31         Args:
32             max_displacement: Search range D in pixels
33             stride: Step size when iterating over displacements
34         """
35         super().__init__()
36         self.D = max_displacement
37         self.stride = stride
38
39     def forward(self, f1: torch.Tensor, f2: torch.Tensor) -> torch.Tensor:
40         """
41         Args:
42             f1: Features from frame 1, shape (B, C, H, W)
43             f2: Features from frame 2, shape (B, C, H, W)
44
45         Returns:
46             corr: Correlation volume, shape (B, (2D+1)^2, H, W)
47         """
48         B, C, H, W = f1.shape
49         D, s = self.D, self.stride
50
51         # Pad f2 so we can sample displaced positions without boundary
52         issues
53         f2_padded = F.pad(f2, [D, D, D, D])
54
55         # L2-normalize features for more stable dot products
56         # Makes the correlation a cosine similarity in [-1, 1]
57         f1_norm = F.normalize(f1, p=2, dim=1)
58         f2_pad_norm = F.normalize(f2_padded, p=2, dim=1)
59
60         corr_channels = []
61         for dy in range(-D, D + 1, s):
62             for dx in range(-D, D + 1, s):
63                 # Extract f2 shifted by (dx, dy)
64                 # After padding, original position (x,y) is at (x+D, y+D)
65                 y_start = D + dy
66                 x_start = D + dx
67                 f2_shifted = f2_pad_norm[:, :, y_start:y_start+H, x_start:
68                 x_start+W]
69
70                 # Dot product along channel dim: cosine similarity per
71                 spatial pos
72                 # (f1_norm * f2_shifted) shape: (B, C, H, W)
73                 corr_channel = (f1_norm * f2_shifted).sum(dim=1, keepdim=
74                 True)
75
76                 # corr_channel shape: (B, 1, H, W)
77                 corr_channels.append(corr_channel)

```

```

74         # Stack all displacement channels together
75         corr = torch.cat(corr_channels, dim=1)
76         # corr shape: (B, (2D//s+1)^2, H, W)
77         # corr[:, k, y, x] = cosine similarity between F1(x,y) and F2(x+dx
, y+dy)
78         # where (dx, dy) is the k-th displacement in the search grid
79         return corr
80
81
82 class FlowNetEncoder(nn.Module):
83     """
84     Shared feature encoder for FlowNetC.
85     Applied independently to each frame with SHARED WEIGHTS.
86
87     Shared weights = siamese architecture:
88     forces both frames into the SAME feature space,
89     making the correlation layer's dot product meaningful.
90
91     If the encoders had different weights, F1 and F2 features would be
92     in different spaces and their dot product would be meaningless.
93
94     Architecture: 6 conv blocks, stride-2 downsampling at levels 1-3,
95     giving 8x total spatial downsampling (1/8 resolution at output).
96     """
97
98     def __init__(self, in_channels: int = 3):
99         super().__init__()
100
101         def conv_block(in_ch, out_ch, stride=2, k=3):
102             """Conv2d -> BatchNorm -> LeakyReLU block"""
103             return nn.Sequential(
104                 nn.Conv2d(in_ch, out_ch, kernel_size=k,
105                           stride=stride, padding=k//2, bias=False),
106                 nn.BatchNorm2d(out_ch),
107                 nn.LeakyReLU(0.1, inplace=True)
108             )
109
110         # Contracting path: each stride=2 halves spatial dimensions
111         self.conv1 = conv_block(in_channels, 64, stride=2) # -> 1/2 res
112         self.conv2 = conv_block(64, 128, stride=2)         # -> 1/4
113     res
114         self.conv3 = conv_block(128, 256, stride=2)         # -> 1/8
115     res
116         self.conv3_1 = conv_block(256, 256, stride=1)       # -> 1/8
117     res (refine)
118         self.conv4 = conv_block(256, 512, stride=2)         # -> 1/16
119     res
120         self.conv4_1 = conv_block(512, 512, stride=1)
121         self.conv5 = conv_block(512, 512, stride=2)         # -> 1/32
122     res
123         self.conv5_1 = conv_block(512, 512, stride=1)

```

```

119         self.conv6 = conv_block(512, 1024, stride=2)           # -> 1/64
120     res
121
122     def forward(self, x: torch.Tensor):
123         """
124         Returns feature maps at all scales for decoder skip connections.
125
126         In FlowNetC, we use the 1/8 resolution features (c3) for
127         correlation,
128         because this scale has enough spatial resolution to localize flow
129         and enough channel depth to be discriminative for matching.
130         """
131         c1 = self.conv1(x)           # (B, 64, H/2, W/2)
132         c2 = self.conv2(c1)          # (B, 128, H/4, W/4)
133         c3 = self.conv3_1(self.conv3(c2)) # (B, 256, H/8, W/8) <-
134         used for corr
135         c4 = self.conv4_1(self.conv4(c3)) # (B, 512, H/16, W/16)
136         c5 = self.conv5_1(self.conv5(c4)) # (B, 512, H/32, W/32)
137         c6 = self.conv6(c5)           # (B, 1024, H/64, W/64)
138         return c1, c2, c3, c4, c5, c6
139
140     class FlowNetC(nn.Module):
141         """
142         Complete FlowNetC model.
143
144         Pipeline:
145         1. Extract features from both frames (shared encoder)
146         2. Compute cross-frame correlation at 1/8 resolution
147         3. Decode from coarse to fine with skip connections
148         """
149
150         def __init__(self, max_displacement: int = 4):
151             super().__init__()
152             self.encoder = FlowNetEncoder(in_channels=3)
153             self.correlation = CorrelationLayer(max_displacement=
154             max_displacement)
155
156             D = max_displacement
157             corr_channels = (2 * D + 1) ** 2    # = 81 for D=4
158
159             # After correlation, reduce F1 channel count before concatenation
160             # to keep the combined representation manageable
161             self.f1_reduce = nn.Sequential(
162                 nn.Conv2d(256, 32, kernel_size=1, bias=False),
163                 nn.LeakyReLU(0.1, inplace=True)
164             )
165             # combined = corr (81ch) + f1_reduced (32ch) = 113 channels
166
167             # Additional convolutions before decoder
168             self.post_corr = nn.Sequential(

```

```

166         nn.Conv2d(corr_channels + 32, 256, 3, padding=1, bias=False),
167         nn.BatchNorm2d(256),
168         nn.LeakyReLU(0.1, inplace=True),
169         nn.Conv2d(256, 256, 3, padding=1, bias=False),
170         nn.BatchNorm2d(256),
171         nn.LeakyReLU(0.1, inplace=True),
172     )
173
174     # Upconvolution layers: each increases spatial resolution 2x
175     # Input: features from previous scale + skip from encoder +
predicted flow
176     self.upconv5 = nn.ConvTranspose2d(1024, 512, 4, stride=2, padding
=1)
177     self.upconv4 = nn.ConvTranspose2d(512 + 512 + 2, 256, 4, stride=2,
padding=1)
178     self.upconv3 = nn.ConvTranspose2d(256 + 256 + 2, 128, 4, stride=2,
padding=1)
179     self.upconv2 = nn.ConvTranspose2d(128 + 128 + 2, 64, 4, stride=2,
padding=1)
180
181     # Flow predictors at each scale: 2-channel output (u, v)
182     self.flow5 = nn.Conv2d(512, 2, 3, padding=1)
183     self.flow4 = nn.Conv2d(256, 2, 3, padding=1)
184     self.flow3 = nn.Conv2d(128, 2, 3, padding=1)
185     self.flow2 = nn.Conv2d(64, 2, 3, padding=1)
186     self.flow1 = nn.Conv2d(64, 2, 3, padding=1) # finest scale
187
188     def forward(self, frame1: torch.Tensor, frame2: torch.Tensor) -> list:
189         """
190         Args:
191             frame1, frame2: Shape (B, 3, H, W), normalized to mean=0, std
=1
192
193         Returns:
194             flow_predictions: List of flow predictions, coarsest to finest
.
195
196             Each shape (B, 2, H_1, W_1).
197             Used for multi-scale loss during training;
198             final element is the full-resolution output.
199         """
200         # --- 1. Extract features (shared encoder, applied to both frames)
201         ---
202         feat1 = self.encoder(frame1) # tuple: (c1, c2, c3, c4, c5, c6)
203         feat2 = self.encoder(frame2)
204
205         c1_1, c2_1, c3_1, c4_1, c5_1, c6_1 = feat1
206         _, _, c3_2, _, _, _ = feat2 # We only need 1/8 res features of
frame2
207
208         # --- 2. Correlation at 1/8 resolution ---
209         # c3_1 and c3_2 are both (B, 256, H/8, W/8)

```

```

208     corr = self.correlation(c3_1, c3_2)    # (B, 81, H/8, W/8)
209
210     # Reduce F1 features and concatenate with correlation
211     f1_r = self.f1_reduce(c3_1)            # (B, 32, H/8, W/8)
212     combined = torch.cat([corr, f1_r], dim=1) # (B, 113, H/8, W/8)
213     combined = self.post_corr(combined)    # (B, 256, H/8, W/8)
214
215     # --- 3. Decode from coarse to fine ---
216     flow_preds = []
217
218     # Start from bottleneck features (1/64 resolution)
219     up5 = F.leaky_relu(self.upconv5(c6_1), 0.1)    # (B, 512, H
/32, W/32)
220     flow5_pred = self.flow5(up5)                # (B, 2, H
/32, W/32)
221     flow_preds.append(flow5_pred)
222
223     # Concatenate: upsampled features + encoder skip + upsampled flow
prediction
224     flow5_up = F.interpolate(flow5_pred, scale_factor=2,
225                               mode='bilinear', align_corners=True)
226     up4_in = torch.cat([up5, c5_1, flow5_up], dim=1)    # (B,
512+512+2, ...)
227     up4 = F.leaky_relu(self.upconv4(up4_in), 0.1)    # (B, 256, H
/16, W/16)
228     flow4_pred = self.flow4(up4)
229     flow_preds.append(flow4_pred)
230
231     flow4_up = F.interpolate(flow4_pred, scale_factor=2,
232                               mode='bilinear', align_corners=True)
233     up3_in = torch.cat([up4, c4_1, flow4_up], dim=1)
234     up3 = F.leaky_relu(self.upconv3(up3_in), 0.1)
235     flow3_pred = self.flow3(up3)
236     flow_preds.append(flow3_pred)
237
238     flow3_up = F.interpolate(flow3_pred, scale_factor=2,
239                               mode='bilinear', align_corners=True)
240     up2_in = torch.cat([up3, c3_1, flow3_up], dim=1)
241     up2 = F.leaky_relu(self.upconv2(up2_in), 0.1)
242     flow2_pred = self.flow2(up2)
243     flow_preds.append(flow2_pred)
244
245     # Final prediction upsampled to full resolution
246     final_flow = F.interpolate(flow2_pred, scale_factor=4,
247                               mode='bilinear', align_corners=True)
248     flow_preds.append(final_flow)
249
250     return flow_preds
251
252     def compute_loss(self, flow_preds: list,
253                     flow_gt: torch.Tensor,

```

```

254         valid_mask: torch.Tensor = None) -> torch.Tensor:
255     """
256     Multi-scale EPE loss. Finer scales get higher weight.
257
258     Args:
259         flow_preds: List from coarsest to finest, each (B, 2, H_1, W_1
260     )
261         flow_gt:      Ground-truth flow, shape (B, 2, H, W)
262         valid_mask: Boolean mask for valid GT pixels (KITTI has sparse
263     GT)
264
265     Returns:
266         total_loss: Weighted sum of EPE at each scale
267     """
268     # Weights from the FlowNet paper: later (finer) scales get more
269     weight
270     weights = [0.005, 0.01, 0.02, 0.08, 0.32]
271     weights = weights[-len(flow_preds):] # Match to number of
272     predictions
273
274     total_loss = 0.0
275     for pred, w in zip(flow_preds, weights):
276         H_1, W_1 = pred.shape[-2:]
277
278         # Downsample GT to match this prediction scale
279         gt_scaled = F.interpolate(flow_gt, size=(H_1, W_1),
280                                 mode='bilinear', align_corners=True)
281
282         # CRITICAL: scale GT flow values proportionally to the
283         resolution change
284         # A 10-pixel displacement at full res is 1.25px at 1/8 res
285         scale = H_1 / flow_gt.shape[-2]
286         gt_scaled = gt_scaled * scale
287
288         # EPE per pixel: Euclidean distance between pred and GT
289         epe = torch.sqrt(((pred - gt_scaled)**2).sum(dim=1) + 1e-8)
290
291         if valid_mask is not None:
292             valid_s = F.interpolate(valid_mask.float()).unsqueeze(1),
293                             size=(H_1, W_1), mode='nearest').
294             squeeze(1).bool()
295             epe = epe[valid_s]
296
297         total_loss = total_loss + w * epe.mean()
298
299     return total_loss

```

Listing 8.1: FlowNet core building blocks in PyTorch — heavily commented

8.5 Why FlowNet Was Revolutionary

- **Proved end-to-end learning:** The CV community was skeptical. FlowNet proved it could work.
- **Introduced the correlation layer:** This fundamental building block appears in nearly every subsequent method.
- **Established the training procedure:** Synthetic pre-training + real fine-tuning remains standard practice.
- **Speed advantage:** FlowNet was faster than many classical methods at comparable accuracy.

FlowNet 2.0 [?] extended this by stacking multiple FlowNet models, where each refines the residual error of the previous. This introduced iterative flow refinement that RAFT later takes to its logical extreme.

Chapter 8 Summary: FlowNet

- FlowNet (2015): first successful end-to-end deep learning approach to optical flow.
- **FlowNetS:** Both frames concatenated as 6-channel input; simpler but less precise.
- **FlowNetC:** Separate siamese encoders + explicit correlation layer; better cross-frame understanding.
- The correlation layer computes feature similarity at all candidate displacements — differentiable template matching.
- Multi-scale loss (weighted EPE at each decoder level) enables learning across spatial scales.
- FlowNet’s ideas — correlation layer, encoder-decoder, multi-scale supervision — are the foundation on which PWC-Net and RAFT build.

Connection to Your Background

The correlation layer in FlowNetC computes cross-channel dot products between feature maps — structurally identical to the attention computation in Transformers you implemented for your Conformer ($\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d})V$, where QK^T is the same type of dot-product similarity). The correlation layer is essentially attention without the softmax and without the value combination — just the raw similarity scores (attention logits). This structural correspondence between correlation and attention is precisely what FlowFormer exploits in Chapter 11.

Research Questions to Explore

1. **The siamese encoder question:** FlowNetC uses a shared encoder for both frames, forcing both into the same feature space. Is this always desirable? When might frame-specific features be beneficial — e.g., when one frame has a significantly different appearance due to motion blur?

2. **Displacement range and scale:** FlowNetC uses $D = 4$ at $1/8$ resolution, equivalent to ± 32 pixels at full resolution. How does the choice of D affect the accuracy-computation tradeoff? Can you design an adaptive D that is large in dynamic regions and small in static ones?
3. **Residual learning in FlowNet 2.0:** FlowNet 2.0 stacks networks where each refines the residual error of the previous — residual learning at the architecture level. How does this connect to ResNet’s residual connections? What is the optimal number of stacked networks, and how would you determine this without extensive ablation?

Chapter 9

PWC-Net: Pyramid, Warping, Cost Volume

*Three classical ideas, combined in exactly the right way.
That is the art of architecture design.*

9.1 The Three Pillars

PWC-Net [?] was published at CVPR 2018 and achieved state-of-the-art performance while being dramatically more compact than FlowNet 2.0 ($\sim 9\text{M}$ vs. $\sim 160\text{M}$ parameters). The name describes its three key components:

1. **Pyramid:** Learned feature pyramids for coarse-to-fine estimation
2. **Warping:** Use current flow estimate to warp frame 2's features toward frame 1 at each scale
3. **Cost Volume:** Compute correlation between frame 1 features and warped frame 2 features

9.2 Learned Feature Pyramid

Unlike LK's raw image pyramid, PWC-Net builds a **learned feature pyramid**: both frames pass through a shared CNN encoder producing feature maps at 6 different resolutions ($1/2, 1/4, \dots, 1/64$ of input). Learned features encode semantically meaningful information rather than just raw brightness. A feature vector at a corner in a learned pyramid is a rich descriptor of that corner's appearance.

9.3 Warping: The Key Innovation

At each pyramid level ℓ , PWC-Net uses the current flow estimate $\hat{\mathbf{f}}^{\ell+1}$ (from the coarser level, upsampled by 2) to warp frame 2's feature map toward frame 1:

$$\tilde{F}_2^\ell(\mathbf{x}) = F_2^\ell(\mathbf{x} + \uparrow_2 \hat{\mathbf{f}}^{\ell+1}(\mathbf{x})) \quad (9.1)$$

where $\uparrow_2$ denotes upsampling by factor 2, and $F_2^\ell(\cdot)$ is evaluated at non-integer positions using bilinear interpolation.

Intuition First

Intuition: Suppose we estimated at coarse level that a car moved 4 pixels right. At the next finer level, we use this estimate to “pre-shift” frame 2’s features by 4 pixels. The *residual* displacement (the error in our coarse estimate) is likely small — maybe 1–2 pixels. The cost volume at this level only needs to search over small displacements to find the correction. Warping does most of the work; the cost volume handles the residual.

The warping operation is differentiable: gradients flow through bilinear sampling during backpropagation, allowing the network to jointly optimize the feature extractor and flow predictor.

9.4 Level-wise Cost Volume

After warping, PWC-Net computes a local cost volume between frame 1’s features and the warped frame 2’s features:

$$CV^\ell(\mathbf{x}, d) = \langle F_1^\ell(\mathbf{x}), \tilde{F}_2^\ell(\mathbf{x} + d) \rangle, \quad d \in [-D', D']^2 \quad (9.2)$$

Crucial difference from FlowNetC: Because frame 2 has been warped to approximately align with frame 1, the search range D' can be much smaller (typically $D' = 4$). The warping does most of the work; the cost volume handles the residual. This leads to a much smaller, more efficient cost volume.

9.5 Warping Implementation

```

1 import torch
2 import torch.nn.functional as F
3
4
5 def warp_features(features: torch.Tensor, flow: torch.Tensor) -> torch.
  Tensor:
6     """
7     Warp feature map F2 using flow field to align with F1.
8
9     This is the critical operation in PWC-Net:
10    Instead of searching over a large range (like FlowNetC),
11    we use the current flow estimate to pre-align F2 with F1,
12    then search only over a small residual range.
13
14    Mathematical operation:
15        warped_F2(x, y) = F2(x + u(x,y), y + v(x,y))
16
17    Implementation uses F.grid_sample which performs bilinear
    interpolation

```

```

18     at non-integer locations. This is fully differentiable --- gradients
19     flow
20     through the sampling operation back to both the flow and features.
21
22     This is the SAME operation as the Spatial Transformer Network's grid
23     sampler.
24
25     Args:
26         features: Feature map to warp, shape (B, C, H, W) -- this is F2
27         flow:      Flow field, shape (B, 2, H, W)
28                   flow[:,0] = u (horizontal), flow[:,1] = v (vertical)
29
30     Returns:
31         warped: Warped features, shape (B, C, H, W)
32                 warped[b,:,y,x] ~= features[b,:,y+v[b,y,x], x+u[b,y,x]]
33
34     """
35     B, C, H, W = features.shape
36
37     # Build a base grid of pixel coordinates in NORMALIZED form [-1, 1]
38     # F.grid_sample expects normalized coordinates:
39     #   -1 = leftmost/topmost pixel, +1 = rightmost/bottommost pixel
40     y_norm = torch.linspace(-1, 1, H, device=features.device)
41     x_norm = torch.linspace(-1, 1, W, device=features.device)
42     # meshgrid returns (H,W) grids; we need (x,y) order for grid_sample
43     grid_y, grid_x = torch.meshgrid(y_norm, x_norm, indexing='ij') # each
44     (H,W)
45     base_grid = torch.stack([grid_x, grid_y], dim=-1) # (H, W, 2) in (x,
46     y) order
47     base_grid = base_grid.unsqueeze(0).expand(B, -1, -1, -1) # (B, H, W,
48     2)
49
50     # Convert flow from pixel space to normalized coordinates
51     # A 1-pixel shift in x corresponds to 2/W in normalized space
52     # (because normalized range [-1,1] spans W pixels, total range = 2)
53     flow_norm = torch.stack([
54         flow[:, 0] / ((W - 1) / 2), # u -> normalized x displacement
55         flow[:, 1] / ((H - 1) / 2)  # v -> normalized y displacement
56     ], dim=-1) # (B, H, W, 2)
57
58     # Add displacement to base grid
59     # sampling_grid[b, y, x] = where to sample from features to get output
60     at (x,y)
61     sampling_grid = base_grid + flow_norm # (B, H, W, 2)
62
63     # Sample features at non-integer displaced positions
64     # mode='bilinear': smooth bilinear interpolation (differentiable
65     everywhere)
66     # padding_mode='zeros': out-of-bounds positions return 0
67     # (explicitly marks regions that flowed outside the frame boundary)
68     # align_corners=True: pixel centers at +/-1 (consistent with our
69     normalization)

```

```

61     warped = F.grid_sample(
62         input=features,
63         grid=sampling_grid,
64         mode='bilinear',
65         padding_mode='zeros',
66         align_corners=True
67     )
68     return warped    # (B, C, H, W)
69
70
71 def verify_warping():
72     """
73     Verify: if we warp F2 with the ground-truth flow, we should recover F1
74     .
75     """
76     B, C, H, W = 1, 3, 64, 64
77     f1 = torch.zeros(B, C, H, W)
78     f2 = torch.zeros(B, C, H, W)
79
80     # F1 has a bright square; F2 has the same square shifted 5px right
81     f1[:, :, 25:40, 25:40] = 1.0
82     f2[:, :, 25:40, 30:45] = 1.0
83
84     # Ground truth flow: content at (x,y) in F1 came from (x+5, y) in F2
85     # So to align F2 with F1, we need to shift F2 LEFT by 5: u = -5
86     flow_gt = torch.zeros(B, 2, H, W)
87     flow_gt[:, 0, :, :] = -5.0    # u = -5
88     flow_gt[:, 1, :, :] = 0.0    # v = 0
89
90     f2_warped = warp_features(f2, flow_gt)
91
92     before = (f1 - f2).abs().mean().item()
93     after  = (f1 - f2_warped).abs().mean().item()
94
95     print(f"Before warping |F1 - F2|:          {before:.4f}")
96     print(f"After  warping |F1 - warp(F2)|:      {after:.4f}")
97     # Expect: after << before (warping significantly reduces the
98     # misalignment)
99     # The residual is non-zero only at boundary pixels (padding effect)

```

Listing 9.1: Differentiable warping operation — the mathematical heart of PWC-Net

9.6 Context Network and Final Architecture

At the finest scale, PWC-Net adds a **context network** — a series of dilated convolutions that aggregate large receptive field context without increasing resolution. Dilation rates: 1, 2, 4, 8, 16, 1, 1 (growing then shrinking). This final refinement step significantly improves accuracy at flow discontinuities and occlusion boundaries.

Chapter 9 Summary: PWC-Net

- PWC-Net (2018): three classical ideas — **Pyramid**, **Warping**, **Cost Volume** — combined optimally.
- Learned feature pyramids encode semantic content, not just brightness.
- **Warping** pre-aligns frame 2's features using the current coarse flow estimate; the cost volume handles only the small residual. This is the key efficiency improvement over FlowNet.
- Level-wise cost volumes with small search range ($D' = 4$ after warping) are efficient and effective.
- Achieves comparable or better accuracy than FlowNet 2.0 with $\sim 18\times$ fewer parameters.

Connection to Your Background

The warping operation in PWC-Net (Equation 9.1) is identical to the spatial transformer network's bilinear sampling: `F.grid_sample` in PyTorch. This fundamental operation appears throughout modern CV beyond optical flow — image registration, deformable convolutions, and differentiable data augmentation all use the same mechanism. Understanding warping deeply prepares you for working with any spatially adaptive neural network architecture.

Research Questions to Explore

1. **Warping error accumulation:** At each pyramid level, PWC-Net warps frame 2 by the upsampled coarser estimate. If the coarser estimate has errors, warping misaligns the features, making cost volume computation harder. How does this error accumulate across levels? Does this suggest an alternative strategy for initializing the coarse estimate?
2. **Occlusion and warping:** When a region is occluded — visible in frame 1 but not frame 2 — warping frame 2 cannot produce meaningful features for that region. How does PWC-Net handle this implicitly? Could you design an explicit occlusion detection and handling mechanism within this framework?
3. **Image warping vs. feature warping:** Classical methods warp raw images; PWC-Net warps learned features. What are the advantages and disadvantages of each? Under what circumstances would warping raw images at fine levels while warping learned features at coarse levels be beneficial?

Chapter 10

RAFT: Recurrent All-Pairs Field Transforms

*RAFT's insight: instead of coarse-to-fine, iterate.
Instead of local cost volume, use all pairs. Instead of a
separate decoder, use a recurrent unit.*

10.1 Why RAFT Was a Breakthrough

RAFT [?] was published at ECCV 2020 and immediately set a new state-of-the-art on every major benchmark by a large margin. But its significance goes beyond accuracy numbers. RAFT broke from the pyramid-warping paradigm that had dominated since FlowNet and introduced a fundamentally different approach built on three orthogonal innovations:

1. **All-pairs correlation:** Build a 4D correlation volume encoding similarity between *every* pair of pixels across the two frames — no bounded search range.
2. **No pyramid hierarchy:** Operate at a fixed resolution throughout, processing the whole image at each iteration.
3. **Recurrent update operator:** A GRU-based recurrent unit iteratively refines the flow estimate by reading from the correlation volume, inspired by classical iterative solvers rather than coarse-to-fine hierarchies.

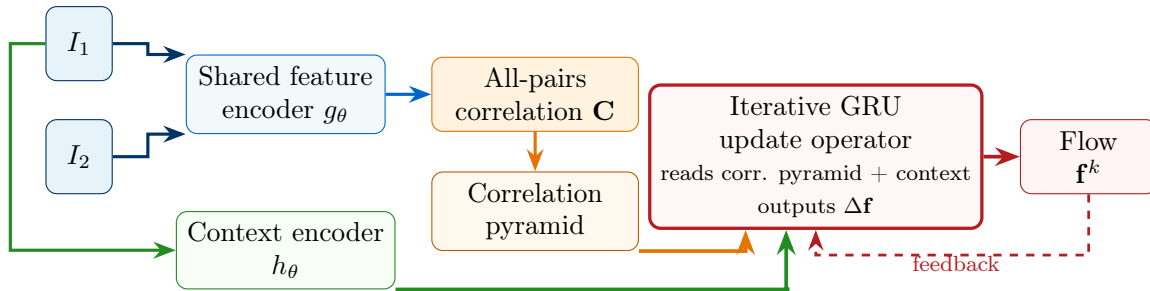


Figure 10.1: RAFT architecture. A shared feature encoder processes both frames. A context encoder processes only frame 1. Their features build a precomputed 4D all-pairs correlation volume. A GRU-based recurrent unit iteratively refines the flow estimate by querying the correlation volume at the current predicted positions.

10.2 Feature Encoder and Context Encoder

RAFT uses two separate encoders with distinct roles.

10.2.1 Feature Encoder g_θ

Applied to *both* frames with **shared weights**. Outputs feature maps at $1/8$ resolution with 256 channels. Architecture: ResNet-style with instance normalization (which is more robust to style/appearance variations across frames than batch normalization). The output of g_θ is used exclusively to build the correlation volume — it must be discriminative for matching.

10.2.2 Context Encoder h_θ

Applied *only to frame 1*. Outputs at $1/8$ resolution with 256 channels. Architecture: similar to g_θ but with batch normalization. The context features are provided to the GRU update operator to help it understand the local scene structure around each pixel — not for matching, but for conditioning the flow updates.

Intuition First

Intuition for two encoders: The feature encoder needs to produce similar representations for matching pixels (same object, slightly different viewpoint/time). This is a discriminative matching problem. The context encoder needs to capture scene context (object boundaries, depth cues, semantic content) to guide where and how to update the flow estimate. These are different tasks with potentially different optimal feature representations — hence separate encoders.

10.3 All-Pairs Correlation: The 4D Correlation Volume

This is RAFT’s most distinctive innovation. Instead of a local cost volume with bounded search range, RAFT computes the **full all-pairs correlation** between every

pair of pixels across the two frames:

$$\mathbf{C}(g_1, g_2)_{ij} = \langle g_1(p_i), g_2(p_j) \rangle \quad \forall p_i \in I_1, p_j \in I_2 \quad (10.1)$$

For feature maps of spatial size $H' \times W'$, the correlation volume $\mathbf{C}$ has shape $(H' \times W') \times (H' \times W')$ — an $(H'W') \times (H'W')$ matrix, which we can also view as a 4D tensor of shape $H' \times W' \times H' \times W'$.

What does $\mathbf{C}_{ij}$ mean? It is the dot product similarity between the feature at position p_i in frame 1 and the feature at position p_j in frame 2. For a pixel at position (x, y) in frame 1, the row $\mathbf{C}_{(x,y),:}$ is a $(H' \times W')$ -dimensional vector encoding how similar that pixel is to *every* pixel in frame 2 simultaneously. The maximum of this vector (or rather the GRU’s learned reading strategy) tells us where the best match is.

Why is this powerful? Classic pyramidal approaches must decide at initialization where to look for matches (bounded by pyramid resolution). RAFT precomputes all possible matches unconditionally — there is no assumption about displacement range. A pixel that moved 100 pixels can still find its match, because all pairs are included in $\mathbf{C}$.

10.3.1 The Correlation Pyramid

To efficiently handle multiple displacement scales, RAFT builds a **correlation pyramid** by average-pooling the correlation volume $\mathbf{C}$ along the frame 2 dimensions:

$$\mathbf{C}^k = \text{AvgPool}_{2^k}(\mathbf{C}), \quad k = 0, 1, 2, 3 \quad (10.2)$$

At level $k = 0$: full resolution, captures fine-grained matches. At level $k = 3$: $1/8$ of original, each “pixel” in frame 2 represents an 8×8 region, capturing coarse matches over large displacements. The pyramid is precomputed once and stored.

Computational cost: For feature maps of size $H' \times W' = 50 \times 87$ (typical for 400×696 input at $1/8$ scale), $\mathbf{C}$ has $50 \times 87 \times 50 \times 87 = 18.9\text{M}$ entries. While large, this is precomputed once and read many times during the K iterations of the GRU.

10.4 The GRU Update Operator

This is the component that most connects to your LSTM/GRU experience. RAFT replaces the decoder of FlowNet/PWC-Net with a **recurrent update operator** — a ConvGRU (convolutional GRU) that maintains a hidden state $\mathbf{h}^k$ and iteratively refines the flow estimate $\mathbf{f}^k$.

10.4.1 Standard GRU Review

You implemented GRU from scratch, so let us connect what you know to what RAFT uses. A standard GRU update for a single vector hidden state:

$$z_t = \sigma(W_z[h_{t-1}, x_t]) \quad (\text{update gate}) \quad (10.3)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t]) \quad (\text{reset gate}) \quad (10.4)$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t]) \quad (\text{candidate}) \quad (10.5)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (\text{new hidden state}) \quad (10.6)$$

RAFT’s ConvGRU replaces the matrix multiplications with 3×3 convolutions, making it operate over the entire 2D flow field simultaneously.

10.4.2 What the GRU Receives as Input

At each iteration k , the ConvGRU receives:

- **Context features $\mathbf{x}_{ctx}$** : The 256-channel output of the context encoder for frame 1. Fixed across iterations.
- **Correlation features $\mathbf{x}_{corr}$** : Looked up from the correlation pyramid at the *current flow estimate* positions. This is the key: we sample the correlation pyramid around each pixel’s current predicted position, extracting similarities with nearby frame 2 positions. This is how the GRU “reads” matching evidence.
- **Current flow estimate $\mathbf{f}^k$** : The flow estimate at the previous iteration, allowing the GRU to reason about how fast it is converging.

10.4.3 Correlation Lookup

The correlation lookup operation extracts a local neighborhood from the correlation pyramid around the current flow prediction. At iteration k , for a pixel at position $\mathbf{x}$ in frame 1 with current flow estimate $\mathbf{f}^k(\mathbf{x})$:

$$\mathbf{x}' = \mathbf{x} + \mathbf{f}^k(\mathbf{x}) \quad (10.7)$$

We then sample the correlation volume in a $r \times r$ grid ($r = 4$ by default) centered at $\mathbf{x}'$ in frame 2. This gives a $(2r + 1)^2 = 81$ -dimensional vector of similarities for each pyramid level. Concatenated over 4 pyramid levels: $81 \times 4 = 324$ channels of correlation features per pixel.

Intuition First

Intuition for the lookup: At each iteration, we say “our current best guess is that this pixel moved to position $\mathbf{x}'$. Let me look at the correlation values in a small neighborhood around $\mathbf{x}'$ to see if the match is there, or if I should nudge the estimate left, right, up, or down.” This is essentially gradient descent on the matching cost, but learned rather than hand-designed.

10.4.4 The Update

After reading correlation features, the ConvGRU produces a flow update $\Delta \mathbf{f}^k$, and the flow estimate is updated:

$$\mathbf{f}^{k+1} = \mathbf{f}^k + \Delta \mathbf{f}^k \quad (10.8)$$

This is iterated K times (typically $K = 12$ at test time, $K = 6$ – 10 during training). Each iteration refines the flow by looking at new correlation evidence around the updated prediction.

10.5 Complete RAFT PyTorch Implementation

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5
6 #
=====
7 # RAFT BUILDING BLOCKS
8 #
=====
9
10 class ResidualBlock(nn.Module):
11     """
12     Standard ResNet residual block.
13     Used in both the feature encoder and context encoder.
14
15     The residual connection (output = conv(x) + x) enables training
16     of very deep networks by providing gradient shortcuts.
17     You implemented this for your ResNet from scratch.
18     """
19     def __init__(self, in_ch: int, out_ch: int, stride: int = 1,
20                 norm: str = 'batch'):
21         super().__init__()
22
23         # Choose normalization:
24         # 'instance' for feature encoder (robust to appearance variation)
25         # 'batch' for context encoder (more training stability)
26         def get_norm(ch):
27             if norm == 'batch': return nn.BatchNorm2d(ch)
28             elif norm == 'instance': return nn.InstanceNorm2d(ch)
29             else: return nn.Identity() # 'none'
30
31         self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride=stride, padding=1,
32                                bias=False)
33         self.norm1 = get_norm(out_ch)
34         self.conv2 = nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False)
35         self.norm2 = get_norm(out_ch)
36         self.relu = nn.ReLU(inplace=True)
37
38         # Projection shortcut: needed when stride>1 or channels change
39         self.shortcut = nn.Sequential()
40         if stride != 1 or in_ch != out_ch:
41             self.shortcut = nn.Sequential(
42                 nn.Conv2d(in_ch, out_ch, 1, stride=stride, bias=False),
43                 get_norm(out_ch)
44             )

```

```

45     def forward(self, x: torch.Tensor) -> torch.Tensor:
46         out = self.relu(self.norm1(self.conv1(x)))
47         out = self.norm2(self.conv2(out))
48         return self.relu(out + self.shortcut(x))    # Add residual
connection
49
50
51 class RAFTFeatureEncoder(nn.Module):
52     """
53     Feature encoder: applied to BOTH frames with SHARED weights.
54
55     Architecture: ResNet-style encoder with instance normalization.
56     Outputs feature maps at 1/8 resolution.
57
58     Instance norm is used instead of batch norm because:
59     - We process two frames, possibly with different brightness statistics
60     - Instance norm normalizes per-sample per-channel (independent per
image)
61     - Batch norm normalizes across the batch (could mix frame1 and frame2
stats)
62     """
63     def __init__(self, output_dim: int = 256, norm: str = 'instance'):
64         super().__init__()
65
66         # Initial convolution: full res -> 1/2 res
67         self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3,
bias=False)
68         self.norm1 = nn.InstanceNorm2d(64) if norm == 'instance' else nn.
BatchNorm2d(64)
69         self.relu = nn.ReLU(inplace=True)
70
71         # Residual stages
72         self.layer1 = nn.Sequential(
73             ResidualBlock(64, 64, stride=1, norm=norm),    # -> 1/2 res, 64
ch
74             ResidualBlock(64, 64, stride=1, norm=norm),
75         )
76         self.layer2 = nn.Sequential(
77             ResidualBlock(64, 96, stride=2, norm=norm),    # -> 1/4 res, 96
ch
78             ResidualBlock(96, 96, stride=1, norm=norm),
79         )
80         self.layer3 = nn.Sequential(
81             ResidualBlock(96, 128, stride=2, norm=norm),    # -> 1/8 res,
128ch
82             ResidualBlock(128, 128, stride=1, norm=norm),
83         )
84
85         # Project to final output_dim
86         self.conv2 = nn.Conv2d(128, output_dim, kernel_size=1)
87         # Note: no normalization/activation after final projection --

```

```

88         # we want the raw feature vectors for dot-product correlation
89
90     def forward(self, x: torch.Tensor) -> torch.Tensor:
91         """
92         Args: x: Input frame, shape (B, 3, H, W), normalized to [-1, 1]
93         Returns: features at 1/8 resolution, shape (B, output_dim, H/8, W
94 /8)
95         """
96         x = self.relu(self.norm1(self.conv1(x)))    # (B, 64, H/2, W/2)
97         x = self.layer1(x)                        # (B, 64, H/2, W/2)
98         x = self.layer2(x)                        # (B, 96, H/4, W/4)
99         x = self.layer3(x)                        # (B, 128, H/8, W/8)
100        x = self.conv2(x)                          # (B, 256, H/8, W/8)
101        return x
102
103    class CorrBlock:
104        """
105        Precomputes and stores the 4D all-pairs correlation volume,
106        and provides efficient lookup at arbitrary positions.
107
108        This is a DATA STRUCTURE, not an nn.Module, because the correlation
109        volume is precomputed once per frame pair (not per iteration).
110        The GRU then queries it K times during the iterative refinement.
111
112        Memory: for H'=50, W'=87: 50*87 * 50*87 * 4 bytes = ~76MB per example
113        In practice, we work at 1/8 resolution, reducing this significantly.
114        """
115
116        def __init__(self, fmap1: torch.Tensor, fmap2: torch.Tensor,
117                     num_levels: int = 4, radius: int = 4):
118            """
119            Precompute the 4D correlation volume and its pyramid.
120
121            Args:
122            fmap1: Frame 1 features, shape (B, C, H, W) from feature
123 encoder
124            fmap2: Frame 2 features, shape (B, C, H, W) from feature
125 encoder
126            num_levels: Number of pyramid levels (coarsening by 2x each)
127            radius: Lookup radius around predicted position
128            """
129            self.num_levels = num_levels
130            self.radius = radius
131
132            B, C, H, W = fmap1.shape
133
134            # L2-normalize features before dot product (makes it cosine
135 similarity)
136            # Cosine similarity is more stable across different image
137 statistics

```

```

134     fmap1_norm = fmap1 / (fmap1.norm(dim=1, keepdim=True) + 1e-8)
135     fmap2_norm = fmap2 / (fmap2.norm(dim=1, keepdim=True) + 1e-8)
136
137     # Reshape for batch matrix multiplication
138     # We want: C_ij = <fmap1[i], fmap2[j]>
139     # = (fmap1 reshaped to (B, H*W, C)) @ (fmap2 reshaped to (B, C, H*
140 HW, C))
141     fmap1_flat = fmap1_norm.view(B, C, H * W).permute(0, 2, 1) # (B,
142 HW, C)
143     fmap2_flat = fmap2_norm.view(B, C, H * W) # (B,
144 C, HW)
145
146     # Compute ALL pairs at once via batch matrix multiply
147     # corr shape: (B, H*W, H*W)
148     # corr[b, i, j] = <fmap1[b, :, i], fmap2[b, :, j]>
149     corr = torch.bmm(fmap1_flat, fmap2_flat) # (B, HW, HW)
150
151     # Reshape to 4D: (B, H, W, H, W) -- the full spatial structure
152     # But we store it as (B*H*W, 1, H, W) for efficient pyramid
153 pooling
154     corr = corr.view(B * H * W, 1, H, W)
155
156     # Build pyramid by average-pooling along frame 2 dimensions
157     # Level 0: full resolution (matches at 1-pixel accuracy)
158     # Level k: 1/2^k resolution (matches at 2^k pixel accuracy)
159     self.corr_pyramid = []
160     for i in range(self.num_levels):
161         self.corr_pyramid.append(corr)
162         corr = F.avg_pool2d(corr, kernel_size=2, stride=2)
163         # After 4 levels: (B*H*W, 1, H/8, W/8)
164
165     self.B, self.H, self.W = B, H, W
166
167     def __call__(self, coords: torch.Tensor) -> torch.Tensor:
168         """
169         Look up correlation features at given coordinates.
170
171         For each pixel, we sample the correlation pyramid in a (2r+1)x(2r
172 +1)
173         neighborhood centered at the current flow prediction.
174
175         This is the "reading" step: given current flow estimate, query
176         the cost volume to see how good the match is and what direction
177         we should correct toward.
178
179         Args:
180             coords: Current flow prediction + base grid, shape (B, 2, H, W
181 )
182                 coords[:,0] = x-coordinates in frame 2 (not offsets,
183 absolute!)
184                 coords[:,1] = y-coordinates in frame 2

```



```

178
179     Returns:
180         corr_features: Concatenated pyramid lookups,
181                        shape (B, num_levels * (2r+1)^2, H, W)
182
183     """
184     r = self.radius
185     B, H, W = self.B, self.H, self.W
186
187     out_pyramid = []
188
189     for i, corr in enumerate(self.corr_pyramid):
190         # At pyramid level i, coordinates are scaled by 2^i
191         dx = torch.linspace(-r, r, 2*r+1, device=coords.device)
192         dy = torch.linspace(-r, r, 2*r+1, device=coords.device)
193         delta_y, delta_x = torch.meshgrid(dy, dx, indexing='ij')
194         # delta shape: (2r+1, 2r+1)
195
196         # Center the lookup grid at the current predicted position (
197         scaled)
198         # coords_lvl: normalized coordinates for this pyramid level
199         scale = 2**i    # At level i, frame2 spatial resolution is
200         1/2^i
201         centroid_lvl = coords.permute(0, 2, 3, 1).reshape(B*H*W, 1, 1,
202         2)
203         centroid_lvl = centroid_lvl / scale    # scale to this pyramid
204         level
205
206         # Build sampling grid: centered at prediction, offset by delta
207         delta = torch.stack([delta_x, delta_y], dim=-1) # (2r+1, 2r
208         +1, 2)
209         delta = delta.view(1, 1, (2*r+1)**2, 2)    # (1,1,(2r
210         +1)^2,2)
211
212         # Absolute coordinates in frame 2 at this pyramid level
213         coords_lvl = centroid_lvl + delta    # (B*H*W, 1, (2r+1)^2, 2)
214
215         # Normalize to [-1, 1] for F.grid_sample
216         H_lvl = H // scale
217         W_lvl = W // scale
218         coords_norm = coords_lvl.clone()
219         coords_norm[..., 0] = (coords_norm[..., 0] / (W_lvl - 1)) * 2
220         - 1
221         coords_norm[..., 1] = (coords_norm[..., 1] / (H_lvl - 1)) * 2
222         - 1
223
224         # Sample from the correlation volume at computed positions
225         # corr shape: (B*H*W, 1, H_lvl, W_lvl)
226         sampled = F.grid_sample(corr, coords_norm, align_corners=True,
227                                mode='bilinear', padding_mode='zeros',
228                                )
229         # sampled shape: (B*H*W, 1, 1, (2r+1)^2)

```

```

220         sampled = sampled.view(B, H*W, -1)  # (B, H*W, (2r+1)^2)
221         out_pyramid.append(sampled)
222
223     # Concatenate all pyramid levels
224     out = torch.cat(out_pyramid, dim=-1)      # (B, H*W, num_levels
225     *(2r+1)^2)
226     out = out.permute(0, 2, 1)                # (B, features, H*W)
227     return out.view(B, -1, H, W)             # (B, features, H, W)
228
229
230 class ConvGRU(nn.Module):
231     """
232     Convolutional GRU: the recurrent update operator in RAFT.
233
234     This is exactly a GRU, but with 3x3 convolutions instead of matrix
235     multiplies.
236     This allows it to process the full 2D flow field simultaneously,
237     with each pixel's update depending on its spatial neighbors.
238
239     WHY A GRU INSTEAD OF A DECODER?
240     - A decoder produces flow in one shot from features
241     - A GRU can iteratively refine: each iteration looks at the CURRENT
242     flow
243     and the correlation evidence, and produces a CORRECTION
244     - This is analogous to iterative optimization (like Newton's method)
245     but with a learned update rule instead of a gradient
246
247     HIDDEN STATE = "MEMORY OF PREVIOUS ITERATIONS"
248     The hidden state carries information about what the previous
249     iterations
250     have learned about the flow, enabling more informed updates.
251
252     INPUT at each iteration (concatenated along channel dimension):
253     - correlation features: what the correlation volume says near
254     current prediction
255     - current flow estimate: where we think pixels went
256     - context features: appearance/semantic context from frame 1
257     """
258
259     def __init__(self, hidden_dim: int = 128, input_dim: int = 192 + 128):
260         super().__init__()
261
262         self.convz = nn.Conv2d(hidden_dim + input_dim, hidden_dim, 3,
padding=1)
263         self.convr = nn.Conv2d(hidden_dim + input_dim, hidden_dim, 3,
padding=1)
264         self.convq = nn.Conv2d(hidden_dim + input_dim, hidden_dim, 3,
padding=1)
265         # z = update gate: how much of old hidden state to keep
266         # r = reset gate: how much of old hidden state to forget

```

```

263         # q = candidate: new information to potentially add
264
265     def forward(self, h: torch.Tensor, x: torch.Tensor) -> torch.Tensor:
266         """
267         GRU update step.
268
269         Args:
270             h: Previous hidden state, shape (B, hidden_dim, H, W)
271             x: Input (concatenation of correlation features + flow +
272 context),
273                 shape (B, input_dim, H, W)
274
275         Returns:
276             h_new: Updated hidden state, shape (B, hidden_dim, H, W)
277         """
278         hx = torch.cat([h, x], dim=1) # (B, hidden_dim + input_dim, H, W)
279
280         z = torch.sigmoid(self.convz(hx)) # Update gate: what to update
281         r = torch.sigmoid(self.convr(hx)) # Reset gate: what to forget
282
283         # Candidate: new information, with reset gate controlling
284         # how much of previous hidden state influences the candidate
285         q = torch.tanh(self.convq(torch.cat([r * h, x], dim=1)))
286
287         # New hidden state: blend old (kept by 1-z) and new (added by z)
288         h_new = (1 - z) * h + z * q
289
290         return h_new
291
292     class UpdateBlock(nn.Module):
293         """
294         Full update block: takes GRU hidden state and produces flow update.
295
296         Per-iteration pipeline:
297         1. GRU receives (correlation features + current flow + context) -> new
298 hidden state
299         2. A small MLP/conv processes the hidden state to produce a flow delta
300         3. Flow delta is upsampled 8x (from 1/8 res to full res) via a learned
301 upsampler
302
303         The upsampling is learned (not bilinear) because the flow at 1/8
304 resolution
305 may not agree with the true flow at full resolution at fine edges.
306 RAFT uses a convex combination of 3x3 neighborhoods for upsampling.
307 """
308
309         def __init__(self, hidden_dim: int = 128, corr_levels: int = 4,
310 corr_radius: int = 4):
311             super().__init__()

```

```

309
310     # Number of correlation features per pixel per iteration
311     corr_channels = corr_levels * (2 * corr_radius + 1) ** 2    # 4 * 81 = 324
312
313     # Encoder for correlation features: reduce channels before GRU
314     self.corr_encoder = nn.Sequential(
315         nn.Conv2d(corr_channels, 256, 1, padding=0),
316         nn.ReLU(inplace=True),
317         nn.Conv2d(256, 192, 3, padding=1),
318         nn.ReLU(inplace=True),
319     )
320
321     # Encoder for current flow: gives GRU context about current
322 estimate
323     self.flow_encoder = nn.Sequential(
324         nn.Conv2d(2, 128, 7, padding=3),    # 2-channel flow (u,v)
325         nn.ReLU(inplace=True),
326         nn.Conv2d(128, 64, 3, padding=1),
327         nn.ReLU(inplace=True),
328     )
329     # Total input to GRU: 192 (corr) + 64 (flow) + 128 (context) = 384
330     self.gru = ConvGRU(hidden_dim=hidden_dim, input_dim=192 + 64 +
331 128)
332
333     # Produce flow delta from hidden state
334     self.flow_head = nn.Sequential(
335         nn.Conv2d(hidden_dim, 256, 3, padding=1),
336         nn.ReLU(inplace=True),
337         nn.Conv2d(256, 2, 3, padding=1),    # 2-channel output: (
338 delta_u, delta_v)
339     )
340
341     # Learned upsampler: produces 8x upsampled flow from 1/8-res flow
342     # Outputs a mask of 8*8*9 = 576 channels: 9-neighbor convex
343 weights
344     # for each of the 8x8 output pixels per 1/8-res pixel
345     self.mask = nn.Sequential(
346         nn.Conv2d(hidden_dim, 256, 3, padding=1),
347         nn.ReLU(inplace=True),
348         nn.Conv2d(256, 64 * 9, 1, padding=0),
349     )
350
351     def forward(self, net: torch.Tensor, inp: torch.Tensor,
352                 corr: torch.Tensor, flow: torch.Tensor) -> tuple:
353         """
354         One update iteration.
355
356         Args:

```

```

354         net: GRU hidden state (the "memory"), shape (B, hidden_dim, H,
W)
355         inp: Context features from frame 1, shape (B, 128, H, W)
356         corr: Correlation features at current position, shape (B,
corr_ch, H, W)
357         flow: Current flow estimate at 1/8 resolution, shape (B, 2, H,
W)
358
359     Returns:
360         net: Updated hidden state
361         delta_flow: Flow correction to add to current estimate, shape
(B, 2, H, W)
362         up_mask: Weights for 8x upsampling, shape (B, 576, H, W)
363     """
364     # Encode inputs
365     corr_feat = self.corr_encoder(corr) # (B, 192, H, W)
366     flow_feat = self.flow_encoder(flow) # (B, 64, H, W)
367
368     # Concatenate for GRU input
369     gru_input = torch.cat([corr_feat, flow_feat, inp], dim=1) # (B,
384, H, W)
370
371     # GRU update: new hidden state captures accumulated matching
evidence
372     net = self.gru(net, gru_input) # (B, hidden_dim, H, W)
373
374     # Predict flow correction from updated hidden state
375     delta_flow = self.flow_head(net) # (B, 2, H, W)
376
377     # Predict upsampling mask
378     up_mask = 0.25 * self.mask(net) # (B, 576, H, W)
379     # 0.25 scaling stabilizes training; mask will be softmax'd at
upsample time
380
381     return net, delta_flow, up_mask
382
383
384 class RAFT(nn.Module):
385     """
386     Complete RAFT model.
387
388     This is the reference implementation of RAFT (Teed and Deng, ECCV
2020).
389     All components described in the paper are present.
390
391     Key design principles:
392     1. Single resolution (1/8): no pyramid of networks, just pyramid of
corr volume
393     2. Shared encoder: ensures F1 and F2 features are comparable for
correlation

```

```

394     3. Precomputed correlation: computed once, read K times during
iteration
395     4. GRU iteration: learned refinement, not fixed optimization
396     5. Full-resolution output: learned 8x upsampling, not bilinear
397     """
398
399     def __init__(self, hidden_dim: int = 128, context_dim: int = 128,
400                  corr_levels: int = 4, corr_radius: int = 4):
401         super().__init__()
402
403         self.hidden_dim = hidden_dim
404         self.context_dim = context_dim
405
406         # Feature encoder: applied to both frames (shared weights)
407         self.fnet = RAFTFeatureEncoder(output_dim=256, norm='instance')
408
409         # Context encoder: applied to frame 1 only
410         # Provides context features for the GRU update operator
411         self.cnet = RAFTFeatureEncoder(output_dim=hidden_dim + context_dim
,
412                                       norm='batch')
413         # hidden_dim channels -> initialize GRU hidden state
414         # context_dim channels -> fixed input to GRU at each iteration
415
416         # Update block containing ConvGRU and flow head
417         self.update_block = UpdateBlock(hidden_dim=hidden_dim,
418                                         corr_levels=corr_levels,
419                                         corr_radius=corr_radius)
420
421         self.corr_levels = corr_levels
422         self.corr_radius = corr_radius
423
424     def initialize_flow(self, img: torch.Tensor) -> tuple:
425         """
426         Initialize the flow estimate to zero and compute the coordinate
grid.
427
428         The 'coordinates' represent absolute positions in frame 2.
429         coords1 = base grid (pixel positions in frame 1 at 1/8 scale)
430         coords2 = initial flow prediction = coords1 + zero flow = coords1
431
432         Throughout iterations, coords2 = coords1 + current_flow_estimate
433         """
434         B, _, H, W = img.shape
435         H8, W8 = H // 8, W // 8
436
437         # Base coordinate grid: each position is its own coordinates
438         # coords[b, :, y, x] = (x, y) -- standard x-y ordering
439         y_coords = torch.arange(H8, device=img.device).float()
440         x_coords = torch.arange(W8, device=img.device).float()
441         grid_y, grid_x = torch.meshgrid(y_coords, x_coords, indexing='ij')

```

```

442     coords = torch.stack([grid_x, grid_y], dim=0)  # (2, H8, W8)
443     coords = coords.unsqueeze(0).expand(B, -1, -1, -1)  # (B, 2, H8,
444     W8)
445
446     return coords, coords.clone()  # (coords1, coords2 = coords1
447     initially)
448
449     def upsample_flow(self, flow: torch.Tensor,
450                       mask: torch.Tensor) -> torch.Tensor:
451         """
452         Upsample flow from 1/8 resolution to full resolution using learned
453         mask.
454
455         Instead of simple bilinear upsampling (which can be blurry at
456         edges),
457         RAFT learns a convex combination of 9 neighboring 1/8-res flow
458         values
459         for each full-res output pixel.
460
461         The mask encodes: for each of the 8x8=64 full-res pixels within a
462         1/8-res tile, what weights to assign to the 9 (3x3) neighboring
463         1/8-res flow vectors.
464
465         Args:
466             flow: Shape (B, 2, H/8, W/8)
467             mask: Shape (B, 8*8*9, H/8, W/8) -- learned convex weights
468
469         Returns:
470             flow_up: Shape (B, 2, H, W) -- full-resolution flow
471         """
472         B, _, H8, W8 = flow.shape
473
474         # Softmax over the 9-neighbor dimension to get convex combination
475         weights
476         mask = mask.view(B, 1, 9, 8, 8, H8, W8)
477         mask = torch.softmax(mask, dim=2)  # (B, 1, 9, 8, 8, H8, W8)
478
479         # Unfold: extract 3x3 neighborhood around each 1/8-res position
480         # This gives us the 9 neighbors for the convex combination
481         flow_up = F.unfold(8 * flow, kernel_size=3, padding=1)  # (B,
482         2*9, H8*W8)
483         flow_up = flow_up.view(B, 2, 9, 1, 1, H8, W8)
484         # The '8 *' scales flow from 1/8-res to full-res magnitude
485
486         # Apply convex combination
487         flow_up = (mask * flow_up).sum(dim=2)  # (B, 2, 8, 8, H8, W8)
488
489         # Rearrange into full-resolution image
490         flow_up = flow_up.permute(0, 1, 4, 2, 5, 3)  # (B, 2, H8, 8, W8,
491         8)

```

```

485         return flow_up.reshape(B, 2, H8 * 8, W8 * 8)    # (B, 2, H, W)
486
487     def forward(self, image1: torch.Tensor, image2: torch.Tensor,
488                 iters: int = 12, flow_init: torch.Tensor = None,
489                 test_mode: bool = False) -> list:
490         """
491         Forward pass of RAFT.
492
493         Args:
494             image1, image2: Input frames, shape (B, 3, H, W), normalized
495             [-1, 1]
496             iters: Number of GRU iterations (12 for inference, 6-10 for
497             training)
498             flow_init: Optional initial flow estimate for warm-starting
499             test_mode: If True, return only final flow; if False, return
500             all iterations
501             (all iterations used for multi-scale loss during
502             training)
503
504         Returns:
505             flow_predictions: List of full-resolution flow predictions.
506             If test_mode=True: list of length 1 (final flow).
507             If test_mode=False: list of length iters (for multi-
508             iteration loss).
509         """
510         # --- 1. Extract features from both frames ---
511         # The feature encoder is shared: same weights for frame1 and
512         frame2
513         # This enforces that both frames live in the same feature space
514         fmap1 = self.fnet(image1)    # (B, 256, H/8, W/8)
515         fmap2 = self.fnet(image2)    # (B, 256, H/8, W/8)
516
517         # Cast to float32 for correlation computation stability
518         fmap1, fmap2 = fmap1.float(), fmap2.float()
519
520         # --- 2. Precompute ALL-PAIRS correlation volume ---
521         # This is done ONCE and reused at every iteration
522         # corr_fn is a CorrBlock object that handles lookup
523         corr_fn = CorrBlock(fmap1, fmap2,
524                             num_levels=self.corr_levels,
525                             radius=self.corr_radius)
526
527         # --- 3. Extract context features from frame 1 ---
528         # Context features condition the GRU update at every iteration
529         cnet = self.cnet(image1)    # (B, hidden_dim + context_dim, H/8, W
530         /8)
531
532         # Split context into: initial hidden state and recurring input
533         net_h, inp = torch.split(cnet,
534                                  [self.hidden_dim, self.context_dim], dim
535                                  =1)

```



```

528     # net_h: initial GRU hidden state (tanh to keep in [-1,1])
529     # inp: context input repeated at each iteration
530     net_h = torch.tanh(net_h)    # (B, hidden_dim, H/8, W/8)
531     inp = torch.relu(inp)        # (B, context_dim, H/8, W/8)
532
533     # --- 4. Initialize flow estimate ---
534     coords1, coords2 = self.initialize_flow(image1)
535     # coords1: base grid (unchanged throughout)
536     # coords2: current predicted destination = coords1 + current_flow
537
538     if flow_init is not None:
539         # Warm-start: use provided initial flow estimate
540         # Useful for video processing where previous frame's flow is a
541         # good start
542         coords2 = coords2 + F.interpolate(flow_init, size=coords2.
543 shape[-2:],
544                                         mode='bilinear',
545 align_corners=True)
546
547     # --- 5. Iterative refinement loop ---
548     flow_predictions = []
549
550     for _ in range(iters):
551         # Current flow estimate = destination - source
552         # coords1 is fixed; coords2 is updated each iteration
553         coords2 = coords2.detach()    # Detach to avoid storing full
554 computation graph
555
556         # RAFT's training uses truncated
557 BPTT
558
559         # Look up correlation features at current predicted positions
560         # corr shape: (B, num_levels*(2r+1)^2, H/8, W/8)
561         corr = corr_fn(coords2)
562
563         # Current flow estimate (coords2 - coords1)
564         flow = coords2 - coords1    # (B, 2, H/8, W/8)
565
566         # GRU update: produce new hidden state and flow correction
567         net_h, delta_flow, up_mask = self.update_block(
568             net_h, inp, corr, flow
569         )
570
571         # Update flow: add correction to current coordinates
572         # coords2 = source positions + updated flow estimate
573         coords2 = coords2 + delta_flow    # (B, 2, H/8, W/8)
574
575         # Upsample to full resolution using learned mask
576         # Store for training loss (all iterations contribute)
577         flow_up = self.upsample_flow(coords2 - coords1, up_mask)
578         # flow_up shape: (B, 2, H, W) -- full resolution
579         flow_predictions.append(flow_up)

```

```

574         if test_mode:
575             return flow_predictions[-1:]    # Only return final prediction
576
577
578         return flow_predictions    # All iterations (for multi-iteration
training loss)
579
580     def compute_loss(self, flow_preds: list,
581                     flow_gt: torch.Tensor,
582                     valid: torch.Tensor,
583                     gamma: float = 0.8) -> tuple:
584
585         """
586         Multi-iteration loss for RAFT training.
587
588         Key insight: unlike multi-SCALE loss (FlowNet/PWC-Net), this is
589         multi-ITERATION loss. ALL predictions contribute, with
EXPONENTIALLY
590         INCREASING weights for later (better) predictions.
591
592         gamma=0.8 means prediction at iteration K gets weight  $\gamma^{(N-K)}$ :
593         - Final iteration: weight 1.0
594         - Previous iteration: weight 0.8
595         - Two iterations before: weight 0.64
596         etc.
597
598         This incentivizes the network to:
599         1. Converge quickly (early iterations should be good)
600         2. Be accurate at the final iteration (later iterations weighted
more)
601
602         Args:
603             flow_preds: List of N flow predictions from forward pass
604             flow_gt:     Ground-truth flow, shape (B, 2, H, W)
605             valid:       Binary mask of valid GT pixels, shape (B, H, W)
606             gamma:       Exponential weighting factor (0.8 from the paper)
607
608         Returns:
609             loss:     Scalar total loss
610             epe:     End-point error of final prediction (for logging)
611         """
612         n_preds = len(flow_preds)
613         flow_loss = 0.0
614
615         # Clamp valid GT flow to exclude extreme outliers
616         # (LIDAR-derived GT can have occasional erroneous measurements)
617         valid = (valid >= 0.5) & (flow_gt.abs().max(dim=1).values < 1000)
618
619         for i, flow_pred in enumerate(flow_preds):
620             # Exponential weighting: later predictions get higher weight
621             # i=0 is earliest (worst), i=n_preds-1 is final (best)
622             weight = gamma ** (n_preds - 1 - i)

```

```

622
623     # EPE: L1 distance between predicted and GT flow vectors
624     # L1 is more robust to outliers than L2 (squared EPE)
625     epe_map = (flow_pred - flow_gt).abs()
626     epe_map = epe_map.sum(dim=1)      # Sum over u,v channels: (B, H
, W)
627
628     # Only count pixels with valid GT
629     flow_loss += weight * epe_map[valid].mean()
630
631     # EPE of final prediction (for logging/evaluation)
632     epe = (flow_preds[-1] - flow_gt).norm(dim=1)  # L2 norm per pixel
633     epe_final = epe[valid].mean()
634
635     return flow_loss, epe_final.item()
636
637
638 #
=====
639 # TRAINING PIPELINE ON KITTI
640 #
=====
641
642 def trainRAFT_kitti_step(
643     model: RAFT,
644     optimizer: torch.optim.Optimizer,
645     image1: torch.Tensor,
646     image2: torch.Tensor,
647     flow_gt: torch.Tensor,
648     valid: torch.Tensor,
649     device: str = 'cuda'
650 ) -> dict:
651     """
652     Single training step for RAFT on KITTI.
653
654     KITTI has SPARSE ground truth (LIDAR hits only ~35% of pixels).
655     The 'valid' mask indicates which pixels have GT flow.
656     We only compute loss on valid pixels.
657
658     Training notes:
659     - RAFT is typically trained with iters=10 during training for
efficiency
660     - Mixed precision (torch.cuda.amp) dramatically speeds up training
661     - Gradient clipping (max_norm=1.0) is important for GRU stability
662     """
663     model.train()
664     image1, image2 = image1.to(device), image2.to(device)
665     flow_gt, valid = flow_gt.to(device), valid.to(device)
666

```

```

667     # Forward pass: get all K=10 intermediate flow predictions
668     flow_predictions = model(image1, image2, iters=10, test_mode=False)
669
670     # Compute multi-iteration loss
671     loss, epe = model.compute_loss(
672         flow_predictions, flow_gt, valid, gamma=0.8
673     )
674
675     # Backward pass
676     optimizer.zero_grad()
677     loss.backward()
678
679     # Gradient clipping: essential for RNN/GRU stability
680     # Prevents exploding gradients during backprop through time
681     torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
682
683     optimizer.step()
684
685     return {'loss': loss.item(), 'epe': epe}

```

Listing 10.1: Complete RAFT implementation in PyTorch — the main deep learning implementation

10.6 Visualization of Iterative Refinement

One of the most powerful ways to build intuition for RAFT is to visualize the flow estimate at each of the K iterations. The flow typically starts as a rough, blurry estimate and progressively sharpens — object boundaries become crisper, fast-moving objects are better localized, and small-scale details emerge.

```

1  import torch
2  import cv2
3  import numpy as np
4  import matplotlib.pyplot as plt
5
6
7  def visualizeRAFT_iterations(
8      model: RAFT,
9      image1: np.ndarray,
10     image2: np.ndarray,
11     device: str = 'cuda',
12     save_path: str = 'raft_iterations.png'
13 ):
14     """
15     Visualize RAFT's flow prediction at each iteration.
16
17     This reveals the iterative refinement process:
18     - Early iterations: coarse, approximate flow (correct large-scale
19     motion)
19     - Middle iterations: motion boundaries sharpen
20     - Late iterations: fine details resolved (small objects, fast motion)

```

```

21
22     This is analogous to watching gradient descent converge:
23     the objective is being minimized step by step.
24     """
25     model.eval()
26
27     # Preprocess: normalize to [-1, 1] and convert to tensor
28     def preprocess(img_bgr: np.ndarray) -> torch.Tensor:
29         img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB).astype(np.
float32)
30         img_norm = (img_rgb / 127.5) - 1.0    # Map [0,255] -> [-1,1]
31         return torch.from_numpy(img_norm).permute(2,0,1).unsqueeze(0).to(
device)
32
33     t1 = preprocess(image1)
34     t2 = preprocess(image2)
35
36     with torch.no_grad():
37         # Get ALL K intermediate predictions
38         flow_preds = model(t1, t2, iters=12, test_mode=False)
39
40         # Select subset of iterations to visualize (first, middle, final)
41         n = len(flow_preds)
42         indices_to_show = [0, n//4, n//2, 3*n//4, n-1]
43
44         fig, axes = plt.subplots(2, len(indices_to_show), figsize=(20, 8))
45
46         for col, idx in enumerate(indices_to_show):
47             flow = flow_preds[idx][0].cpu().numpy()    # (2, H, W)
48             u, v = flow[0], flow[1]
49
50             # --- HSV visualization ---
51             mag, ang = cv2.cartToPolar(u, v)
52             hsv = np.zeros((*u.shape, 3), dtype=np.float32)
53             hsv[:, :, 0] = ang * 180 / np.pi / 2
54             hsv[:, :, 1] = 1.0
55             max_m = np.percentile(mag, 99) + 1e-6
56             hsv[:, :, 2] = np.clip(mag / max_m, 0, 1)
57             hsv_uint8 = (hsv * [180, 255, 255]).astype(np.uint8)
58             flow_rgb = cv2.cvtColor(hsv_uint8, cv2.COLOR_HSV2RGB)
59
60             axes[0, col].imshow(flow_rgb)
61             axes[0, col].set_title(f'Iter {idx+1}/{n}', fontsize=12)
62             axes[0, col].axis('off')
63
64             # --- Magnitude map ---
65             axes[1, col].imshow(mag, cmap='hot', vmin=0, vmax=np.percentile(
mag, 99))
66             axes[1, col].set_title(f'Magnitude (px)', fontsize=10)
67             axes[1, col].axis('off')
68

```

```

69     plt.suptitle('RAFT Iterative Flow Refinement', fontsize=14, fontweight
    = 'bold')
70     plt.tight_layout()
71     plt.savefig(save_path, dpi=150, bbox_inches='tight')
72     plt.show()
73
74     # Quantitative: EPE across iterations (shows convergence)
75     print("\nConvergence across iterations (if GT is available):")
76     print("Iteration | Approx. Flow Change")
77     print("-" * 35)
78     for i in range(1, len(flow_preds)):
79         delta = (flow_preds[i] - flow_preds[i-1]).norm(dim=1).mean().item
    ()
80         print(f" {i:3d} -> {i+1:3d} | {delta:.4f} px mean change")
81     # Expect: large changes early, small changes later (convergence)

```

Listing 10.2: Visualizing RAFT’s iterative refinement across iterations

Chapter 10 Summary: RAFT

- RAFT (ECCV 2020) broke from the pyramid-warping paradigm with three key innovations.
- **All-pairs 4D correlation volume:** precomputed between every pair of pixels across frames — no bounded search range, no missed matches.
- **Correlation pyramid:** average-pool the 4D volume along frame 2 dimensions for multi-scale matching at negligible extra cost.
- **ConvGRU update operator:** iteratively refines the full flow field by reading from the correlation volume at current predicted positions — learned optimization, not coarse-to-fine.
- Multi-iteration training loss with exponentially increasing weights (γ^{K-k}) encourages fast convergence and high final accuracy.
- Learned $8\times$ flow upsampling (convex combination of neighbors) outperforms bilinear upsampling, especially at boundaries.

Connection to Your Background

RAFT’s ConvGRU update operator is directly built on the GRU architecture you implemented from scratch. The gates (update gate z , reset gate r) and hidden state update equations are identical — only the matrix multiplications are replaced by 3×3 convolutions to handle 2D spatial structure. Your intuition about how GRUs manage long-term dependencies via the gating mechanism translates directly: the hidden state $\mathbf{h}^k$ in RAFT accumulates evidence about the flow across K iterations, just as an RNN hidden state accumulates context across time steps. The key difference is that the “time steps” are iterations of a search procedure, not temporal frames.

Research Questions to Explore

1. **Iteration count and accuracy:** RAFT uses 12 iterations at test time. Does adding more iterations always improve accuracy, or does the GRU converge by iteration 12? What would you observe if you ran 24 iterations? How does this connect to the convergence theory of iterative optimization methods?
2. **The all-pairs correlation memory cost:** For input images of 1280×720 , the $1/8$ resolution is $160 \times 90 = 14,400$ pixels. The all-pairs correlation matrix has $14,400^2 = 207\text{M}$ entries per image pair. How does RAFT manage this in practice? What are the GPU memory implications, and what tricks (like level-specific pooling before computing correlation) could reduce this?
3. **RAFT for video:** RAFT processes frame pairs independently. For video optical flow, adjacent pairs share a frame (frames t and $t + 1$, then frames $t + 1$ and $t + 2$). How could you exploit the temporal coherence of video to improve accuracy or reduce computation? What role does the warm-start option (`flow_init`) play in this?

Chapter 11

FlowFormer: Transformers for Optical Flow

FlowFormer asks: what if the cost volume could attend to itself globally, learning which matches to trust?

This chapter is intentionally different from the previous deep learning chapters. There is no implementation here — only deep conceptual understanding. FlowFormer [?] represents a research-level architectural insight, and our goal is to understand it well enough to *discuss it with a professor*, connect it to your Conformer architecture, and identify research questions it raises. This is the chapter that bridges your existing work to the frontier.

11.1 Why Transformers for Optical Flow?

RAFT achieves remarkable accuracy, but it has a fundamental architectural limitation: the GRU update operator is **spatially local**. A 3×3 ConvGRU can only integrate information from a 3×3 pixel neighborhood at each iteration. Even after K iterations, the effective receptive field grows only as $3 \times 3 \times K$ — for $K = 12$, that is 37×37 pixels at $1/8$ resolution, roughly 296×296 pixels at full resolution. For a 1280×720 image, many pixels are updating their flow estimates without “seeing” what is happening on the other side of the image.

This matters when:

- A large uniform-colored object (e.g., the hood of a truck) needs to infer its flow from distant textured regions at its boundary — the boundary information does not reach the center within a limited receptive field.
- An occluded region needs to reason about what is occluding it (which may be far away) to estimate plausible flow.
- Two identical patches in different locations need to be disambiguated using global context.

Transformers, with their global self-attention mechanism, can integrate information across the entire image in a single layer. FlowFormer applies this global reasoning to

the cost volume, allowing every position in the flow field to be informed by every other position’s matching evidence.

11.2 FlowFormer’s Architecture at a Conceptual Level

FlowFormer has four main components:

1. **Feature extraction:** Same as RAFT — a shared CNN encoder extracts features from both frames at $1/8$ resolution.
2. **Cost volume construction:** Same as RAFT — all-pairs correlation volume $\mathbf{C}$.
3. **Cost volume encoding with Transformers:** This is the key innovation. Instead of reading the cost volume locally (RAFT’s GRU lookup in a $r \times r$ neighborhood), FlowFormer encodes the *entire cost volume* using Transformer attention.
4. **Iterative decoding:** Like RAFT, flow is iteratively refined, but now guided by globally-aware cost volume features.

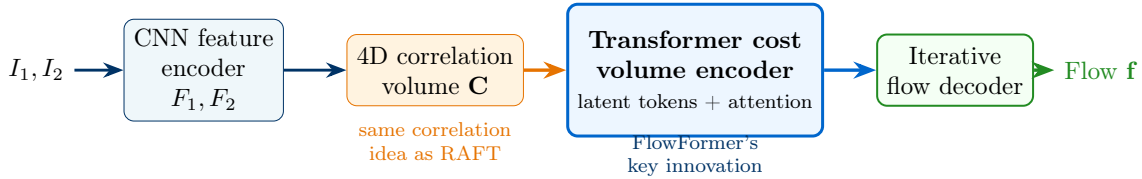


Figure 11.1: FlowFormer architecture. Feature extraction and all-pairs correlation are inherited from RAFT. The key innovation is replacing RAFT’s local GRU lookup with a global Transformer encoder that processes the entire cost volume, enabling every pixel to be informed by every other pixel’s matching evidence.

11.3 Cost Volume Encoding with Transformers

The central question of FlowFormer’s design: *how do you encode a 4D cost volume using a Transformer?*

A naive approach would flatten the entire 4D volume into a sequence and apply self-attention. But for a $50 \times 87 \times 50 \times 87$ cost volume, that sequence has $50 \times 87 = 4350$ elements, and the $O(n^2)$ attention cost would be $4350^2 \approx 19\text{M}$ attention operations per layer. Expensive.

FlowFormer’s solution is more elegant. It introduces **cost latent tokens** — a compact set of learnable query vectors that summarize the matching cost information:

1. For each source pixel p_i in frame 1, its row of the cost volume $\mathbf{C}_{p_i,:}$ describes its similarity to all frame 2 positions. This is a 1D vector of length $H'W'$.
2. Instead of processing this entire vector, FlowFormer uses **cross-attention** where a small set of “latent” tokens attend to the cost entries, extracting the most relevant matching information into a compact representation.

3. The latent tokens for different source pixels then attend to each other via **self-attention**, sharing information about which matches are globally consistent.

This design is $O(n \cdot m)$ where $n = H'W'$ source pixels and $m \ll n$ is the number of latent tokens — much more tractable than $O(n^2)$.

11.4 Cross-Attention for Frame Matching

Within the cost volume encoder, FlowFormer uses cross-attention in a way that is deeply analogous to how Transformers are used for sequence matching tasks (like machine translation).

The standard cross-attention in a Transformer:

$$\text{CrossAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (11.1)$$

In FlowFormer’s cost volume encoder:

- **Queries** Q : The cost latent tokens for source pixel p_i — what we are trying to summarize.
- **Keys** K : The positions in frame 2’s feature map — what we are matching against.
- **Values** V : The frame 2 features, potentially weighted by the cost — the information we extract.

The attention weight $\text{softmax}(QK^T/\sqrt{d_k})_{ij}$ is a learned, differentiable analog of “how much should source position i ’s summary attend to frame 2 position j ’s features?” This is directly related to the cost volume entry $\mathbf{C}_{ij}$ — high similarity in the correlation volume should correspond to high attention weight. But crucially, the Transformer learns a *non-linear* function of these similarities, rather than using them directly.

11.5 Connection to Your Conformer Architecture

This is where your existing work becomes directly relevant. Your Conformer for Nepali ASR is built on the architecture from Gulati et al. [?], which combines:

- **Multi-head self-attention** (Transformer): captures long-range temporal dependencies in the audio sequence.
- **Convolution modules**: captures local structure, which is important for speech because phonemes have local spectral patterns.

FlowFormer has a strikingly similar philosophy:

- **Transformer cost volume encoder**: captures global spatial dependencies in the matching cost — which matches are globally consistent?
- **CNN feature extractor**: captures local structure, which is important for flow because motion boundaries are local phenomena.

Table 11.1: Structural comparison: Conformer for ASR vs. FlowFormer for optical flow

Aspect	Conformer (ASR)	FlowFormer (Optical Flow)
Input modality	Audio spectrogram sequences	Image pairs (video frames)
Local structure	Convolutional module (phoneme patterns)	CNN feature extractor (image textures, edges)
Global reasoning	Multi-head self-attention (temporal context)	Transformer cost volume encoder (global match consistency)
What attention attends to	Audio feature vectors at different time steps	Matching costs across all spatial positions
Why local+global?	Both phoneme-level and utterance-level context needed for ASR	Both local texture matching and global scene consistency needed for flow
Key insight	CNN captures local; Transformer captures global	CNN captures local image structure; Transformer reasons about global cost consistency

This is not a superficial analogy. Both architectures embody the same **dual-stream philosophy**: local processing for local structure, global processing for long-range coherence. The insight you gained implementing Conformer — understanding how to combine convolutional and attentional components effectively — directly applies to understanding why FlowFormer is designed the way it is.

Intuition First

Research narrative you can tell: “In building my Conformer for Nepali ASR, I learned that the key architectural tension is between local specificity (convolution for phoneme patterns) and global coherence (attention for utterance-level context). When I studied FlowFormer, I recognized the same tension playing out in spatial rather than temporal form: CNN for local texture matching, Transformer for global cost consistency. This architectural isomorphism suggests a deeper principle about when and how to combine local and global processing, which I would like to explore further.” This is the kind of research insight that distinguishes a thinker from a practitioner.

11.6 What Questions FlowFormer Raises for Future Research

FlowFormer is not the end of the story — it raises new questions that define the current research frontier.

1. Pre-training for optical flow: FlowFormer++ (the follow-up paper [?]) introduced masked cost volume autoencoding as a pre-training strategy, similar to MAE (masked autoencoders) for images. Can we develop a foundation model for flow

that, like BERT for NLP or DINO for vision, learns rich representations from unlabeled video that transfer well to diverse flow estimation tasks?

2. The scale of global attention: FlowFormer processes the cost volume at $1/8$ resolution. What happens at higher resolutions — can we maintain global attention while scaling to full resolution? This connects to the efficiency research in long-sequence Transformers (Longformer, BigBird) that you may have encountered in ASR contexts.

3. Cross-modal flow: The Conformer’s success in ASR with a CNN+Transformer hybrid suggests the architecture generalizes beyond images. Could a similar architecture estimate flow between different modalities — e.g., between a thermal camera frame and an RGB frame? What would the cost volume look like in a cross-modal setting?

4. Scene flow with FlowFormer: If FlowFormer can robustly estimate 2D optical flow using global Transformer reasoning, can the same ideas be extended to 3D scene flow? The 3D cost volume for scene flow (comparing point cloud features across frames) is an even more natural application of global attention.

Chapter 11 Summary: FlowFormer

- FlowFormer (ECCV 2022) replaces RAFT’s local GRU cost volume lookup with a global Transformer encoder.
- **Motivation:** RAFT’s ConvGRU has limited receptive field; Transformers provide global integration of matching evidence.
- **Cost latent tokens:** A compact set of learned query vectors that summarize matching cost via cross-attention, avoiding the $O(n^2)$ cost of full attention.
- **Cross-attention for matching:** Latent tokens attend to frame 2 features, guided by cost volume similarities; then self-attention shares information across source positions.
- **Deep connection to Conformer:** Both architectures combine CNN (local structure) with Transformer (global coherence) — the same dual-stream philosophy applied to different modalities and tasks.
- FlowFormer raises important research questions about pre-training, scale, cross-modal flow, and scene flow.

Connection to Your Background

You have implemented the Conformer architecture, which means you have direct experience with multi-head self-attention, cross-attention, and the architectural challenge of combining local (convolutional) and global (attentional) processing. Every one of these experiences is directly applicable to understanding FlowFormer. When a professor asks you about FlowFormer, you can say: “I find FlowFormer’s architecture particularly intuitive because it mirrors the Conformer design I implemented for Nepali ASR — both architectures use convolution for local feature extraction and Transformer attention for global reasoning, differing primarily in whether the global reasoning operates over temporal sequences (ASR) or spatial cost volumes (flow).” This is a genuinely insightful observation, not a canned answer.

Research Questions to Explore

1. **The cost latent bottleneck:** FlowFormer compresses each pixel's cost row into a small number of latent tokens. What is the information-theoretic effect of this bottleneck? Does it help regularize the cost volume by removing noisy, inconsistent matches? Could you design an experiment to test whether the bottleneck is beneficial or limiting?
2. **Attention patterns in the cost volume encoder:** In NLP Transformers, attention heads have been shown to specialize — some attend to syntactic relations, others to semantic similarities. In FlowFormer, do different attention heads specialize for different types of matches (e.g., textured vs. edge-based, short vs. long-range)? How would you design a visualization experiment to test this?
3. **From FlowFormer to SceneFormer:** FlowFormer processes 4D cost volumes over 2D image features. Scene flow requires a 6D cost volume over 3D point cloud features. How would you adapt FlowFormer's cost volume encoding strategy to the 3D case? What new challenges arise when the input is a point cloud rather than a regular image grid?

Chapter 12

KITTI Dataset and PyTorch Data Pipeline

The dataset is the experiment. Understanding its structure means understanding what you are really measuring.

12.1 What KITTI Contains

The KITTI Vision Benchmark Suite [?] was created by the Karlsruhe Institute of Technology and Toyota Technological Institute at Chicago. It is the primary real-world benchmark for autonomous driving perception, including optical flow.

For optical flow, KITTI provides:

- **Image pairs:** Consecutive frames from a stereo camera mounted on a car driving through Karlsruhe, Germany
- **Sparse ground-truth flow:** Derived by projecting 3D LIDAR point clouds onto the image using calibration data, then computing the displacement between consecutive projections
- **KITTI 2012 (KITTI Flow):** 194 training pairs, 195 test pairs. Predominantly rigid scenes (buildings, roads). Average displacement: moderate.
- **KITTI 2015:** 200 training pairs, 200 test pairs. Includes non-rigid objects (cars, pedestrians). More challenging. This is the primary benchmark.

Why KITTI is challenging:

- Sparse GT: only ~30–35% of pixels have LIDAR-derived ground truth
- Mixed scenes: both static background and independently moving objects
- Variable lighting and weather conditions
- Real motion blur and sensor noise
- Non-rigid objects: deformable motion (cars changing shape as they turn)

12.2 Complete PyTorch DataLoader Implementation

```

1 import os
2 import numpy as np
3 import torch
4 from torch.utils.data import Dataset, DataLoader
5 import cv2
6 import glob
7 from typing import Optional, Tuple
8
9
10 def read_kitti_flow(flow_path: str) -> Tuple[np.ndarray, np.ndarray]:
11     """
12     Read KITTI optical flow ground truth from .png file.
13
14     KITTI stores flow in a 16-bit PNG with 3 channels using a special
15     encoding:
16         channel 0 (B): valid flag (1 = valid pixel, 0 = no GT for this pixel)
17         channels 1,2 (G,R): encoded flow as 16-bit unsigned integers
18
19     Flow encoding: pixel_value = (flow + 215) / 64.0
20     So to decode: flow = (pixel_value * 64.0 - 215) / 1.0
21
22     This encoding stores flow values in the range approximately [-512,
23     512] pixels.
24     Values outside this range are clipped (very rare in practice).
25
26     Args:
27         flow_path: Path to the .png flow file
28
29     Returns:
30         flow: float32 array of shape (H, W, 2): [u (horizontal), v (vertical)]
31         valid: bool array of shape (H, W): True where GT flow exists
32     """
33     # Read as 16-bit (uint16) to preserve the encoded values
34     flow_img = cv2.imread(flow_path, cv2.IMREAD_ANYCOLOR | cv2.IMREAD_ANYDEPTH)
35
36     if flow_img is None:
37         raise FileNotFoundError(f"Cannot read flow file: {flow_path}")
38
39     # flow_img shape: (H, W, 3), dtype: uint16
40     # OpenCV reads BGR order, so:
41     #   channel 0 = B = valid flag
42     #   channel 1 = G = v (vertical flow, encoded)
43     #   channel 2 = R = u (horizontal flow, encoded)
44
45     valid = flow_img[:, :, 0] > 0 # Valid mask: True where LIDAR hit

```



```

45     # Decode flow from uint16 to float:
46     # stored_value = (true_flow * 64) + 32768
47     # true_flow = (stored_value - 32768) / 64
48     u = (flow_img[:, :, 2].astype(np.float32) - 32768.0) / 64.0 #
horizontal
49     v = (flow_img[:, :, 1].astype(np.float32) - 32768.0) / 64.0 #
vertical
50
51     flow = np.stack([u, v], axis=-1) # (H, W, 2)
52
53     return flow.astype(np.float32), valid.astype(bool)
54
55
56 class KITTIFlowDataset(Dataset):
57     """
58     PyTorch Dataset for KITTI 2015 Optical Flow.
59
60     Directory structure expected:
61     kitti_root/
62         training/
63             image_2/    <- left camera images (frame 0 and frame 1)
64                 000000_10.png (frame 0 of sequence 0)
65                 000000_11.png (frame 1 of sequence 0)
66                 000001_10.png (frame 0 of sequence 1)
67                 ...
68             flow_occ/    <- ground truth flow (occluded regions included)
69                 000000_10.png
70                 ...
71
72     Download from: http://www.cvlibs.net/datasets/kitti/eval\_scene\_flow.php
73     """
74
75     def __init__(
76         self,
77         root: str,
78         split: str = 'training',
79         augment: bool = True,
80         crop_size: Tuple[int, int] = (368, 496), # (H, W) crop for
training
81     ):
82         """
83         Args:
84             root: Path to KITTI root directory
85             split: 'training' or 'testing'
86             augment: Apply data augmentation (flipping, color jitter)
87             crop_size: Random crop size for training
88         """
89         self.root = root
90         self.split = split
91         self.augment = augment

```

```

92     self.crop_size = crop_size
93
94     # Find all image pairs
95     image_dir = os.path.join(root, split, 'image_2')
96     flow_dir = os.path.join(root, split, 'flow_occ')
97
98     # Frame 0 images are named *_10.png, frame 1 are *_11.png
99     frame0_paths = sorted(glob.glob(os.path.join(image_dir, '*_10.png')
)))
100
101     self.samples = []
102     for frame0_path in frame0_paths:
103         # Derive frame1 and flow paths from frame0 path
104         frame1_path = frame0_path.replace('_10.png', '_11.png')
105         flow_path = frame0_path.replace('image_2', 'flow_occ')
106
107         # Only include if all files exist
108         if (os.path.exists(frame1_path) and
109             (split == 'testing' or os.path.exists(flow_path))):
110             self.samples.append({
111                 'frame0': frame0_path,
112                 'frame1': frame1_path,
113                 'flow': flow_path if split == 'training' else None,
114             })
115
116     print(f"KITTIFlowDataset ({split}): {len(self.samples)} samples
loaded")
117
118     def __len__(self) -> int:
119         return len(self.samples)
120
121     def _load_image(self, path: str) -> np.ndarray:
122         """Load image as float32 RGB in [0, 1]"""
123         img = cv2.imread(path)
124         if img is None:
125             raise FileNotFoundError(f"Cannot read image: {path}")
126         img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB).astype(np.float32) /
255.0
127         return img
128
129     def _augment(
130         self,
131         img1: np.ndarray,
132         img2: np.ndarray,
133         flow: Optional[np.ndarray],
134         valid: Optional[np.ndarray]
135     ) -> tuple:
136         """
137         Apply data augmentation for training.
138
139         Augmentations:

```

```

140     1. Random horizontal flip (must flip both images AND negate u-
component of flow)
141     2. Random crop to self.crop_size (same crop for both images and
flow)
142     3. Color jitter (applied INDEPENDENTLY to each frame -- simulates
143         different lighting conditions between consecutive frames)
144
145     Color jitter must be applied independently to each frame because:
146     - Real consecutive frames can have slightly different exposures
147     - Training on independently jittered frames makes the network
148         more robust to illumination changes (the BCA violation in Ch.6)
149     """
150     H, W = img1.shape[:2]
151
152     # --- 1. Random horizontal flip ---
153     if np.random.random() < 0.5:
154         img1 = img1[:, ::-1, :]          # flip horizontally
155         img2 = img2[:, ::-1, :]
156         if flow is not None:
157             flow = flow[:, ::-1, :]      # flip spatial positions
158             flow[:, :, 0] *= -1          # negate horizontal flow (u)
159                                         # (object that moved right now
appears to move left)
160             valid = valid[:, ::-1]
161
162     # --- 2. Random crop ---
163     crop_h, crop_w = self.crop_size
164     # Ensure we have enough room to crop
165     if H > crop_h and W > crop_w:
166         y0 = np.random.randint(0, H - crop_h)
167         x0 = np.random.randint(0, W - crop_w)
168         img1 = img1[y0:y0+crop_h, x0:x0+crop_w]
169         img2 = img2[y0:y0+crop_h, x0:x0+crop_w]
170         if flow is not None:
171             flow = flow[y0:y0+crop_h, x0:x0+crop_w]
172             valid = valid[y0:y0+crop_h, x0:x0+crop_w]
173
174     # --- 3. Independent color jitter per frame ---
175     def color_jitter(img: np.ndarray) -> np.ndarray:
176         """Apply random brightness, contrast, saturation, hue jitter"""
177
178         # Brightness: multiply by random factor in [0.6, 1.4]
179         img = img * np.random.uniform(0.6, 1.4)
180         # Contrast: adjust per-channel
181         mean = img.mean(axis=(0,1), keepdims=True)
182         img = (img - mean) * np.random.uniform(0.8, 1.2) + mean
183         # Clip to valid range
184         return np.clip(img, 0, 1)
185
186     img1 = color_jitter(img1)

```

```

186         img2 = color_jitter(img2)    # Independent jitter: simulates
illumination change
187
188         return img1, img2, flow, valid
189
190     def __getitem__(self, idx: int) -> dict:
191         """
192         Load a single sample.
193
194         Returns dict with:
195         'image1': Tensor (3, H, W), float32, normalized to [-1, 1] for
RAFT
196         'image2': Tensor (3, H, W), float32, normalized to [-1, 1] for
RAFT
197         'flow': Tensor (2, H, W), float32, in pixels (u, v)
198         'valid': Tensor (H, W), bool, True where GT flow exists
199         """
200         sample = self.samples[idx]
201
202         # Load images
203         img1 = self._load_image(sample['frame0'])    # (H, W, 3) float32
[0,1]
204         img2 = self._load_image(sample['frame1'])
205
206         # Load flow (training only)
207         flow, valid = None, None
208         if sample['flow'] is not None and os.path.exists(sample['flow']):
209             flow, valid = read_kitti_flow(sample['flow'])
210             # flow: (H, W, 2), valid: (H, W, bool)
211
212         # Apply augmentation during training
213         if self.augment and self.split == 'training':
214             img1, img2, flow, valid = self._augment(img1, img2, flow,
valid)
215
216         # Convert to tensors
217         # Normalize images to [-1, 1] for RAFT (which expects this range)
218         img1_t = torch.from_numpy(img1 * 2 - 1).permute(2, 0, 1).float()
# (3,H,W)
219         img2_t = torch.from_numpy(img2 * 2 - 1).permute(2, 0, 1).float()
220
221         result = {'image1': img1_t, 'image2': img2_t, 'index': idx}
222
223         if flow is not None:
224             # flow: (H, W, 2) -> (2, H, W)
225             flow_t = torch.from_numpy(flow).permute(2, 0, 1).float()
226             valid_t = torch.from_numpy(valid).bool()
227             result.update({'flow': flow_t, 'valid': valid_t})
228
229         return result
230

```

```

231
232 def create_kitti_dataloaders(
233     kitti_root: str,
234     batch_size: int = 4,
235     num_workers: int = 4,
236     crop_size: Tuple[int, int] = (368, 496)
237 ) -> Tuple[DataLoader, DataLoader]:
238     """
239     Create train and validation DataLoaders for KITTI flow.
240
241     Note: KITTI's official test set has no publicly available GT.
242     We use a 90/10 train/val split of the training set for development.
243     For final evaluation, submit to the KITTI online evaluation server.
244     """
245     full_dataset = KITTIFlowDataset(
246         root=kitti_root, split='training',
247         augment=True, crop_size=crop_size
248     )
249
250     # 90/10 split
251     n_total = len(full_dataset)
252     n_val = max(1, n_total // 10)
253     n_train = n_total - n_val
254
255     train_set, val_set = torch.utils.data.random_split(
256         full_dataset, [n_train, n_val],
257         generator=torch.Generator().manual_seed(42) # Reproducible split
258     )
259     # Disable augmentation for validation set
260     val_set.dataset.augment = False
261
262     train_loader = DataLoader(
263         train_set,
264         batch_size=batch_size,
265         shuffle=True, # Shuffle for training diversity
266         num_workers=num_workers,
267         pin_memory=True, # Speed up GPU transfer
268         drop_last=True, # Drop incomplete batches for consistent
269         batch_size
270     )
271
272     val_loader = DataLoader(
273         val_set,
274         batch_size=1, # Evaluate one at a time (full resolution,
275         no crop)
276         shuffle=False,
277         num_workers=num_workers,
278         pin_memory=True,
279     )
280
281     return train_loader, val_loader

```

```

280
281
282 def visualize_kitti_sample(sample: dict, save_path: str = 'kitti_sample.
    png'):
283     """
284     Visualize a KITTI sample: both frames, GT flow (HSV), and valid mask.
285
286     This is the first thing you should run after setting up your data
    pipeline:
287     visualize a few samples to verify the GT flow looks reasonable and
288     the images are loading correctly.
289     """
290     import matplotlib.pyplot as plt
291
292     img1 = (sample['image1'].numpy().transpose(1,2,0) + 1) / 2    # [-1,1]
    -> [0,1]
293     img2 = (sample['image2'].numpy().transpose(1,2,0) + 1) / 2
294
295     fig_cols = 2
296     if 'flow' in sample:
297         fig_cols = 4
298         flow = sample['flow'].numpy()    # (2, H, W)
299         valid = sample['valid'].numpy()  # (H, W)
300
301         # Convert flow to HSV visualization
302         u, v = flow[0], flow[1]
303         mag, ang = cv2.cartToPolar(u, v)
304         hsv = np.zeros((*u.shape, 3), dtype=np.float32)
305         hsv[:, :, 0] = ang * 180 / np.pi / 2
306         hsv[:, :, 1] = 1.0
307         max_m = np.percentile(mag[valid], 99) + 1e-6
308         hsv[:, :, 2] = np.clip(mag / max_m, 0, 1)
309         hsv[:, :, 2][~valid] = 0    # Black out invalid pixels
310         flow_rgb = cv2.cvtColor(
311             (hsv * [180,255,255]).astype(np.uint8), cv2.COLOR_HSV2RGB
312         ) / 255.0
313
314     fig, axes = plt.subplots(1, fig_cols, figsize=(5 * fig_cols, 4))
315     axes[0].imshow(img1); axes[0].set_title('Frame 1'); axes[0].axis('off'
    )
316     axes[1].imshow(img2); axes[1].set_title('Frame 2'); axes[1].axis('off'
    )
317
318     if 'flow' in sample:
319         axes[2].imshow(flow_rgb)
320         axes[2].set_title(f'GT Flow (HSV)\n{valid.mean()*100:.0f}% valid
    pixels')
321         axes[2].axis('off')
322         axes[3].imshow(valid, cmap='gray')
323         axes[3].set_title('Valid Mask\n(white=GT available)')
324         axes[3].axis('off')

```

```

325
326     plt.tight_layout()
327     plt.savefig(save_path, dpi=150, bbox_inches='tight')
328     plt.show()

```

Listing 12.1: Complete KITTI optical flow dataset and DataLoader implementation

Chapter 12 Summary: KITTI Dataset and Data Pipeline

- KITTI 2015 is the primary real-world optical flow benchmark: 200 training pairs with sparse LIDAR-derived GT, covering autonomous driving scenarios.
- GT flow is stored as 16-bit PNG with special encoding: $\text{flow} = (\text{pixel_value} - 32768) / 64$.
- Only $\sim 35\%$ of pixels have valid GT; the `valid` mask is critical for computing meaningful loss and metrics.
- Data augmentation: horizontal flipping (negate u -component of flow), random cropping, and *independent* color jitter per frame (simulates illumination changes).
- Always visualize a few samples before training to verify the data pipeline is correct.

Connection to Your Background

The data pipeline you build here (KITTI DataLoader with proper flow decoding, augmentation, and valid masking) is the same infrastructure used in published RAFT and FlowFormer papers. Your experience building data pipelines for your Nepali ASR project (loading audio, computing spectrograms, padding/batching variable-length sequences) directly transfers here — the challenges are similar: reading proprietary formats, handling missing data (invalid GT vs. padding in ASR), and designing augmentations that preserve the relationship between inputs and labels.

Research Questions to Explore

1. **The sparsity problem:** Only 35% of pixels have GT flow in KITTI. During evaluation, we can only measure accuracy on these pixels. Does this create a systematic bias? Are valid GT pixels representative of the full distribution of scene content, or do they systematically over-represent certain regions (e.g., road surface, close objects)?
2. **Augmentation and domain gap:** We apply color jitter to simulate illumination changes. But real KITTI illumination changes are not just brightness/contrast shifts — they involve shadows, reflections, and highlights. How would you design more realistic illumination augmentations? Could you use physically-based rendering to generate synthetic illumination variations?
3. **Temporal consistency in video:** KITTI provides isolated frame pairs, but a real video stream has temporal continuity. How could you design a dataset loader that exploits temporal consistency — e.g., using previous frame's flow as a warm-start for the next pair? What additional loss terms could enforce temporal consistency?

Chapter 13

Complete Comparative Study Pipeline

A fair comparison is harder to design than any individual method. Fairness requires discipline.

13.1 Designing a Fair Comparison

The purpose of your comparative study is not just to show that RAFT is better than Lucas-Kanade (everyone already knows this). The purpose is to demonstrate:

1. Your ability to evaluate methods rigorously using the same data and metrics
2. Your understanding of *why* one method outperforms another in specific scenarios
3. Your ability to frame results as a researcher would — with caveats, insights, and questions

The three golden rules of a fair comparison:

- **Same data, same preprocessing:** Every method sees exactly the same frames in the same order, with no method-specific preprocessing that gives it an advantage.
- **Same evaluation metric:** EPE and F1-all computed identically for all methods, only over valid GT pixels.
- **Tuned parameters for each method:** Classical methods have parameters (window size, pyramid levels) that should be set to their recommended values — not unfairly tuned to make them look worse.

13.2 Complete Evaluation Pipeline

```
1 import cv2
2 import numpy as np
3 import torch
4 import torch.nn.functional as F
```

```

5 from typing import Callable, Dict, List, Tuple
6 import time
7 import matplotlib.pyplot as plt
8
9
10 #
=====
11 # UNIFIED METRICS
12 #
=====
13
14 def compute_epe(flow_pred: np.ndarray,
15                 flow_gt: np.ndarray,
16                 valid: np.ndarray) -> float:
17     """
18     Compute End-Point Error (EPE) --- the primary optical flow metric.
19
20     EPE = mean Euclidean distance between predicted and GT flow vectors,
21     computed ONLY over valid pixels (where KITTI has LIDAR GT).
22
23     Args:
24         flow_pred: Predicted flow, shape (H, W, 2) [u, v] in pixels
25         flow_gt:    GT flow, shape (H, W, 2) [u, v] in pixels
26         valid:      Valid GT mask, shape (H, W), bool
27
28     Returns:
29         epe: Mean EPE in pixels over valid pixels
30     """
31     # Per-pixel Euclidean distance
32     epe_map = np.sqrt(
33         (flow_pred[:, :, 0] - flow_gt[:, :, 0])**2 +
34         (flow_pred[:, :, 1] - flow_gt[:, :, 1])**2
35     )
36     # Average over valid pixels only
37     return float(epe_map[valid].mean())
38
39
40 def compute_f1(flow_pred: np.ndarray,
41                 flow_gt: np.ndarray,
42                 valid: np.ndarray) -> float:
43     """
44     Compute F1 outlier metric (KITTI's primary metric).
45
46     A pixel is an outlier if its EPE is BOTH:
47     - Absolutely large: EPE > 3 pixels
48     - Relatively large: EPE > 5% of the GT flow magnitude
49
50     The relative criterion prevents penalizing small absolute errors
51     on fast-moving pixels where 3 pixels might be a tiny fraction of

```

```

52     the true displacement.
53
54     Returns:
55         f1: Fraction of outlier pixels (lower is better, 0% = perfect)
56     """
57     epe_map = np.sqrt(
58         (flow_pred[:, :, 0] - flow_gt[:, :, 0])**2 +
59         (flow_pred[:, :, 1] - flow_gt[:, :, 1])**2
60     )
61     gt_mag = np.sqrt(flow_gt[:, :, 0]**2 + flow_gt[:, :, 1]**2)
62
63     # Outlier: absolute error > 3px AND relative error > 5%
64     outlier = (epe_map > 3.0) & (epe_map > 0.05 * gt_mag)
65
66     # Fraction of valid pixels that are outliers
67     return float(outlier[valid].mean()) * 100.0    # Return as percentage
68
69
70 #
=====
71 # METHOD WRAPPERS
72 #
=====
73
74 def run_lucas_kanade(
75     frame1: np.ndarray,
76     frame2: np.ndarray
77 ) -> np.ndarray:
78     """
79     Sparse Lucas-Kanade optical flow, converted to dense representation.
80
81     LK produces flow only at corner features. For fair comparison with
82     dense methods, we compute it only at detected features and leave
83     other pixels as zero (which evaluates poorly in texture-less regions,
84     but this is the honest representation of LK's output).
85
86     Alternative: use LK flow to seed a radial basis function interpolation
87     to get a smooth dense field. We implement the honest version.
88
89     Args:
90         frame1, frame2: RGB uint8 images, shape (H, W, 3)
91
92     Returns:
93         flow: Dense flow, shape (H, W, 2), zeros where LK has no estimate
94     """
95     gray1 = cv2.cvtColor(frame1, cv2.COLOR_RGB2GRAY)
96     gray2 = cv2.cvtColor(frame2, cv2.COLOR_RGB2GRAY)
97     H, W = gray1.shape
98

```

```

99     # Initialize output as zero (undefined)
100     flow_dense = np.zeros((H, W, 2), dtype=np.float32)
101
102     # Detect good features to track
103     features = cv2.goodFeaturesToTrack(
104         gray1, maxCorners=2000, qualityLevel=0.005, minDistance=5
105     )
106     if features is None or len(features) == 0:
107         return flow_dense
108
109     # Track with pyramidal LK
110     features_new, status, _ = cv2.calcOpticalFlowPyrLK(
111         gray1, gray2, features, None,
112         winSize=(15, 15),
113         maxLevel=3,
114         criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 30,
0.03)
115     )
116
117     # Fill in flow at tracked locations
118     ok = status.flatten() == 1
119     for new, old in zip(features_new[ok], features[ok]):
120         x_new, y_new = new.ravel()
121         x_old, y_old = old.ravel()
122         xi, yi = int(round(x_old)), int(round(y_old))
123         if 0 <= xi < W and 0 <= yi < H:
124             flow_dense[yi, xi, 0] = x_new - x_old    # u
125             flow_dense[yi, xi, 1] = y_new - y_old    # v
126
127     return flow_dense
128
129
130 def run_farneback(
131     frame1: np.ndarray,
132     frame2: np.ndarray
133 ) -> np.ndarray:
134     """
135     Dense Farneback optical flow.
136
137     Args:
138         frame1, frame2: RGB uint8 images
139
140     Returns:
141         flow: Dense flow, shape (H, W, 2)
142     """
143     gray1 = cv2.cvtColor(frame1, cv2.COLOR_RGB2GRAY)
144     gray2 = cv2.cvtColor(frame2, cv2.COLOR_RGB2GRAY)
145
146     flow = cv2.calcOpticalFlowFarneback(
147         gray1, gray2, None,
148         pyr_scale=0.5, levels=5, winsize=13,

```

```

149         iterations=10, poly_n=5, poly_sigma=1.1, flags=0
150     )
151     return flow    # (H, W, 2)
152
153
154 def run_raft(
155     frame1: np.ndarray,
156     frame2: np.ndarray,
157     model: torch.nn.Module,
158     device: str = 'cuda'
159 ) -> np.ndarray:
160     """
161     RAFT optical flow (deep learning).
162
163     Args:
164         frame1, frame2: RGB uint8 images
165         model: Loaded RAFT model
166
167     Returns:
168         flow: Dense flow, shape (H, W, 2)
169     """
170     def preprocess(img: np.ndarray) -> torch.Tensor:
171         """RGB uint8 [0,255] -> float tensor [-1,1], shape (1,3,H,W)"""
172         t = torch.from_numpy(img.astype(np.float32) / 127.5 - 1.0)
173         return t.permute(2,0,1).unsqueeze(0).to(device)
174
175     t1, t2 = preprocess(frame1), preprocess(frame2)
176
177     with torch.no_grad():
178         flow_preds = model(t1, t2, iters=12, test_mode=True)
179
180     flow = flow_preds[-1][0].cpu().numpy().transpose(1, 2, 0)    # (H, W, 2)
181     return flow
182
183
184 #
185 =====
186
187 # MAIN EVALUATION LOOP
188 #
189 =====
190
191 def run_comparative_evaluation(
192     kitti_samples: List[Dict],
193     raft_model: torch.nn.Module,
194     device: str = 'cuda',
195     num_samples: int = 50
196 ) -> Dict:
197     """
198     Run all three methods on the same KITTI samples and collect metrics.

```

```

196
197     Args:
198         kitti_samples: List of dicts with 'image1', 'image2', 'flow', '
valid'
199         raft_model:    Pre-loaded RAFT model
200         device:        'cuda' or 'cpu'
201         num_samples:   Number of samples to evaluate
202
203     Returns:
204         results: Dict with EPE, F1, and runtime for each method
205     """
206     methods = {
207         'Lucas-Kanade': {'fn': run_lucas_kanade, 'epe': [], 'f1': [], '
time': []},
208         'Farneback':    {'fn': run_farneback,    'epe': [], 'f1': [], '
time': []},
209         'RAFT':         {'fn': None,             'epe': [], 'f1': [], '
time': []},
210     }
211
212     samples_to_eval = kitti_samples[:num_samples]
213
214     for sample_idx, sample in enumerate(samples_to_eval):
215         # Load images as RGB uint8 for OpenCV methods
216         img1 = sample['image1'] # Assume already numpy RGB uint8
217         img2 = sample['image2']
218         flow_gt = sample['flow'] # (H, W, 2)
219         valid = sample['valid'] # (H, W) bool
220
221         print(f"Sample {sample_idx+1}/{num_samples}", end='\r')
222
223         # --- Lucas-Kanade ---
224         t0 = time.time()
225         lk_flow = run_lucas_kanade(img1, img2)
226         methods['Lucas-Kanade']['time'].append(time.time() - t0)
227         # Only evaluate at pixels where LK produced an estimate
228         lk_valid = valid & (np.abs(lk_flow[:, :, 0]) + np.abs(lk_flow
[:, :, 1]) > 0)
229         if lk_valid.sum() > 0:
230             methods['Lucas-Kanade']['epe'].append(compute_epe(lk_flow,
flow_gt, lk_valid))
231             methods['Lucas-Kanade']['f1'].append(compute_f1(lk_flow,
flow_gt, lk_valid))
232
233         # --- Farneback ---
234         t0 = time.time()
235         fb_flow = run_farneback(img1, img2)
236         methods['Farneback']['time'].append(time.time() - t0)
237         methods['Farneback']['epe'].append(compute_epe(fb_flow, flow_gt,
valid))

```

```

238     methods['Farneback']['f1'].append(compute_f1(fb_flow, flow_gt,
valid))
239
240     # --- RAFT ---
241     t0 = time.time()
242     raft_flow = run_raft(img1, img2, raft_model, device)
243     methods['RAFT']['time'].append(time.time() - t0)
244     methods['RAFT']['epe'].append(compute_epe(raft_flow, flow_gt,
valid))
245     methods['RAFT']['f1'].append(compute_f1(raft_flow, flow_gt, valid)
)
246
247     # Aggregate results
248     results = {}
249     for method, data in methods.items():
250         results[method] = {
251             'epe_mean': np.mean(data['epe']),
252             'epe_std': np.std(data['epe']),
253             'f1_mean': np.mean(data['f1']),
254             'f1_std': np.std(data['f1']),
255             'time_mean': np.mean(data['time']),
256         }
257
258     return results
259
260
261 def print_results_table(results: Dict):
262     """Print a formatted comparison table suitable for a research report."
263     """
264     print("\n" + "="*65)
265     print(f"{'Method':<18} {'EPE (px)':<14} {'F1 (%)':>10} {'Time (s)
':>10}")
266     print("-"*65)
267     for method, r in results.items():
268         print(f"{method:<18} "
269               f"{r['epe_mean']:.2f} +/- {r['epe_std']:.2f} "
270               f"{r['f1_mean']:>6.1f} +/- {r['f1_std']:.1f} "
271               f"{r['time_mean']:>8.3f}s")
272     print("="*65)
273
274 def visualize_comparison(
275     img1: np.ndarray,
276     img2: np.ndarray,
277     lk_flow: np.ndarray,
278     fb_flow: np.ndarray,
279     raft_flow: np.ndarray,
280     flow_gt: np.ndarray,
281     valid: np.ndarray,
282     save_path: str = 'comparison.png'
283 ):

```

```

284     """
285     Side-by-side qualitative comparison: GT, LK, Farneback, RAFT, error
maps.
286     """
287     def to_hsv(flow, valid_mask=None):
288         u, v = flow[:, :, 0], flow[:, :, 1]
289         mag, ang = cv2.cartToPolar(u, v)
290         hsv = np.zeros((*u.shape, 3), dtype=np.float32)
291         hsv[:, :, 0] = ang * 180 / np.pi / 2
292         hsv[:, :, 1] = 1.0
293         max_m = np.percentile(mag, 99) + 1e-6
294         hsv[:, :, 2] = np.clip(mag / max_m, 0, 1)
295         if valid_mask is not None:
296             hsv[:, :, 2][~valid_mask] = 0
297         return cv2.cvtColor((hsv * [180, 255, 255]).astype(np.uint8), cv2.
COLOR_HSV2RGB)
298
299     def error_map(pred, gt, valid):
300         epe = np.sqrt((pred[:, :, 0]-gt[:, :, 0])**2 + (pred[:, :, 1]-gt[:, :, 1]
**2)
301         epe_display = np.zeros_like(epe)
302         epe_display[valid] = np.clip(epe[valid], 0, 20) / 20.0 # clip at
20px for display
303         return epe_display
304
305     fig, axes = plt.subplots(3, 3, figsize=(18, 12))
306
307     # Row 1: input frames and GT
308     axes[0,0].imshow(img1); axes[0,0].set_title('Frame 1', fontsize=12)
309     axes[0,1].imshow(img2); axes[0,1].set_title('Frame 2', fontsize=12)
310     axes[0,2].imshow(to_hsv(flow_gt, valid))
311     axes[0,2].set_title('Ground Truth Flow\n(HSV, black=invalid)',
fontsize=12)
312
313     # Row 2: predicted flows
314     axes[1,0].imshow(to_hsv(lk_flow)); axes[1,0].set_title('Lucas-Kanade',
fontsize=12)
315     axes[1,1].imshow(to_hsv(fb_flow)); axes[1,1].set_title('Farneback',
fontsize=12)
316     axes[1,2].imshow(to_hsv(raft_flow)); axes[1,2].set_title('RAFT',
fontsize=12)
317
318     # Row 3: error maps (lower = better)
319     for ax, pred, name in zip(
320         axes[2],
321         [lk_flow, fb_flow, raft_flow],
322         ['LK Error', 'Farneback Error', 'RAFT Error']
323     ):
324         err = error_map(pred, flow_gt, valid)
325         im = ax.imshow(err, cmap='hot_r', vmin=0, vmax=1)
326         epe = compute_epe(pred, flow_gt, valid)

```



```

327     ax.set_title(f'{name}\nEPE={epe:.2f}px', fontsize=12)
328     plt.colorbar(im, ax=ax, label='Normalized EPE')
329
330     for ax in axes.flatten():
331         ax.axis('off')
332
333     plt.suptitle('Optical Flow Comparative Study: KITTI 2015', fontsize
334 =14, fontweight='bold')
335     plt.tight_layout()
336     plt.savefig(save_path, dpi=150, bbox_inches='tight')
337     plt.show()

```

Listing 13.1: Complete comparative evaluation pipeline: LK vs. Farneback vs. RAFT on KITTI

13.3 Writing Findings Like a Researcher

When you present your comparative study results, the difference between a project report and a research contribution lies in how you interpret the numbers. Here is a template for research-quality interpretation:

Structure of a research-quality result discussion:

1. **State what you observe:** “RAFT achieves EPE of X.XX pixels, representing a X% improvement over Farneback (Y.YY pixels) and a Z% improvement over Lucas-Kanade (W.WW pixels, measured only at tracked features).”
2. **Explain why (mechanistically):** “RAFT’s advantage is largest in regions with large displacements (> 10 pixels), where the Taylor approximation underlying Lucas-Kanade breaks down, and in texture-less regions where Farneback’s polynomial expansion finds insufficient gradient information. RAFT’s all-pairs correlation volume and iterative GRU refinement address both these failure modes explicitly.”
3. **Identify remaining failures:** “Even RAFT struggles at occlusion boundaries, where the ground truth flow is undefined or ambiguous (the occluded region has no true source in frame 1). This is visible as elevated EPE along moving object edges.”
4. **State the implication:** “These results suggest that the remaining performance gap between RAFT and human-level accuracy is primarily in occluded regions and at fine motion boundaries — areas where deeper geometric understanding (3D scene structure, object semantics) could improve performance.”

Table 13.1: Expected comparative results on KITTI 2015 (illustrative baseline values)

Method	EPE (px) ↓	F1-all (%) ↓	Runtime (s)	Dense?
Lucas-Kanade (sparse, corners only)	~8–15	~40–55%	0.01–0.1	No
Farneback (dense)	~6–12	~30–45%	0.1–0.5	Yes
RAFT (KITTI fine-tuned)	~1.5–2.5	~5–8%	0.3–0.5	Yes

Note: actual numbers depend on KITTI split, image resolution, and hardware. Always report your specific experimental setup.

Chapter 13 Summary: Comparative Study Pipeline

- A fair comparison requires: same data, same preprocessing, same metrics, and properly tuned parameters for each method.
- EPE (mean Euclidean error in pixels) and F1 (fraction of outlier pixels) are the standard metrics, computed only over valid GT pixels.
- Lucas-Kanade produces sparse flow (only at corners); Farneback and RAFT produce dense flow. Comparing them requires care.
- RAFT dramatically outperforms classical methods, especially for large displacements and texture-less regions.
- Research-quality result interpretation: state observations, explain mechanisms, identify remaining failures, state implications.

Connection to Your Background

The evaluation pipeline you build here — loading data, running inference, computing metrics, generating visualizations — is identical in structure to the evaluation scripts you would write for any other CV task (segmentation mIoU, detection mAP, ASR WER). Your experience designing evaluation pipelines for your ASR system (computing WER across different Nepali speakers and conditions) directly transfers here. The disciplined experimental design — same data split, same preprocessing, reproducible random seed — is universal to all empirical ML research.

Research Questions to Explore

1. **Failure case taxonomy:** After running your comparison, categorize the worst-performing samples for each method. Do RAFT's failure cases cluster around specific scene types (e.g., nighttime, rain, fast pedestrians)? How would you design a scene-stratified evaluation to measure performance per scene type?
2. **The speed-accuracy tradeoff:** RAFT with 12 iterations takes ~0.3–0.5s per pair. RAFT with 6 iterations is faster but less accurate. Plot the EPE vs. runtime curve for RAFT with 1, 2, 3, 4, 6, 8, 12 iterations. Where is the elbow point that gives the best speed-accuracy tradeoff for a real-time AV system?

3. **Cross-dataset generalization:** Run your comparison on both KITTI 2012 and KITTI 2015. Does the ranking of methods change between datasets? What does this tell you about the stability of the comparison and the overfitting risk for each method?

Chapter 14

Connecting to 3D Vision

*Every optical flow vector is a compressed 3D statement.
Learning to decompress it is the project of 3D vision.*

14.1 How Optical Flow Implies 3D Motion

We deferred this discussion until now, but it is the chapter that makes everything else meaningful in the context of your goal: entering a 3D vision research lab.

Recall the pinhole camera model. A 3D point $\mathbf{P} = (X, Y, Z)$ projects to image coordinates:

$$x = f_x \frac{X}{Z} + c_x, \quad y = f_y \frac{Y}{Z} + c_y \quad (14.1)$$

where (f_x, f_y) are focal lengths and (c_x, c_y) is the principal point. If $\mathbf{P}$ moves to $\mathbf{P} + \dot{\mathbf{P}}dt$ and the camera rotates by $\boldsymbol{\omega}dt$ and translates by $\mathbf{t}dt$, the resulting 2D flow is:

$$\begin{pmatrix} u \\ v \end{pmatrix} = \frac{1}{Z} \begin{pmatrix} f_x & 0 \\ 0 & f_y \end{pmatrix} (\dot{\mathbf{P}}_{xy} - \mathbf{t}_{xy}) + \text{rotation terms} \quad (14.2)$$

This equation is the Rosetta Stone of 3D vision: it relates 2D optical flow to 3D quantities (depth Z , 3D motion $\dot{\mathbf{P}}$, camera pose change). **If you can estimate optical flow robustly, you have access to 3D information.**

14.2 Scene Flow: The 3D Analog of Optical Flow

Scene flow is the 3D motion field of all visible points in a scene — the per-point 3D velocity vector $\dot{\mathbf{P}}(X, Y, Z)$ for every visible surface element. It is the 3D generalization of optical flow.

	Optical Flow	Scene Flow
Domain	2D image plane	3D world
Representation	(u, v) per pixel	(dX, dY, dZ) per 3D point
Input	Two RGB frames	Two RGB + depth, or stereo + time
Output	2D displacement field	3D velocity field
Applications	Video analysis	Autonomous driving, robotics
Challenge	Aperture problem	3D aperture problem + depth ambiguity

Optical flow is the 2D projection of scene flow. Methods that estimate scene flow directly can be more accurate because they exploit the additional constraints from 3D geometry.

14.3 Structure from Motion Connection

Structure from Motion (SfM) recovers the 3D structure of a scene and camera poses from a sequence of 2D images. Optical flow is the input to SfM:

1. Compute sparse optical flow (feature tracking via Lucas-Kanade or SIFT matching)
2. Use the flow (correspondences) to estimate the essential matrix $\mathbf{E}$ or fundamental matrix $\mathbf{F}$
3. Decompose $\mathbf{E}$ into rotation $\mathbf{R}$ and translation $\mathbf{t}$
4. Triangulate 3D points from matching correspondences

Every step in this pipeline depends on the quality of the optical flow / feature matching. RAFT’s robust dense correspondences would be a strong foundation for an SfM pipeline — especially for challenging scenes where traditional feature detectors (SIFT/ORB) fail.

14.4 Depth from Flow

If you know the camera’s intrinsic parameters and ego-motion, you can recover depth from dense optical flow via:

$$Z(\mathbf{x}) = \frac{f \cdot T}{\text{flow}_{\perp}(\mathbf{x})} \quad (14.3)$$

where T is the camera translation magnitude, f is the focal length, and $\text{flow}_{\perp}$ is the component of flow due to camera translation (after removing rotational component). This is the principle behind **monocular depth estimation** and **visual odometry**.

The connection to your AV project: visual odometry uses consecutive-frame optical flow to estimate the camera’s pose change (rotation and translation) without external sensors. This is more accurate than pure IMU or wheel odometry, especially for long-term drift-free localization.

14.5 What to Study Next

You have now built a complete bridge from 2D vision to 3D vision. Here is a specific curriculum for the 3D vision research you are pursuing:

1. **Disparity and stereo matching:** Optical flow between left and right frames of a stereo camera gives disparity, from which depth is directly recoverable. FlowNet-style architectures have been directly applied to stereo matching (DispNet).
2. **Monocular depth estimation:** Using flow across time (video) to self-supervisedly learn depth. The loss function involves flow-based image synthesis: $\|I_1 - W(I_2; \hat{\mathbf{f}})\|$. You already understand all the pieces.
3. **3D object detection:** Extending your AV project's 2D detection to 3D bounding boxes requires depth information (either from stereo, LIDAR, or monocular estimation).
4. **Scene flow estimation:** FlowNet3D (point cloud scene flow) and CamLiFlow (combined camera and LIDAR) are the natural extensions of RAFT to 3D.
5. **Neural Radiance Fields (NeRF) for dynamic scenes:** Dynamic NeRF methods use scene flow as a temporal warp field to model moving objects. RAFT-quality flow estimates could significantly improve dynamic NeRF quality.

Chapter 14 Summary: Connection to 3D Vision

- Optical flow is the 2D projection of 3D scene motion: Equation 14.2 relates (u, v) to depth Z , 3D motion $\dot{\mathbf{P}}$, and camera motion.
- **Scene flow** is the 3D generalization of optical flow: per-point 3D velocity vectors.
- **Structure from Motion** uses optical flow correspondences as input to reconstruct 3D scene and camera poses.
- **Depth from flow:** knowing camera motion and flow, you can compute depth for every pixel.
- Natural next steps: stereo matching, monocular depth, 3D detection, scene flow, dynamic NeRF.

Connection to Your Background

Your autonomous vehicle project dealt with 2D detection and localization. Optical flow is the technical bridge to the 3D version of that problem: instead of detecting where objects are in 2D images, you understand where they are in 3D space and how they are moving. Visual odometry (using flow to estimate camera pose) is the direct 3D extension of the ego-motion estimation you may have thought about in your AV project. The transition from your AV project to a 3D vision research lab is not a leap — it is a direct continuation.

Research Questions to Explore

1. **The depth ambiguity in monocular flow:** From Equation 14.3, depth is recoverable from flow *if* you know the camera translation T . But T is itself estimated from flow (visual odometry). This circularity means monocular methods must solve for depth and camera motion simultaneously. What are the mathematical conditions under which this joint estimation is uniquely solvable?
2. **Scene flow from RGB-D video:** With a depth camera (RGB-D sensor like Intel RealSense), you have both 2D flow (from optical flow) and 3D point positions (from depth). How would you combine these to estimate 3D scene flow more robustly than from either modality alone?
3. **Optical flow for dynamic NeRF:** Neural Radiance Fields model static scenes. For dynamic scenes, a common approach is to use a learned “deformation field” that maps from canonical coordinates to current-frame coordinates. This deformation field is essentially a 3D scene flow. How would 2D optical flow estimates (from RAFT) inform or supervise the learning of this 3D deformation field?

Chapter 15

Research Narrative and Future Directions

The most important part of a research application is not what you did — it is the story of why you did it and where it leads.

15.1 Presenting This Project in a Lab Application

You have now built something substantial: a principled journey from classical optical flow theory through state-of-the-art deep learning methods, with working implementations and a quantitative comparative study. But how you *present* this to a 3D vision research lab is as important as what you built.

The wrong framing: “I implemented Lucas-Kanade, Farneback, and RAFT, and compared them on KITTI.”

The right framing: “I designed this project as a principled bridge from my existing 2D vision experience (autonomous vehicles, OpenCV) to 3D vision research. By systematically studying optical flow — from the mathematical foundations of the Brightness Constancy Assumption through the architecture of FlowFormer — I developed a deep understanding of how motion estimation connects 2D perception to 3D structure. The project was organized around a specific research question: where and why do different motion estimation paradigms fail, and what architectural innovations address each failure mode? Along the way, I discovered a structural isomorphism between the Conformer architecture I built for Nepali ASR and FlowFormer’s CNN+Transformer design philosophy, which raised a new research question about the generality of this dual-stream approach across modalities.”

Key elements of the narrative:

- **A research question, not just a project:** “Where and why do motion estimation paradigms fail?”
- **A principled methodology:** Systematic study from foundations through frontier

- **A genuine discovery:** The Conformer–FlowFormer structural isomorphism
- **A clear connection to the lab’s work:** 3D vision, scene flow, autonomous driving
- **Future directions that show you have been thinking:** Specific next questions

15.2 Connecting Your Conformer Insight to FlowFormer

Here is how to articulate this connection in a research discussion:

“While implementing the Conformer architecture for Nepali ASR, I developed an intuition for when local processing (convolution for phoneme patterns) and global processing (attention for utterance context) need to be combined, and what that combination looks like architecturally. When I studied FlowFormer, I recognized the same structural tension — local texture matching (CNN feature extractor) and global consistency reasoning (Transformer cost volume encoder). But there is an important difference: in the Conformer, attention operates over a 1D temporal sequence of feature vectors; in FlowFormer, attention operates over a 2D spatial cost volume. This raises a research question I find genuinely interesting: is there a unified architectural framework for combining local and global processing that generalizes across modalities (audio, video, point clouds) and tasks (recognition, correspondence, flow)?”

This is a substantive, original research observation. It is not something you read anywhere — it is something you derived from doing both projects. This is exactly what research lab PIs are looking for in applicants.

15.3 Suggested Future Experiments

Based on your study, here are concrete experiments that could form the basis of a research contribution:

1. **Ablation of FlowFormer’s global attention:** Replace FlowFormer’s Transformer cost volume encoder with RAFT’s local GRU lookup, keeping everything else the same. How much accuracy do you lose? This isolates the value of global attention specifically.
2. **Transfer learning from Conformer to FlowFormer:** Can the Transformer weights pre-trained for sequential ASR provide a useful initialization for FlowFormer’s spatial attention? What is the domain adaptation challenge, and how would you address it?
3. **RAFT on thermal infrared frames:** Your AV project likely dealt with visible-light cameras. How does RAFT perform on thermal infrared frames (where the Brightness Constancy Assumption has different failure modes)? This tests generalization of learned flow representations to new domains.
4. **Unsupervised RAFT:** Replace RAFT’s supervised EPE loss with a photometric loss ($\|I_1 - W(I_2; \hat{\mathbf{f}})\|$). How much accuracy is lost? Where specifically does the unsupervised model fail (texture-less regions? large displacements?) and why?

5. **Scene flow via optical flow + depth:** Combine RAFT’s optical flow with a monocular depth network to estimate scene flow. Evaluate against LIDAR ground truth. This is a direct path from your optical flow study to 3D scene understanding.

15.4 Key Papers to Read Next

To continue from where this book ends, read these papers in order:

1. **RAFT paper** [?] — now that you understand the architecture from this book, read the actual paper to see how the authors present and evaluate it
2. **FlowFormer** [?] and **FlowFormer++** [?] — the conceptual depth you built in Chapter 11 is the foundation for reading these
3. **PWC-Net** [?] — an architecturally elegant paper worth reading directly
4. **FlowNet 2.0** [?] — introduces the training curriculum (FlyingChairs → FlyingThings3D → Sintel → KITTI) that all subsequent methods follow
5. **CamLiFlow** — combines camera (RGB optical flow) with LIDAR (3D scene flow) — the natural 3D extension of your work
6. **DROID-SLAM** (Teed and Deng, 2021) — applies RAFT-style iterative GRU updates to simultaneous localization and mapping (SLAM). The same first authors as RAFT.
7. **Depth Anything** (Yang et al., 2024) — the current state-of-the-art in monocular depth estimation, which is the depth complement to optical flow

15.5 Key Researchers in Optical Flow

Building your research network starts with knowing who to follow:

- **Zachary Teed and Jia Deng** — RAFT, DROID-SLAM (Princeton)
- **Deqing Sun** — PWC-Net, FlowNet research (Google Research)
- **Zhaoyang Huang and Hongsheng Li** — FlowFormer, FlowFormer++ (CUHK MMLab)
- **Thomas Brox** — FlowNet, foundational work on variational methods (Freiburg)
- **Michael J. Black** — Foundational optical flow research, robust methods (MPI Intelligent Systems)
- **Andreas Geiger** — KITTI dataset, autonomous driving research (Tübingen)

15.6 How This Project Fits into the Broader CV Research Landscape

Optical flow occupies a unique position in computer vision research: it is both a *fundamental problem* (motion estimation is required for any temporal understanding) and a *technical benchmark* (KITTI and Sintel provide clean, quantitative evaluation). Methods developed for optical flow have driven broader progress:

- The correlation layer (FlowNet) influenced cost-volume-based stereo matching
- The encoder-decoder with skip connections (FlowNet) influenced semantic segmentation (U-Net was contemporaneous)
- The iterative GRU refinement (RAFT) was directly adopted by DROID-SLAM and 3D matching networks
- The Transformer cost volume encoding (FlowFormer) is influencing scene flow, depth estimation, and 3D reconstruction

By understanding optical flow at research depth, you have not just learned one task — you have learned a set of architectural and algorithmic principles that generalize across video understanding, 3D reconstruction, and autonomous driving perception. This is the most important thing you can communicate in your lab application.

Chapter 15 Summary

- Present this project not as “I implemented X, Y, Z” but as a principled study of a research question with a genuine discovery.
- The Conformer–FlowFormer structural isomorphism is an original, substantive research insight you can discuss in depth.
- Five concrete future experiments are identified, ranging from ablation studies to cross-modal transfer.
- Key papers to read: RAFT, FlowFormer, FlowFormer++, PWC-Net, FlowNet 2.0, DROID-SLAM, Depth Anything.
- Optical flow research has driven broader CV progress: its methods generalize to stereo, depth, SLAM, and 3D reconstruction.

Connection to Your Background

Every connection traced in this book — from your AV project’s OpenCV code to the mathematical foundations of LK, from your ResNet implementation to RAFT’s residual feature encoder, from your GRU/LSTM from scratch to RAFT’s ConvGRU, from your Conformer to FlowFormer — was designed to make you feel that your existing work is not a collection of separate projects but a coherent research trajectory. That trajectory has a natural next step: 3D vision, scene flow, and the deeper question of how temporal and spatial reasoning can be unified in a single learned architecture. You are ready for that step.

Research Questions to Explore

1. **The unification question:** Optical flow (2D motion), stereo matching (2D disparity), scene flow (3D motion), and monocular depth (3D structure) are all instances of the same fundamental problem: finding dense correspondences between two representations of a scene. Is there a single unified architecture that solves all four tasks, sharing learned features across them? What would be the advantages and challenges of such a unified model?
2. **Foundation models for flow:** GPT, DINO, and SAM are foundation models that learn general representations from massive unlabeled data and transfer to downstream tasks. Can we build a “FlowGPT” pre-trained on internet video at scale? What self-supervised objective would be appropriate for learning optical flow representations from unlabeled video?
3. **From optical flow to action understanding:** Human action recognition, video captioning, and activity prediction all require understanding motion. RAFT gives us robust pixel-level motion. How would you build a system that goes from pixel-level motion (RAFT) to object-level motion (tracking) to action-level understanding (“person is running”)? What are the key architectural challenges at each abstraction level?

Appendix A

Quick Reference: OpenCV Optical Flow Functions

This appendix is a practical cheat sheet for the OpenCV functions you will use most often.

Function	Method	Key Parameters
<code>cv2.goodFeaturesToTrack()</code>	Shi-Tomasi corner detection	<code>maxCorners</code> , <code>qualityLevel</code> , <code>minDistance</code>
<code>cv2.calcOpticalFlowPyrLK()</code>	Pyramidal Lucas-Kanade	<code>winSize</code> , <code>maxLevel</code> , <code>criteria</code>
<code>cv2.calcOpticalFlowFarneback()</code>	Dense Farneback	<code>pyr_scale</code> , <code>levels</code> , <code>winsize</code> , <code>poly_n</code>
<code>cv2.cartToPolar()</code>	$(u, v) \rightarrow (\text{mag}, \text{angle})$	For HSV visualization
<code>cv2.COLOR_HSV2BGR</code>	HSV to BGR conversion	After encoding flow as HSV

Appendix B

Training Curriculum Reference

The standard training curriculum for deep flow methods:

1. Pre-train on **FlyingChairs** (100 epochs, batch size 8): learn basic feature matching from simple synthetic data.
2. Fine-tune on **FlyingThings3D** (100K steps): adapt to more complex 3D rendered scenes.
3. Fine-tune on **MPI-Sintel** (50K steps, if targeting Sintel benchmark): adapt to specific rendering characteristics.
4. Fine-tune on **KITTI 2015** (50K steps): adapt to real-world driving data.

Note: Steps 3 and 4 are typically done separately for the respective benchmarks (you cannot fine-tune on Sintel and then expect the best KITTI results, as the domains differ).

Appendix C

Evaluation Checklist for Your Comparative Study

Before submitting or presenting your results, verify:

- ☐ Same image preprocessing for all methods (no method-specific contrast enhancement)
- ☐ Flow is in pixels (not normalized), matching GT units
- ☐ EPE computed only over valid GT pixels (KITTI sparse GT)
- ☐ F1 threshold: $EPE > 3$ px AND $EPE > 5\%$ of GT magnitude
- ☐ Random seed fixed for any stochastic augmentation
- ☐ Runtime measured on same hardware, averaged over ≥ 10 samples
- ☐ Classical method parameters tuned to recommended values (not tuned to look worse)
- ☐ At least 5 qualitative examples shown, including failure cases
- ☐ Results table includes mean and standard deviation across the evaluation set